

Halide for Qualcomm Hexagon Vector Extensions (HVX) User Guide

User Guide

80-PD002-1 G

July 8, 2024

Contents

1	Getting started	3
1.1	Download the Hexagon SDK	3
1.2	Examples of running Halide	3
1.3	Release notes for Version 2.6	4
1.3.1	Support for Windows on Snapdragon (WoS)	4
1.3.2	Support for HVX v69, v73, v75 and v79	4
1.3.3	Deprecate hvx_v66 and hvx_v68 target features and make v68 minimum supported version	4
1.3.4	Support TURBO_L2, TURBO_L3, MAX power modes for HVX	4
1.3.5	Migration to Multi-Domain RPC in Halide Runtime	5
1.3.6	Security Version for Halide Runtime Libraries	5
1.3.7	Transition to RPCMEM library for Ion-Allocation	5
1.3.8	System Requirements	5
2	The Halide programming language	6
2.1	Algorithm	7
2.1.1	Func	8
2.1.2	Var	8
2.1.3	Expr	9
2.1.4	Pure and Update definitions	9
2.1.4.1	RDom	10
2.2	Schedule	10
2.2.1	hexagon	12
2.2.2	reorder	12
2.2.3	split	13
2.2.4	vectorize	14
2.2.5	unroll	14
2.2.6	parallel	14
2.2.7	fuse	15
2.2.8	tile	15
2.2.9	prefetch	16
2.2.10	compute_root	17
2.2.11	compute_at	19
2.2.12	store_root	20
2.2.13	store_at	21
2.2.14	hexagon_user_dma	22
2.2.15	align_storage	22
2.2.16	TailStrategy	23
2.3	Memory	23

2.3.1	External Buffers	24
2.3.2	Internal Buffers	24
2.3.3	Device Interface	24
2.4	Ahead-of-time compilation	24
2.4.1	Generators	25
3	Halide for HVX	29
3.1	Halide execution modes	30
3.1.1	Offload mode	30
3.1.2	Standalone mode	31
3.2	Relevant target features for HVX	31
3.2.1	hvx	31
3.2.2	hvx_v66	31
3.2.3	hvx_v68	31
3.2.3.1	User DMA	31
3.2.3.2	Vector floating point	32
3.2.4	hexagon_dma	33
3.2.5	profile	33
3.2.6	debug	33
3.2.7	trace_realizations	33
3.2.8	trace_pipeline	33
3.3	Performance on HVX	33
3.3.1	Alignment	33
3.3.1.1	Alignment of external buffers	34
3.3.1.2	Alignment of internal buffers	35
3.3.1.3	Align when splitting dimensions	35
3.3.2	Memory locality	36
3.3.2.1	Tiling	36
3.3.2.2	Line buffering	37
3.3.3	Zero-copy buffers	38
3.3.3.1	rpcmem_alloc and rpcmem_free	39
3.3.4	VTM and scatter-gather operations	40
3.3.4.1	Internal (intermediate) buffers	40
3.3.4.2	Scatter-gather operations	40
3.3.4.3	Scatter operations on 8-bit data	42
3.3.5	Power and performance APIs	43
3.3.6	Profiling	44
3.3.6.1	Target feature for profiling	44
3.3.6.2	Use hexagon-profiler for profiling	46
3.3.6.3	Use Instrumented Trace (ITRACE) for profiling	46
3.4	Halide HVX built-ins	47
3.4.1	Supported Halide HVX built-ins	48
3.4.2	Usage and examples for HVX built-ins	55
3.5	Troubleshooting	97
3.5.1	User controllable exits	98
3.5.2	Target malloc tracing	98
3.5.3	Set heap grow size	100
3.5.4	Debug target feature	100
3.5.5	Useful environment variables	101
3.5.6	Common compile time or runtime errors	103

3.5.6.1	Crashed thread due to TLBMISS X (execution)	103
3.5.6.2	Accessing buffers out of bounds and violating constraints	105
3.6	Signing for deployment	107
3.6.1	Tools for signing Offload mode pipelines	107
3.6.2	Unsigned mode execution	108
4	Autoscheduler	110
4.1	Use the Autoscheduler	110
4.1.1	Set up a generator for autoscheduling	111
4.1.2	User-specified offloading	111
4.1.3	Call the pipeline generator with the Autoscheduler	112
4.1.4	Environment variables	113
4.2	Autotuning	113
4.3	Schedule files	115
4.4	Autoscheduler examples	115
5	Halide with UBWC DMA	117
5.1	UBWC DMA basics	117
5.1.1	Supported data formats	118
5.1.2	Supported Snapdragon devices and Hexagon local memory	119
5.2	Program UBWC DMA with Halide	119
5.2.1	UBWC DMA with Halide runtime model	119
5.2.2	Halide device interface for UBWC DMA	120
5.2.3	Scheduling	121
5.2.4	De-interleaving of UV for 4:2:0	122
5.2.5	Known limitations	122
5.3	Performance considerations	122
5.3.1	Memory locality	122
5.3.2	Power API for UBWC DMA	123
5.3.3	UBWC DMA concurrency	123
5.4	Examples	124
6	Python bindings (Linux only)	125
6.1	Dependencies and setup	125
6.2	Defining a Halide pipeline in Python	125
6.2.1	Generators in Python	126
6.3	Differences between Python bindings and the C++ API	128
6.3.1	Enhancements to the C++ API	129
6.4	Halide Python examples	129
7	Halide examples	130
7.1	Setup	130
7.1.1	Device setup	131
7.2	Run examples	131
7.2.1	Run only a single execution mode	132
8	Indices and tables	133

1 Getting started

To use Halide for HVX, you need the Qualcomm® Hexagon™ Software Development Kit (SDK) in addition to the Halide installer. Follow these steps to download and install the Hexagon SDK.

1.1 Download the Hexagon SDK

1. Visit <https://developer.qualcomm.com/software/hexagon-dsp-sdk/tools>.
2. Download and install **Qualcomm Package Manager** for Linux.
3. Run QPM, and on the top tool bar, click **All**.
4. Scroll down and click the **Qualcomm Hexagon SDK Product Suite**.
5. Click **Hexagon SDK 6.x**.
6. Select a version, at least **6.0.0.0**.
7. Click the pull-down icon next to the version number and select **Install**.
8. Accept the terms of the license agreement.
9. Click **Customize**.
10. Accept or specify the SDK install directory.
11. Click **Next**.
12. Select **Full Android NDK** and **Eclipse**, and de-select **Minimal Android NDK**.
13. Click **Next**.
14. Proceed with the installation.

Note: If you do not have Android NDK r25c installed, choose the option to do a full install so it will also be installed.

1.2 Examples of running Halide

See [Halide examples](#).

1.3 Release notes for Version 2.6

Following are the changes made in this release (Qualcomm® Halide 2.6) in comparison to the previous major release (Qualcomm® Halide 2.5.0).

1.3.1 Support for Windows on Snapdragon (WoS)

This release adds support for cross-compiling Halide examples for Windows on Snapdragon (WoS) in device standalone mode. For Halide windows installer the individual scripts are present in device standalone mode examples `conv3x3`, `dilate3x3`, `gaussian5x5`, `median3x3`, `morph_edge`, `sobel` examples with the name `test-<example>-wos.cmd` which uses the central simplified `test-wos.cmd` script. The `test-wos.cmd` script present in `%HALIDE_ROOT%\Examples\standalone\device\apps` can be used by any user defined device standalone example if `TEST` variable is set to respective example name.

1.3.2 Support for HVX v69, v73, v75 and v79

This release adds support for HVX versions 69, 73, 75, and 79. Use the `hvx-hvx_v69`, `hvx-hvx_v73`, `hvx-hvx_v75`, and `hvx-hvx_v79` target features to build for their respective versions. The `Q6_VERSION` must be set to the respective HVX version for which it is building. For example, for `hvx-hvx_v75` the `Q6_VERSION` must be set to 75.

1.3.3 Deprecate hvx_v66 and hvx_v68 target features and make v68 minimum supported version

This release deprecates the `hvx_v66` and `hvx_v68` target features. The default ISA is now HVX v68. Using the `hvx` target feature now instructs the Halide compiler to generate code for HVX v68 ISA. See the examples provided with this release for correct usage, both in C++ and in the build system.

1.3.4 Support TURBO_L2, TURBO_L3, MAX power modes for HVX

This release adds support for new DCVS target voltage corners called `HAP_DCVS_VCORNER_TURBO_L2`, `HAP_DCVS_VCORNER_TURBO_L3` and `HAP_DCVS_VCORNER_MAX`. These voltage corners can be voted for by using the `halide_hexagon_power_mode_t` values of `halide_hexagon_power_turbo_l2`, `halide_hexagon_power_turbo_l3` and `halide_hexagon_power_max` respectively when calling `halide_hexagon_set_performance_mode`. On Snapdragon chipsets SM8650 and later, `halide_power_mode_turbo_l2`, `halide_power_mode_turbo_l3` and `halide_power_mode_max` will result in a vote for `HAP_DCVS_VCORNER_TURBO_L2`, `HAP_DCVS_VCORNER_TURBO_L3` and `HAP_DCVS_VCORNER_MAX` voltage corners respectively. On older Snapdragon chipsets where these voltage corners are not supported, Halide will ensure a vote for a lower supported voltage corner. The use of `halide_power_mode_max` will result in a vote for the voltage corner `HAP_DCVS_VCORNER_MAX` on all Snapdragon chipsets.

1.3.5 Migration to Multi-Domain RPC in Halide Runtime

This release migrates the Halide runtime from non-domain RPC to multi-domain RPC, enhancing support for multi-DSP applications and frameworks in Halide.

1.3.6 Security Version for Halide Runtime Libraries

This release adds versioning to libhalide_remote_skel.so, the shared runtime (libhalide_shared_runtime.so), and kernel code (libhalide_hexagon_code.so) generated by Halide. This allows OEMs to whitelist or blacklist libraries in the signed DSP PD.

1.3.7 Transition to RPCMEM library for Ion-Allocation

This release uses RPCMem APIs instead of DMABUFHeap APIs for host allocations, while ensuring no external impact to users.

1.3.8 System Requirements

Minimum system requirements are as follows

- Clang 7.0 with libc++ as the native (x86) compiler and C++ runtime library respectively.
- Ubuntu 20.04
- SDK 6.0.0.2 or higher.

Note: SDK 6.0.0.2 contains an updated QAIC IDL compiler that includes important security fixes. Note that while these fixes are recommended for all devices, on SM8650 and newer devices the QAIC generated skel shared objects will also be version checked when run. On SM8650 and newer devices, only skel objects that have been built with all the QAIC security fixes are allowed to run on the DSP. As a general rule of thumb, if a QAIC IDL compiler version mismatch is reported in the DSP logs, please upgrade to the latest Hexagon SDK available.

2 The Halide programming language

- *Algorithm*
 - *Func*
 - *Var*
 - *Expr*
 - *Pure and Update definitions*
- *Schedule*
 - *hexagon*
 - *reorder*
 - *split*
 - *vectorize*
 - *unroll*
 - *parallel*
 - *fuse*
 - *tile*
 - *prefetch*
 - *compute_root*
 - *compute_at*
 - *store_root*
 - *store_at*
 - *hexagon_user_dma*
 - *align_storage*
 - *TailStrategy*
- *Memory*
 - *External Buffers*
 - *Internal Buffers*
 - *Device Interface*

- *Ahead-of-time compilation*
 - *Generators*

Halide is a domain-specific language for vision and image processing algorithms, which allow you to author efficient image processing pipelines. Halide separates the program into two conceptual parts:

- **Algorithm** – Defines what is computed at each pixel
- **Schedule** – Defines how the computation should be organized

Every Halide program must consist of both parts, which you are to specify. Together, they are called the *pipeline*.

Note: This section is only a brief introduction to the Halide programming language. It is not intended to be a complete guide to the Halide programming language. More resources for learning the language can be found at <https://halide-lang.org>

The following example shows a pipeline that is blurring an image in the horizontal dimension.

```

1 int main() {
2     // Image with 8 bits per pixel.
3     ImageParam input(UInt(8), 2)
4     Halide::Func f;
5     // Algorithm.
6     f(x, y) = (input(x-1, y) + input(x, y) + input(x+1, y))/3;
7     // Schedule
8     f.vectorize(x, 128).parallel(y, 16);
9 }
```

The algorithm concisely defines the `blur` computation in terms of the `x` and `y` coordinates of a pixel. The schedule directs how the computation should be conducted. This example directs Halide to vectorize the `blur` algorithm by a factor of 128 pixels, parallelize the algorithm by a factor of 16 rows, and launch it on the Hexagon cDSP.

The Halide HVX compiler automatically translates your high-level image algorithm into efficient HVX object code.

2.1 Algorithm

The algorithm defines what is computed at each pixel. Following are the constructs that Halide provides to write an algorithm.

2.1.1 Func

A `Func` represents a pipeline stage. It describes what value a pixel should have.

A `Func` is backed by a memory buffer, which can be thought of as an image (Halide pipelines pass buffers as `halide_buffer_t` objects; see [Memory](#)). In the following example, `f` is a `Func` that is used to compute an image where every pixel is the sum of its `x` and `y` coordinates.

```
f(x, y) = x + y;
```

A `Func` can be an input to another `Func`. In this case, an image processing pipeline can be constructed by cascading multiple `Func`s (where each is a pipeline stage) in producer-consumer relationships.

The following example is a Gaussian Blur filter represented in Halide.

```
ImageParam input(UInt(8), 2)
Func bounded_input{"bounded_input"};
Func input_16{"input_16"};
Func rows{"rows"};
Func cols{"cols"};
Func output{"output"};

bounded_input(x, y) = BoundaryConditions::repeat_edge(input)(x, y);
input_16(x, y) = cast<int16_t>(bounded_input(x, y));
rows(x, y) = input_16(x, y-2) + 4*input_16(x, y-1) + 6*input_16(x, y) +
    4*input_16(x, y+1) + input_16(x, y+2);
cols(x, y) = rows(x-2, y) + 4*rows(x-1, y) + 6*rows(x, y) + 4*rows(x+1,
    y) + rows(x+2, y);
output(x, y) = cast<uint8_t>(cols(x, y) >> 8);
```

In this example:

- `bounded_input`, `input_16`, `rows`, `cols`, and `output` are all `Func`s, each representing a stage of this 5x5 Gaussian Blur filter.
- `output` is the final stage of the pipeline, and the memory buffer that is associated with `output` contains the blurred image.

2.1.2 Var

A `Var` is a name to use as a variable when defining a `Func`. `Var` instances are used to refer to the dimensions of a `Func`, although that is not their only use. For example, a `Func` can be defined using a `Var`, as shown in the following example.

```
blur_x(x, y) = (input(x-1, y) + input(x, y) + input(x+1, y))/3;
```

Tip: It is important to understand that `x` and `y` have no meaning on their own. However, when used with `blur_x` and `input`, they signify the two dimensions of these two `Func`s.

2.1.3 Expr

An Expr in Halide is composed of Funcs, Vars, and other Exprs. Following are some examples.

```
Expr e = x + y;
Output(x, y) = 3 * e + x;
```

In these examples, x and y are Vars, and Output is a Func.

2.1.4 Pure and Update definitions

A Func can be defined in multiple passes. The first definition is a simple definition for all the points in the domain of the Func. This is called the Pure definition of the Func

For example, following is a Pure definition of f.

```
Func f{"f"};
Var x{"x"}, y{"y"};
f(x, y) = x + y;
```

This example is simply a mapping from Vars (x and y) to an Expr. However, later definitions can include computed expressions on both sides of the assignment.

```
f(3, 7) = 42;
```

These definitions are called Update definitions of a Func. A special case of Update definitions is a class of definitions called Reduction definitions. They are Update definitions that recursively refer to the value of Func at the same site.

```
f(x, y) = f(x, y) * 10;
```

You can confine the update to a single row or a single column.

```
// Updates restricted to a row allow us to refer to
// other values in the same column (x)
f(x, 1) = f(x, 2) - f(x, 0);

// Updates restricted to a column allow us to refer to
// other values in the same row (y)
f(1, y) = f(2, y) - f(0, y);
```

Pure definitions and Update definitions can be scheduled independently of each other. To schedule Update definitions, use `Func::update(int)` to get a handle to an update step. This is shown in the following example for the Pure definition and the two update steps in the previous example.

```
// Schedule pure definition
f.vectorize(x, 128);
// Schedule first update definition that works on one
// row.
f.update(0).vectorize(x, 128);
// Schedule second update definition that works on one
```

(continues on next page)

(continued from previous page)

```
// column.
f.update(1).parallel(y);
```

2.1.4.1 RDom

An `RDom` is short for a reduction domain, and it is a handy way to write updates in a loop. Consider the following Pure definition of a `Func`:

```
f(x, y) = x + y;
```

To square the first ten rows requires ten `Update` definitions, as shown in the following example.

```
f(x, 0) = f(x, 0) * f(x, 0);
f(x, 1) = f(x, 1) * f(x, 1);
f(x, 2) = f(x, 2) * f(x, 2);
f(x, 3) = f(x, 3) * f(x, 3);
f(x, 4) = f(x, 4) * f(x, 4);
f(x, 5) = f(x, 5) * f(x, 5);
f(x, 6) = f(x, 6) * f(x, 6);
f(x, 7) = f(x, 7) * f(x, 7);
f(x, 8) = f(x, 8) * f(x, 8);
f(x, 9) = f(x, 9) * f(x, 9);
```

However, it would be much easier to put this in a loop that can be generated for you. Use an `RDom` to rewrite the previous example as follows.

```
RDom r(0, 10);
f(x, r) = f(x, r) * f(x, r);
```

`RDom` can also be used over multiple dimensions.

```
ImageParam input(type_of<uint8_t>(), 2);
Func histogram{"histogram"};
RDom r(0, input.width(), 0, input.height());

// Pure definition.
histogram(x, y) = 0;

// Update definition - a reduction definition
histogram(input(r.x, r.y)) += 1;
```

2.2 Schedule

The schedule is the other integral part of a Halide program that specifies how the computation of the algorithm is to be structured. It entails the specification of storage allocation for, and the relative order of, computation of the different stages (`Func`) of the algorithm with each other. It also specifies how each stage is to be computed.

The following example shows a schedule for the 5x5 Gaussian Blur shown in the previous example.

```

1 ImageParam input(UInt(8), 2)
2 Func bounded_input{"bounded_input"};
3 Func input_16{"input_16"};
4 Func rows{"rows"};
5 Func cols{"cols"};
6 Func output{"output"};
7
8 bounded_input(x, y) = BoundaryConditions::repeat_edge(input)(x, y);
9 input_16(x, y) = cast<int16_t>(bounded_input(x, y));
10 input_16(x, y) = cast<int16_t>(input(x, y));
11 rows(x, y) = input_16(x, y-2) + 4*input_16(x, y-1) + 6*input_16(x, y) +
    ↪ 4*input_16(x, y+1) + input_16(x, y+2);
12 cols(x, y) = rows(x-2, y) + 4*rows(x-1, y) + 6*rows(x, y) + 4*rows(x+1,
    ↪ y) + rows(x+2, y);
13 output(x, y) = cast<uint8_t>(cols(x, y) >> 8);
14
15 output.compute_root()
16     .parallel(y, 16)
17     .vectorize(x, 128);
18 rows.compute_at(op, y);

```

The schedule in the example specifies that the output stage (Output) is to be computed by vectorizing the x dimension while the rows (y dimension) are executed in parallel on multiple threads (each thread works on 16 rows of the output). Further, it says that the Func rows are computed in the y loop of Output. The following example shows the equivalent pseudo C/C++ code for this schedule.

```

// parallel loop over Output.y
for<parallel> (Output.y = Output.y.min; y < Output.y.extent/16;
    ↪ ++Output.y) {

    // Compute the Func rows here, in the 'y' loop of Output
    for (rows.y = rows.y.min; rows.y < rows.y.extent; ++rows.y) {
        for (rows.x = rows.x.min; rows.x < rows.x.extent; ++rows.x) {
            rows(rows.x, rows.y) = ...;
        }
    }

    for (Output.y_inner = 0; y_inner < 16; ++Output.y_inner) {
        for (Output.x = Output.x.min; Output.x < (Output.x.extent/128);
            ↪ ++Output.x) {
            for<vectorized_loop> (Output.x_inner = Output.x*128; Output.x_
            ↪ inner < Output.x*128 + 128; ++Output.x_inner) {
                op(Output.x_inner, Output.y_inner) = ...;
            }
        }
    }
}

```

Note: The other stages of the pipeline are not present by name in the pseudo code. By default, if a schedule is not specified for a `Func`, it is computed inline in its consumer.

2.2.1 hexagon

```
hexagon()
```

Halide can offload parts of a pipeline to the DSP with the `.hexagon` scheduling directive. This directive is used in the two Offload modes (see [Halide execution modes](#)). However, this scheduling directive requires the `hvx` target feature to be used when compiling the pipeline. In the following example, the `Func`, output, and therefore the entire pipeline, are offloaded to the DSP and vectorized for HVX.

```
// Algorithm
bounded_input(x, y) = BoundaryConditions::repeat_edge(input)(x, y);
input_16(x, y) = cast<int16_t>(bounded_input(x, y));
input_16(x, y) = cast<int16_t>(input(x, y));
rows(x, y) = input_16(x, y-2) + 4*input_16(x, y-1) + 6*input_16(x, y) +
    ↪ 4*input_16(x, y+1) + input_16(x, y+2);
cols(x, y) = rows(x-2, y) + 4*rows(x-1, y) + 6*rows(x, y) + 4*rows(x+1,
    ↪ y) + rows(x+2, y);
output(x, y) = cast<uint8_t>(cols(x, y) >> 8);

// Schedule
output.hexagon().vectorize(x, 128);
```

2.2.2 reorder

```
reorder(dim_innermost, ... , dim_outermost)
// dim_innermost : Innermost dimension in the new reordering
// dim_outermost : Outermost dimension in the new reordering
```

The `reorder` directive reorders the dimensions of a `Func` in the order of the dimensions as specified. This reordering leads to the reordering of the loops of the loop nest computing that `Func`. The dimensions are specified from the innermost loop going outwards.

The following example of a `Func` simply adds 10 to each element (or pixel) of its input.

```
f(x, y) = input(x, y) + 10;
```

Halide generates the corresponding pseudo loop nest.

```
for (f.y = f.y.min; f.y < f.y.extent; ++f.y) {
    for (f.x = f.x.min; f.x < f.x.extent; ++f.x) {
        f(f.x, f.y) = input(f.x, f.y) + 10;
    }
}
```

Now, reorder it as follows:

```
f(x, y) = input(x, y) + 10;
f.reorder(y, x); // inner to outer, left to right.
```

Halide generates the corresponding pseudo loop nest.

```
for (f.x = f.x.min; f.x < f.x.extent; ++f.x) {
  for (f.y = f.y.min; f.y < f.y.extent; ++f.y) {
    f(f.x, f.y) = input(f.x, f.y) + 10;
  }
}
```

2.2.3 split

```
split(old_dim, new_outer_dim, new_inner_dim, factor, tail_strategy);
// old_dim           : dimension to be split
// new_outer_dim     : name of the new outer dimension
// new_inner_dim     : name of the new inner dimension
// factor            : factor to split old_dim by
// tail_strategy     : strategy to handle tail or remainder loop if factor_
↳ does not divide old_dim completely
```

The `split` directive splits a dimension by a given factor into an inner and outer dimension.

The inner dimension iterates from 0 to `factor-1`. This is useful because after the `split`, the inner and outer dimensions can be used with their own independent scheduling directives. For example, it is possible to vectorize the inner dimension while unrolling the outer dimension.

The following example of a `Func` simply adds 10 to each element (or pixel) of its input.

```
Var xi{"xi"}, xo{"xo"};
Expr factor;
f(x, y) = input(x, y) + 10;
f.split(x, xo, xi, factor);
```

Halide splits dimensions `x` and then generates the corresponding pseudo loop nest

```
for (f.y = f.y.min; f.y < f.y.extent; ++f.y) {
  for (f.xo = f.x.min; f.xo < (f.x.extent/factor); ++f.xo) {
    for (f.xi = 0; f.xi < factor; ++f.xi) {
      f.x = (f.xo * factor) + f.xi;
      f(f.x, f.y) = input(f.x, f.y) + 10;
    }
  }
}
```

2.2.4 vectorize

```
vectorize(dim)
vectorize(dim, factor)
vectorize(dim, factor, tailstrategy)
// dim           : dimension to be vectorized
// factor        : factor to split dim by for vectorization. In other_
↳ words, the vectorization factor (optional)
// tail_strategy : strategy to handle tail or remainder loop if factor_
↳ does not divide dim completely (optional)
```

The `vectorize` directive vectorizes the dimension, `dim`, by the vectorization factor. `factor` and `tailstrategy` are optional. If both are not specified, the entire `dim` is vectorized. If at least `factor` is specified, `dim` is split into two dimensions by `factor`. The outer dimension retains the name `dim` and the inner dimension remains unnamed. This unnamed inner dimension is vectorized completely.

2.2.5 unroll

```
unroll(dim)
unroll(dim, factor)
unroll(dim, factor, tailstrategy)
// dim           : dimension to be unrolled
// factor        : factor to split dim by for unrolling. In other_
↳ words, the unroll factor (optional)
// tail_strategy : strategy to handle tail or remainder loop if factor_
↳ does not divide dim completely (optional)
```

The `unroll` directive unrolls the dimension, `dim`, by the unroll factor, `factor`. `factor` and `tailstrategy` are optional. If both are not specified, the entire `dim` is unrolled. If at least `factor` is specified, `dim` is split into two dimensions by `factor`. The outer dimension retains the name `dim` and the inner dimension remains unnamed. This unnamed inner dimension is unrolled completely.

2.2.6 parallel

```
parallel(dim)
parallel(dim, task_size)
parallel(dim, task_size, tailstrategy)
// dim           : dimension whose iterations should be executed in_
↳ parallel
// task_size     : The size of each task in terms of number of_
↳ iterations of the loop in dim (optional)
// tail_strategy : strategy to handle tail or remainder loop if task_
↳ size does not divide dim completely (optional)
```

In the loop nest generated for a `Func`, the loop traversing the dimension, `dim`, is executed in parallel. If `task_size` is specified, `dim` is split into two dimensions by `task_size`. The outer dimension retains the name `dim` and the inner dimension remains unnamed.

The iterations of the outer dimension, `dim`, are executed in parallel. If `task_size` is not specified, each iteration of `dim` is executed in parallel, effectively resulting in a `task_size` of 1.

2.2.7 fuse

```
fuse(inner_dim, outer_dim, fused_dim)
// inner_dim      : Inner dimension to be fused
// outer_dim      : Outer dimension to be fused
// fused_dim      : The name of the resulting fused dimension
```

The `fuse` directive merges the two dimensions, `inner_dim` and `outer_dim`, into one dimension, `fused_dim`. Thus, `fused_dim` is the product of the two dimensions.

2.2.8 tile

```
tile(dim_0, dim_1, new_outer_dim_0, new_outer_dim_1, new_inner_dim_0,
    ↪ new_inner_dim_1, factor_dim_0, factor_dim_1)
tile(dim_0, dim_1, new_outer_dim_0, new_outer_dim_1, new_inner_dim_0,
    ↪ new_inner_dim_1, factor_dim_0, factor_dim_1, tailstrategy)
// dim_0          : First (inner) dimension to be split
// dim_1          : Second (outer) dimension to be split
// new_outer_dim_0 : The name of the new outer dimension created by
    ↪ splitting dim_0
// new_outer_dim_1 : The name of the new outer dimension created by
    ↪ splitting dim_1
// new_inner_dim_0 : The name of the new inner dimension created by
    ↪ splitting dim_0
// new_inner_dim_1 : The name of the new inner dimension created by
    ↪ splitting dim_1
// factor_dim_0    : split factor for splitting dim_0
// factor_dim_1    : split factor for splitting dim_1
// tailstrategy    : strategy to handle tail or remainder loop if
    ↪ either of factor_dim_0 and factor_dim_1 does not divide the
    ↪ corresponding dimension completely (optional)
```

The `tile` directive splits the two dimensions, `dim_0` and `dim_1`, and reorders their splits to create a tile of computation for the `Func`.

This directive is shorthand for a recurring pattern of combining the `split` and `reorder` directives as follows:

1. `dim_0` is split into two dimensions, `new_outer_dim_0` and `new_inner_dim_0`, by factor `factor_dim_0`.
2. `dim_1` is split into two dimensions, `new_outer_dim_1` and `new_inner_dim_1`, by factor `factor_dim_1`.
3. These dimensions are then reordered in the order (innermost to outermost): `new_inner_dim_0`, `new_inner_dim_1`, `new_outer_dim_0`, `new_outer_dim_1`.

These steps lead to a tiled pattern of execution of the loop nest. To understand this process, consider the same pipeline from the previous example where `f` is the result of adding 10 to each element of `input`.

```
Var xi{"xi"}, xo{"xo"};
```

(continues on next page)

(continued from previous page)

```

Var yi{"yi"}, yo{"yo"};
Expr xfactor, yfactor;
f(x, y) = input(x, y) + 10;
f.tile(x, y, xo, yo, xi, yi, xfactor, yfactor);

```

Halide generates the corresponding pseudo loop nest.

```

for (f.yo = f.y.min; f.yo < f.y.extent/yfactor; ++f.yo) {
  for (f.xo = f.x.min; f.xo < (f.x.extent/xfactor); ++f.xo) {
    for (f.yi = 0; f.yi < yfactor; ++f.yi) {
      f.y = (f.yo * yfactor) + f.yi;
      for (f.xi = 0; f.xi < factor; ++f.xi) {
        f.x = (f.xo * xfactor) + f.xi;
        f(f.x, f.y) = input(f.x, f.y) + 10;
      }
    }
  }
}

```

Note: Remember that a single loop is split, while a loop nest is tiled.

2.2.9 prefetch

```

prefetch(func_or_image, dim)
prefetch(func_or_image, dim, offset)
prefetch(func_or_image, dim, offset, strategy)
// func_or_image : Func or ImageParam to prefetch
// dim           : dimension to prefetch over
// offset        : iteration offset (optional)
// strategy      : PrefetchBoundStrategy for generated code (optional)

```

The `prefetch` directive is used to prefetch data for a `Func` or `ImageParam` (`func_or_image`). The directive inserts a prefetch to the L2 cache in the generated code. The data is prefetched from the buffer for `func_or_image`. The size of the prefetched data is the size of `func_or_image` required in the `dim` loop of the `Func` on which this directive was used. If `offset` is specified, the prefetch is issued `offset` iterations of the `dim` loop in advance. If `strategy` is not specified, the default prefetch is `PrefetchBoundStrategy::GuardWithIf`.

The available prefetch strategies are:

- `PrefetchBoundStrategy::GuardWithIf` – Guard the prefetch operation with an if statement to keep the prefetch within bounds.
- `PrefetchBoundStrategy::Clamp` – Clamp the computed bounds using minimum and maximum values to keep the prefetch within bounds.
- `PrefetchBoundStrategy::NonFaulting` – Do not guard or clamp the region to prefetch.

`PrefetchBoundStrategy::NonFaulting` assumes that prefetching out of bounds does not cause a

fault. A fault can be avoided if out-of-bound reads do not cross a page boundary, or if the memory buffer was allocated with sufficient padding to keep all accesses within allocated bounds.

Using the same pipeline as an example, wherein the `f` is the result of adding 10 to each element of `input`:

```
Func f{"f"};
Var x{"x"}, y{"y"};
f(x, y) = input(x, y) + 10;
f.prefetch(input, y, 2);
```

The prefetch directive will insert a prefetch to load data from `input` in the `y` loop in the loop nest computing `f`. It will prefetch into the L2 cache only as much data of `input` as is required in the `y` loop. Lastly, the data prefetched is the data required in iteration `y + 2`.

Halide generates the corresponding pseudo loop nest.

```
for (f.y = f.y.min; f.y < f.y.extent; ++f.y) {
  if (y+2 < input.y.extent) {
    // Prefetch only one row of input, because
    // for one row of f only one row of input
    // is needed
    prefetch(&input(0, y+2))
  }
  for (f.x = f.x.min; f.x < f.x.extent; ++f.x) {
    f(f.x, f.y) = input(f.x, f.y) + 10;
  }
}
```

2.2.10 compute_root

```
compute_root()
```

The `compute_root` directive indicates that the `Func` it was called on should be computed before all of its consumers.

Following is an example of the producer-consumer pipeline.

```
g(x, y) = x*y;
f(x, y) = g(x, y) + g(x, y+1) + g(x+1, y) + g(x+1, y+1);
```

If no schedule is specified, Halide generates the corresponding pseudo loop nest.

```
int height = f.y.extent;
int width = f.x.extent;
int f[height][width];
for (int y = 0; y < height; ++y) {
  for (int x = 0; x < width; ++x) {
    f[y][x] = x*y + x*(y+1) + (x+1)*y + (x+1)*(y+1);
  }
}
```

You do not see the computation of `g` in this example because `g` is now considered to have been computed *inline*.

Now, schedule `g` as shown in the following example.

```
g(x, y) = x*y;
f(x, y) = g(x, y) + g(x, y+1) + g(x+1, y) + g(x+1, y+1);
g.compute_root();
```

Following is the loop nest that Halide will generate.

```
// Compute all of g before f
int height = f.y.extent;
int width = f.x.extent;
int g[height+1][width+1];
for (int y = 0; y < height+1; ++y) {
    for (int x = 0; x < width+1; ++x) {
        g[y][x] = x*y;
    }
}
for(int y = 0; y < height; ++y) {
    for (int x = 0; x < width; ++x) {
        f[y][x] = g[y][x] + g[y+1][x] + g[y][x+1] + g[y+1][x+1];
    }
}
```

In the two Offload modes (see [Halide execution modes](#)), the `compute_root` directive can be used to schedule some Funcs (stages) on the CPU.

In the following example, the same pipeline has a slight change to use the offload directive, `hexagon`.

```
g(x, y) = x*y;
f(x, y) = g(x, y) + g(x, y+1) + g(x+1, y) + g(x+1, y+1);
g.compute_root();
```

Following is the loop nest that Halide will generate.

```
// Compute all of g before f
// so, compute g before going to Hexagon
// to compute f
int height = f.y.extent;
int width = f.x.extent;
int g[height+1][width+1];
for (int y = 0; y < height+1; ++y) {
    for (int x = 0; x < width+1; ++x) {
        g[y][x] = x*y;
    }
}
// compute f on Hexagon.
for<Hexagon>(int y = 0; y < height; ++y) {
    for (int x = 0; x < width; ++x) {
```

(continues on next page)

(continued from previous page)

```

    f[y][x] = g[y][x] + g[y+1][x] + g[y][x+1] + g[y+1][x+1];
  }
}

```

In this example, `g` is computed before moving on to the DSP; that is, `g` is computed on the CPU (host processor). Halide generates the code required to copy the data of `g` to the DSP for it to be used in the computation of `f`.

2.2.11 `compute_at`

```

compute_at(consumer, dim)
// consumer  : consumer Func
// dim       : dim of the consumer where this Func should be computed

```

The `compute_at` directive is used on a `Func` that produces data to be consumed by the `consumer` (not necessarily the only consumer). `compute_at` indicates that the producer should be computed at the loop traversing the dimension, `dim`, of `consumer`.

The memory allocated for the producer and the computed elements are both adjusted to match the requirements of `consumer` at `dim`. That is, the `compute_at` directive is used to specify *when* a producer should be computed relative to its consumer. The Halide compiler infers how much of the producer is to be computed.

The following example schedules `g`.

```

g(x, y) = x*y;
f(x, y) = g(x, y) + g(x, y+1) + g(x+1, y) + g(x+1, y+1);
g.compute_at(f, y);
f.hexagon();

```

Halide generates the corresponding pseudo loop nest.

```

int height = f.y.extent;
int width = f.x.extent;
int f[height][width];
for<Hexagon> (int y = 0; y < height; ++y) {
    // Only two rows of g are needed
    // per row of f. So allocate only
    // two rows to store g. Number of
    // columns of g needed for one row
    // of f is width+1.
    int g[2][width+1];
    for (int x = 0; x < width; ++x) {
        g[0][x] = x*y;
        g[1][x] = x*(y+1);
    }
    for (int x = 0; x < width; ++x) {
        f[y][x] = g[0][x] + g[1][x] + g[0][x+1] + g[1][x+1];
    }
}

```

(continues on next page)

(continued from previous page)

}

Because this example scheduled `g` in the `y` loop over `f`, schedule `g` will be computed on the Hexagon DSP.

2.2.12 store_root

```
store_root()
```

This directive says that the memory for the `Func` it was called on should be allocated at the outermost level.

Consider the following producer-consumer pipeline that shows how to use `store_root` and `compute_at` together.

```
g(x, y) = x*y;
f(x, y) = g(x, y) + g(x, y+1) + g(x+1, y) + g(x+1, y+1);
// Tell Halide to allocate storage for all of g at the
// outermost loop level, but still compute it per row
// (y) of f.
g.store_root().compute_at(f, y);
f.hexagon();
```

Halide generates the corresponding pseudo C/C++ code.

```
int height = f.y.extent;
int width = f.x.extent;
int f[height][width];
int g[height+1][width+1];
for (int y = 0; y < height; ++y) {
    if (y==0)
        g[y][x] = x*y;
    g[y+1][x+1] = (x+1)*(y+1);
    for (int x = 0; x < width; ++x) {
        f[y][x] = g[y][x] + g[y][x+1] + g[y+1][x] + g[y+1][x+1];
    }
}
```

As shown in this example, storage for all of `g` is allocated at the top level. However, even if two rows of `g` are required per row (`y` coordinate) of `f`, only one row of `g` is calculated (except at the top edge when `y` is 0). At every row of `f`, one row of `g` is reused because storage for all of `g` has been allocated. This way, redundant computations are reduced at the cost of an increased memory footprint.

2.2.13 store_at

```
store_at(consumer, dim)
// consumer : consumer Func
// dim      : dim of the consumer where storage for this Func should
//           be
//           allocated
```

The `store_at` directive is used on a `Func` that produces data (producer) that is consumed by the consumer (not necessarily the only consumer). This directive indicates that storage for the producer should be allocated at the loop traversing the dimension, `dim`, of `consumer`. The amount of memory allocated for the producer is adjusted to match the requirements of `consumer` at `dim`.

That is, this directive is used to specify *when* memory for a producer should be allocated relative to its consumer. The Halide compiler infers how much memory is to be allocated for the producer.

The following producer-consumer pipeline shows how to use `store_at` and `compute_at` together.

```
g(x, y) = x*y;
f(x, y) = g(x, y) + g(x, y+1) + g(x+1, y) + g(x+1, y+1);
// Tell Halide to allocate storage for g at the y loop
// over f, but compute it per x coordinate of f.
g.store_at(f, y).compute_at(f, x);
f.hexagon();
```

Halide generates the corresponding pseudo C/C++ code.

```
int height = f.y.extent;
int width = f.x.extent;
int f[height][width];
int g[height+1][width+1];
for (int y = 0; y < height; ++y) {
    // Memory is allocated to hold only 2 rows of g
    // because 2 rows of g are needed per row (y coordinate)
    // of f.
    int g[2][width+1];
    for (int x = 0; x < width; ++x) {
        g[0][x] = x*y;
        g[0][x+1] = (x+1)*y;
        g[1][x] = x*(y+1);
        g[1][x+1] = (x+1)*(y+1);
        g[2][x+1] = (x+1)*(y+1);
        f[y][x] = g[y][x] + g[y][x+1] + g[y+1][x] + g[y+1][x+1];
    }
}
```

2.2.14 hexagon_user_dma

New in version 2.3.0.

```
hexagon_user_dma()
```

The `hexagon_user_dma` directive is used on a `Func` that is a simple copy. When used on such a stage, the copy is performed using User DMA, first introduced in Hexagon ISA v68. Following is a typical example of this scheduling directive.

```
Func f{"f"};
Input<Buffer<uint8_t>> input{"input", 2};
Var x{"x"}, y{"y"};

f(x, y) = input(x, y);
f.hexagon_user_dma();
```

`hexagon_user_dma` is a specialization of `halide_buffer_copy` (see `Halide/include/HalideRuntime.h`).

The source and destination buffers can have different shapes when the shape of the destination buffer is completely contained inside the source buffer. This means the `min` of each dimension of the destination buffer must be greater than or equal to the `min` of the corresponding dimension of the source buffer.

Also, `min + extent` of each dimension of the destination buffer must be less than or equal to the `min + extent` of the corresponding dimension of the source buffer.

To use `hexagon_user_dma`, the `hvx_v68` target feature must be specified in the target for the generator (*Ahead-of-time compilation*).

2.2.15 align_storage

```
align_storage(dim, alignment)
// dim      : dimension to be aligned
// alignment : alignment of dimension in number of elements
```

Performance on the DSP, especially HVX, is sensitive to the alignment of data. The `align_storage` directive pads up the extent of storage of the dimension, `dim`, to be an exact multiple of the alignment. This means the dimensions stored outside of `dim` have strides that are multiples of `alignment`. Strides and alignment are measured in the number of elements.

For example, the following schedule ensures that each row (scanline) of `g` will start at an offset that is a multiple of 16 elements from the base address of the buffer for `g`.

```
g(x, y) = x*y;
f(x, y) = g(x, y) + g(x, y+1) + g(x+1, y) + g(x+1, y+1);
// Dimensions outside of x, that is the y dimension of
// g will start at an offset that is a multiple of 16
// elements from the start of g.
g.align_storage(x, 16);
```

2.2.16 TailStrategy

`TailStrategy` is not a scheduling directive. However, several scheduling directives, either explicitly or implicitly, split a dimension in two: an inner and an outer dimension. For example, `split` explicitly splits a dimension, while `vectorize` and `parallel` implicitly split a dimension. In terms of C/C++ code, this means splitting the loop that traverses a dimension into two nested loops.

An important aspect of any such splitting is what should be done when the split factor does not fully divide the extent of the dimension. The so-called remainder loop can be handled in several different ways, and all such directives (such as `vectorize`, `parallel`, and `split`) take one last optional parameter called `TailStrategy`, which defines how the remainder loop should be handled. Following are the different types of `TailStrategy`:

`TailStrategy::RoundUp`

Rounds up the extent to be a multiple of the split factor.

The benefit is that when vectorizing, Halide ensures that even the remainder loop is vectorized.

The drawback, however, is that when this strategy is used to split the dimension of a stage that reads or writes to an external buffer, it assumes the input or output image size is a multiple of the split factor. If the size is not a multiple of the split factor, the image is accessed out of bounds. This case causes the application to fault unless the image allocation is sufficiently padded to allow for the out-of-bounds access.

`TailStrategy::GuardWithIf`

Guards the inner loop with an if condition and thereby ensures that Halide never evaluates beyond the extent of the dimension.

This strategy is always legal and does not constrain the size of the input or the output. However, the drawback is that it inserts a conditional into the inner loop, and, in the remainder case, vectorization generates scalar code.

`TailStrategy::ShiftInWards`

Ensures that Halide does not evaluate beyond the extent of the original dimension by shifting the remainder case inwards.

This strategy is always legal. If it is used on a stage that reads or writes to an external buffer, it only requires that the extent of the buffer be at least as large as the split factor.

Like `TailStrategy::RoundUp`, this strategy also supports vectorization, because the inner loop will always be split-factor wide with no conditionals in it. However, some values are redundantly computed near the end of the dimension.

`TailStrategy::Auto`

For pure definitions, this strategy implies using `TailStrategy::ShiftInwards`.

For pure `Vars` in update definitions, it implies `TailStrategy::RoundUp`. For `RVars` in update definitions, it implies `TailStrategy::GuardWithIf`.

2.3 Memory

Halide pipelines work on memory buffers. However, they do not accept raw pointers to data. Instead, all Halide pipelines accept and operate on instances of `halide_buffer_t`.

`halide_buffer_t` is a C-style structure. `Halide::Runtime::Buffer` is a utility C++ class that

allows you to easily initialize and set up a `halide_buffer_t` object. The rest of this section refers to them interchangeably as buffers. Depending on the scope and lifetime of buffers, they are primarily classified into two types: external and internal.

2.3.1 External Buffers

When a Halide pipeline is compiled, the generated interface accepts buffers (pointers to `halide_buffer_t` objects) as input and output data. Such buffers are called external buffers because you or the caller of the pipeline allocate and initialize their memory outside of the pipeline.

The utility class, `Halide::Runtime::Buffer`, encapsulates a `halide_buffer_t` object and provides convenient methods for managing the `halide_buffer_t` object inside.

2.3.2 Internal Buffers

There are buffers to store data of the various `Func`s inside a pipeline. These buffers are created and used only inside the pipeline. The Halide runtime manages the memory for these buffers in a manner that is transparent to you.

The size of these buffers is determined by the pipeline schedule. For example, if a `Func` scheduled `compute_at` instead of `compute_root`, only part of the entire domain of the `Func` might be required in memory at any time. This means that where the `Func` is scheduled in the pipeline (`compute_at/compute_root`) plays a factor in how much memory is allocated.

2.3.3 Device Interface

An object of `halide_buffer_t` is a raw representation of an image. It is the data that a Halide pipeline operates on and produces. It holds information such as the location of the data held by the buffer in main memory, the number of dimensions of the buffer, and the size of each dimension. It also holds information that tracks whether the buffer is in main memory or on a peripheral device (like a GPU or the Hexagon DSP).

Each device (peripheral) target provides an interface (`halide_device_interface_t`) that manages device allocations. It provides an API to allocate buffer memory on the device, and to copy data from the host/CPU to the device and back.

The `halide_device_interface_t` provided by the Hexagon DSP can be retrieved by calling the function, `halide_hexagon_device_interface()`. Examples are in the following chapter.

2.4 Ahead-of-time compilation

Halide pipelines can be executed in just-in-time (JIT) compilation mode as well as compiled in the more traditional ahead-of-time (AOT) mode before execution. Because we recommend AOT compilation for Halide on HVX, this section discusses only ahead-of-time compilation.

Typically, AOT compilation in Halide is a two-step process:

1. The generator is compiled into a generator executable using the native compiler (for example, x86 compiler).
2. The generator executable is like a cross-compiler: when run, it invokes the Halide compiler and compiles your Halide pipeline into an object file or a static library, depending on the options passed to

the generator.

2.4.1 Generators

Generators in Halide are a more structured way to do AOT compilation of Halide pipelines. A Generator is a C++ class that is used to define a Halide pipeline. By having your generator inherit from `Halide::Generator`, you get a uniform interface to define and schedule the pipeline.

When a generator is compiled with `GenGen.cpp`, it is provided as part of the Halide installation. You get a generator executable that accepts various command line options and produces object code for the pipeline defined in the generator.

In the following example, a pipeline replaces each pixel with the maximum pixel in the 3x3 block around it.

```
#include "Halide.h"

using namespace Halide;

class Dilate3x3 : public Generator<Dilate3x3> {
public:
    // Takes an 8 bit image; one channel.
    Input<Buffer<uint8_t>> input{"input", 2};
    // Outputs an 8 bit image; one channel.
    Output<Buffer<uint8_t>> output{"output", 2};

    GeneratorParam<bool> use_parallel_sched{"use_parallel_sched", true}
    ↪;
    GeneratorParam<bool> use_prefetch_sched{"use_prefetch_sched", true}
    ↪;

    void generate() {
        Expr height = input.height();
        bounded_input(x, y) = BoundaryConditions::repeat_edge(input)(x, ↪
    ↪y);
        max_y(x, y) = max(bounded_input(x, clamp(y-1, 0, height-1)),
        output(x, y) = max(max_y(x-1, y), max_y(x, y), max_y(x+1, y));
    }

    void schedule() {
        if (get_target().has_feature(Target::HVX)) {
            Expr ht = output.dim(1).extent();
            bounded_input
                .compute_at(Func(output), y)
                .align_storage(x, 128)
                .vectorize(x, vector_size*2, TailStrategy::RoundUp);
            output
                .hexagon()
                .split(y, yo, y, ht/2)
                .tile(x, y, xi, yi, vector_size*2, 8, TailStrategy::RoundUp)
                .vectorize(xi)
        }
    }
};
```

(continues on next page)

(continued from previous page)

```

        .unroll(yi);
    if (use_prefetch_sched) {
        output.prefetch(input, y, 2);
    }
    if (use_parallel_sched) {
        output.parallel(yo);
    }
}
}
private:
    Var x{"x"}, y{"y"}, yo{"yo"};
    Func max_y{"max_y"};
    Func bounded_input{"bounded_input"};
};
HALIDE_REGISTER_GENERATOR(Dilate3x3, dilate3x3)

```

The inputs and the outputs of the pipeline are declared as public members of the pipeline and are instances of `Input` and `Output`. They will appear in the signature of the generated function in the order in which they are declared.

To construct the pipeline, the `generate` method must be defined. To schedule the constructed pipeline, the `schedule` method must be defined. Compile them with `GenGen.cpp` provided with the Halide installation. `GenGen.cpp` provides boilerplate code that accepts options on the command line to compile your pipeline. Register the pipeline with this boilerplate code by adding the following line:

```
HALIDE_REGISTER_GENERATOR(Dilate3x3, dilate3x3)
```

This line tells the boilerplate code that `Dilate3x3` defines the class that is a `Generator`. Once the class is compiled, it can be identified on the command line as `dilate3x3`. After compiling this line with `GenGen.cpp`, run the resulting executable to generate an object file that you can link with your application code.

`use_prefetch_sched` and `use_parallel_sched` are parameters that can be passed to the generator on the command line. The following example shows how to compile and run a generator.

```

$(HOST_CXX) -std=c++11 -I $(HALIDE_ROOT)/include -stdlib=libc++ -O3 -g \
↪ -fno-rtti -rdynamic dilate3x3_generator.cpp $(HALIDE_ROOT)/lib/
↪ libHalide.a $(HALIDE_ROOT)/tools/GenGen.cpp
    -o dilate3x3.generator -lz -lrt -ldl -lpthread -lm
    ./dilate3x3.generator -g dilate3x3 -o ./output_dir -e o,h -f \
↪ dilate3x3 target=arm-64-android-hvx use_parallel_sched=true use_
↪ prefetch_sched=true

```

Once this code is run, you will see `dilate3x3.o` and `dilate3x3.h` in the output directory, `output_dir`.

Following are the command line options that a generator accepts:

-g generator_name

A `Generator` file can have multiple generators, and this option selects which one to run (identified by the second argument to `HALIDE_REGISTER_GENERATOR`).

-o directory

Specifies the directory in which to create the outputs.

-e csv-list-of-outputs

A list of comma-separated values that specify the types of output to create.

Acceptable values are `static_library`, `object`, `c_header`, `assembly`, `bitcode`, `stmt`, and `stmt_html`, where:

- `assembly` generates assembly equivalent to the generated object file.
- `bitcode` generates LLVM bitcode for the pipeline.
- `stmt` generates human-readable pseudo code for the pipeline.
- `stmt_html` generates an HTML version of the pseudo code, which can be better to read than the raw STMT file.

target=<target>

The target to compile for.

Typically, the target is a hyphenated string of the form, `arch-bits-OS-feature1-feature2...`, where:

- `arch` can be one of the following: `arch_unknown`, `arm`, `hexagon`, `mips`, `powerpc`, `riscv`, `wasm`, or `x86`.
- `bits` is either 32 or 64.
- `OS` can be one of the following: `android`, `fuchsia`, `ios`, `linux`, `noos`, `os_unknown`, `osx`, `qurt`, `wasmrt`, or `windows`.
- If `arch`, `bits`, or `OS` is omitted, the default target is to the host.

The Halide compiler provides several features to consider from an HVX perspective:

- `hexagon_dma`, `hvx`, `hvx_shared_object`, `hvx_sysmon`, `hvx_v66`, and `hvx_v68`.
- Also, `debug` and `profile` are useful.
- `no_asserts` can be used to disable the injection of runtime asserts in the pipeline code. However, we do not recommend this use because asserts are lightweight and add negligible overhead in terms of execution time.
- The target can begin with `host`, which sets the host's architecture, operating system, and feature set. For example, `host-hvx` is a valid target that will ensure execution in Simulator Offload mode.

The target string implies the execution mode for a Halide-for-HVX pipeline (*Halide execution modes*). The following table provides sample Halide-for-HVX target strings.

Target string	Implied mode
arm-64-android-hvx	Device Offload mode with HVX support
arm-64-linux-hvx	Device Offload mode with HVX support on LE devices
arm-64-android-hvx-hvx_	Device Offload mode with HVX support and Hexagon V66 ISA
arm-64-android-hvx-debug	Device Offload mode with HVX support and debug logs
host-hvx	Simulator Offload mode with HVX support
hexagon-32-noos-hvx	Simulator Standalone mode with HVX support
hexagon-32-qurt-hvx	Device Standalone mode with HVX support

For more information about target features relevant to the Hexagon DSP, see [Relevant target features for HVX](#).

3 Halide for HVX

- *Halide execution modes*
 - *Offload mode*
 - *Standalone mode*
- *Relevant target features for HVX*
 - *hvx*
 - *hvx_v66*
 - *hvx_v68*
 - *hexagon_dma*
 - *profile*
 - *debug*
 - *trace_realizations*
 - *trace_pipeline*
- *Performance on HVX*
 - *Alignment*
 - *Memory locality*
 - *Zero-copy buffers*
 - *VTM and scatter-gather operations*
 - *Power and performance APIs*
 - *Profiling*
- *Halide HVX built-ins*
 - *Supported Halide HVX built-ins*
 - *Usage and examples for HVX built-ins*
- *Troubleshooting*
 - *User controllable exits*
 - *Target malloc tracing*
 - *Set heap grow size*

- *Debug target feature*
- *Useful environment variables*
- *Common compile time or runtime errors*
- *Signing for deployment*
 - *Tools for signing Offload mode pipelines*
 - *Unsigned mode execution*

Qualcomm Halide for HVX is used to target the HVX architecture in two ways:

- As a compiler for generating code to run on a Hexagon DSP with HVX
- As runtime libraries to support running the code compiled by the Halide compiler on a Hexagon DSP with HVX

Compile Halide pipelines to generate a binary that is executed at runtime. Halide for HVX supports two runtime targets:

- Qualcomm® Snapdragon™ devices (Device Offload and Device Standalone modes)
- Hexagon Simulator (Simulator Offload and Simulator Standalone modes)

3.1 Halide execution modes

You can generate two variants of Halide binaries: Offload mode and Standalone mode.

3.1.1 Offload mode

The generated Halide binary is launched from a host processor.

On Snapdragon devices, the host process is the ARM applications processor. This runtime model is also called the *Device Offload* mode. In this mode, you do not have to deal with the underlying details of FastRPC communication between the host processor and the Hexagon compute DSP (cDSP). The Halide Device Offload mode runtime libraries take care of these details.

On the Hexagon simulator runtime target, the host processor is the x86 processor. This runtime model is called the *Simulator Offload* mode.

3.1.2 Standalone mode

The generated Halide binary is a standalone HVX object file. Use this file to integrate Halide pipelines into an existing Hexagon application.

On Snapdragon devices, the generated object file can be launched on the HVX processor. This runtime mode is called the *Device Standalone* mode. Unlike the Device Offload mode, in this mode, you handle the details of communication between the host processor and the cDSP. All stages of the pipeline execute on the cDSP, and you are responsible for authoring IDL files for FastRPC from the host processor to the Hexagon DSP.

On the Hexagon simulator runtime target, the generated object file can be used to simulate an end-to-end vision algorithm running on the Hexagon DSP with HVX. This mode is called the *Simulator Standalone* mode.

3.2 Relevant target features for HVX

Following are the target features that can be used in the `target` command line argument to a generator ([Generators](#)).

3.2.1 hvx

Use the `hvx` target feature to instruct the Halide compiler to compile a pipeline for HVX. The natively supported vector length is assumed to be 128 bytes.

Note: `hvx` is required to target the HVX ISA in all execution modes, even when the host architecture in the target is the Hexagon architecture.

3.2.2 hvx_v66

Use the `hvx_v66` target feature to instruct the Halide compiler to generate code for HVX v66 ISA.

3.2.3 hvx_v68

New in version 2.3.0.

Use the `hvx_v68` target feature to instruct the Halide compiler to generate code for HVX v68 ISA. The following features are enabled by using this target feature.

3.2.3.1 User DMA

HVX ISA v68 provides support for user-space Direct Memory Access (DMA).

Halide supports this feature in four execution modes. It can be requested by using the `hexagon_user_dma` scheduling directive when the `hvx_v68` target feature is used.

3.2.3.2 Vector floating point

Halide provides support for HVX floating points on the Hexagon DSP for both `float16_t` half-precision (16-bit) and `float_t` single precision (32-bit) data types, as well as conversion between floating point and integer data types. Floating point vector support is only available on HVX ISA v68 or later and only in 128-byte mode.

To enable this feature, add both the `hvx` and `hvx_v68` Halide target features to the target when running your generator.

The following functions support both 16- and 32-bit floating point vectors:

```
sin(x)
cos(x)
tan(x)
asin(x)
acos(x)
atan(x)
sinh(x)
cosh(x)
tanh(x)
exp(x)
log(x)
pow(x,y)
sqrt(x)
fast_inverse_sqrt(x)
hypot(x,y)
floor(x)
ceil(x)
trunc(x)
round(x)
is_inf(x)
is_nan(x)
is_finite(x)
```

The following functions support only 32-bit floating point vectors:

```
fast_exp(x)
fast_log(x)
```

Using other functions will result in scalar code generation. Support for more functions will be added in a future release.

3.2.4 hexagon_dma

Use the `hexagon_dma` target feature in the two Standalone modes to use Universal Bandwidth Compression Direct Memory Access (UBWC DMA).

3.2.5 profile

Use the `profile` target feature in Device Offload and Device Standalone modes to profile the Halide pipeline. For more information, see [Target feature for profiling](#).

3.2.6 debug

Use the `debug` target feature to enable debug messages in the Halide pipeline and at runtime.

3.2.7 trace_realizations

Use the `trace_realizations` target feature in Device Standalone mode to profile the complete Halide pipeline as well as each individual `Func` that is not scheduled inline.

The Halide runtime on Hexagon uses the Instrumented Trace (ITRACE) library for gathering runtime statistics using the Performance Monitoring Unit (PMU) on the CDSP. For more information, see [Use Instrumented Trace \(ITRACE\) for profiling](#).

3.2.8 trace_pipeline

Use the `trace_pipeline` target feature in Device Standalone mode to profile a complete Halide pipeline only (not individual functions) using Instrumented Trace (ITRACE) library. For more information, see [Use Instrumented Trace \(ITRACE\) for profiling](#).

3.3 Performance on HVX

Halide is unique in its separation of the algorithm from the schedule. Such a separation allows you to freeze the algorithm and search through the schedules to improve performance. However, there are some things you can do to fine tune performance on the Hexagon DSP. In the following sections, the Halide pipeline is simply called the pipeline or pipeline code. The C++ code that calls the pipeline is called the application.

3.3.1 Alignment

For best performance on the Hexagon DSP with HVX, align vector loads and stores to the native vector width (128 bytes in 128-byte vector mode). For a vector load to load from an aligned address, you need both the base address and the offset from the base address to be multiples of the native vector width.

3.3.1.1 Alignment of external buffers

To ensure that a vector load (or store) from an external buffer is aligned, tell the Halide compiler that the base address of the external buffer is aligned. This is done using the `set_host_alignment` directive in the pipeline (see the example). Further, for a two dimensional buffer, ensure that the stride of the outer dimension is aligned to the native vector width. In other words, for a two dimensional buffer, ensure that every row is aligned to the native vector width by using `set_stride`.

The following example shows the Halide pipeline code for aligning the host pointer and strides of the external buffers.

```
Input<Buffer<uint8_t>> input{ "input", 2};
Output<Buffer<uint8_t>> output{ "output", 2};
// Schedule
constexpr int vector_size = 128;

// Set the stride of dimension 1 (y dimension) to be a multiple of the
// native vector size.
Expr input_stride = input.dim(1).stride();
input.dim(1).set_stride((input_stride/vector_size) * vector_size);
Expr output_stride = output.dim(1).stride();
output.dim(1).set_stride((output_stride/vector_size) * vector_size);

// Set the expected alignment of the host pointer in bytes.
input.set_host_alignment(vector_size);
output.set_host_alignment(vector_size);
```

When the Halide compiler generates code for a pipeline that uses `set_host_alignment` and `set_stride`, the compiler will be able to more efficiently determine the alignment of vector loads and stores. It will also insert runtime asserts into the generated code that check the host alignment and strides of the buffers that are passed at runtime. For example, at runtime, if you pass a buffer that is not aligned to a 128-byte boundary, the pipeline will crash with the error code, `halide_error_code_unaligned_host_ptr`.

Remember, the memory is allocated for an external buffer (see [External Buffers](#)) by the caller of the pipeline. Thus, before calling the pipeline in the C++ application, use the `Halide::Runtime::Buffer` utility class to allocate a buffer whose host pointer and strides are aligned to the natural vector width (as shown in the following example). When memory is allocated using the device interface provided by the DSP, the host pointer is automatically aligned to the natural vector width.

The following example shows the application code for aligning the host pointer and strides of the external buffers.

```
#include "HalideRuntimeHexagonHost.h"
#include "HalideBuffer.h"

// Assume width and height are provided
// already.
// Align the stride
const int VLEN=128;
```

(continues on next page)

(continued from previous page)

```

int stride_y = (width + (VLEN)-1) & ~(VLEN);

// Define the dimensions
halide_dimension_t x_dim{0, width, 1};
halide_dimension_t y_dim{0, height, stride_y};
halide_dimension_t io_shape[2] = {x_dim, y_dim};

Halide::Runtime::Buffer<uint8_t> in(nullptr, 2, io_shape);
Halide::Runtime::Buffer<uint8_t> out(nullptr, 2, io_shape);
// The following call will use the device interface
// provided by the DSP to allocate device memory which
// is aligned to the natural vector width.
in.device_malloc(halide_hexagon_device_interface());
out.device_malloc(halide_hexagon_device_interface());

```

3.3.1.2 Alignment of internal buffers

To align internal pipeline stages, use the `align_storage` ([align_storage](#)) scheduling directive.

The following example shows the Halide pipeline code that aligns internal buffers.

```

Input<Buffer<uint8_t>> input{ "input", 2};

widened_input(x, y) = cast<int16_t>(input(x, y));
// Pad the storage extent of the 'x' dimension of the storage allocated
// for 'widened_input' to be a multiple of 64. This ensures that the
// strides of dimensions outside 'x' are multiples of the specified_
↪alignment.
// Strides and alignment are viewed in terms of number of elements_
↪here.
widened_input
    .compute_at(Func(output), y)
    .align_storage(x, 64)
    .vectorize(x, vector_size, TailStrategy::RoundUp);

```

In this example, `widened_input` will have its rows aligned to an integral multiple of 64 elements, where each element is two bytes long. This will ensure that the rows are aligned to a 128-byte boundary, the natural vector width for HVX.

3.3.1.3 Align when splitting dimensions

Use `TailStrategy::RoundUp` for a directive that splits a dimension regardless of whether the split is explicit ([split](#)) or implicit ([vectorize](#)).

The following example shows the Halide pipeline code that ensures alignment when splitting a dimension.

```

Input<Buffer<uint8_t>> input{ "input", 2};

widened_input(x, y) = cast<int16_t>(input(x, y));

```

(continues on next page)

(continued from previous page)

```
// Schedule:
// vectorize splits the 'x' dimension into an inner dimension of size
// vector_size and an outer dimension. If the extent of the outer_
↳ dimension
// is not a perfect multiple of 'vector_size', then the code below_
↳ will ensure
// that we round up to the next vector boundary. If, however, this is_
↳ used on
// stage that reads from or writes to an external buffer, it_
↳ constrains the
// size of the external buffer to be a multiple of the split_factor_
↳ (vector_size in
// this case).
widened_input
    .compute_at(Func(output), y)
    .align_storage(x, 64)
    .vectorize(x, vector_size, TailStrategy::RoundUp);
```

3.3.2 Memory locality

Locality affects the latency of memory. Achieving good memory locality can significantly improve the performance of a program.

3.3.2.1 Tiling

Use tiling to improve locality in Halide, as shown in the following example.

```
Input<Buffer<uint8_t>> input{ "input", 2};
Func f{"f"}, g{"g"};
Var x{"x"}, y{"y"};
Var xi{"xi"}, yi{"yi"};

f(x, y) = cast<uint16_t>(input(x-1, y)) - cast<uint16_t>(input(x+1,
↳ y));
g(x, y) = f(x, y-1) + 2*f(x, y) + f(x, y+1);
// Schedule:
g.tile(x, y, xi, yi, 256, 32, TailStrategy::RoundUp)
```

This pipeline code is a producer-consumer relationship between `f` and `g`. We have scheduled the computation of `g` in tiles of size 256 x 32 elements. Further, `f` is now computed as required by a tile of `g`.

3.3.2.2 Line buffering

Two issues arise when tiling on HVX:

- Reasonably sized tiles can be smaller than two vectors in width because of the large vector lengths supported by HVX.
- When tiling stencils, it is difficult for the producer functions to satisfy native vector requirements to avoid scalarization.

The solution is to use line buffering to produce lines of the producer as required by lines of the consumer.

The following example shows the Halide pipeline that uses line buffering.

```
Input<Buffer<uint8_t>> input{ "input", 2};
Func f{"f"}, g{"g"};
Var x{"x"}, y{"y"};
Var xi{"xi"}, yi{"yi"};

f(x, y) = cast<uint16_t>(input(x-1, y)) - cast<uint16_t>(input(x+1,
    ↪y));
g(x, y) = f(x, y-1) + 2*f(x, y) + f(x, y+1);
// Schedule:
f.store_root().compute_at(g, y);
```

As with tiling, this pipeline is a producer-consumer relationship between `f` and `g`. For every line (row) of `g`, three lines of `f` are required. By using `compute_at` (*compute_at*), the pipeline is producing the number of lines of `f` required per line of `g`. By allocating storage for `f` at a higher level (loop in the loop nest) using `store_root` (*store_root*), these lines are buffered. This will ensure that in every subsequent iteration of the loop traversing the `y` dimension of `g`, two lines of `f` computed in the previous iteration are reused.

Halide generates the corresponding pseudo C/C++ code.

```
int height = g.y.extent;
int width = g.x.extent;
int g[height][width];
int f[height+2][width];

// For loop over the y dimension of g
for (int y = 0; y < height; y++) {
    if (y==0)
        f[y-1][x] = cast<uint16_t>(input[y-1][x-1]) - cast<uint16_t>
    ↪(input[y-1][x+1]);
        f[y][x] = cast<uint16_t>(input[y][x-1]) - cast<uint16_t>
    ↪(input[y][x+1]);
        f[y+1][x] = cast<uint16_t>(input[y+1][x-1]) - cast<uint16_t>
    ↪(input[y+1][x+1]);

    // For loop over the x dimension of g
    for (int x = 0; x < width; x++) {
        g[y][x] = f[y-1][x] + 2*f[y][x] + f[y+1][x];
    }
}
```

(continues on next page)

(continued from previous page)

}

3.3.3 Zero-copy buffers

Zero-copy buffers are allocated in memory that is visible to the host processor and the Hexagon DSP. When working with zero-copy buffers, you do not pay the penalty of copying data back and forth between the host CPU and the Hexagon DSP.

When allocating memory for a large external buffer and if the `halide_device_interface` provided by the Hexagon DSP is used, zero-copy memory is allocated by default. This is done by using the underlying ION memory manager. The C++ convenience class `Halide::Runtime::Buffer` can be used as shown in the following example.

Following is the application code that allocates zero-copy buffers for external code using `Halide::Runtime::Buffer`.

```
#include "HalideRuntimeHexagonHost.h"
#include "HalideBuffer.h"

// Assume width and height are provided
// already.
// Align the stride
const int VLEN=128;
int stride_y = (width + (VLEN)-1) & ~(VLEN);

// Define the dimensions
halide_dimension_t x_dim{0, width, 1};
halide_dimension_t y_dim{0, height, stride_y};
halide_dimension_t io_shape[2] = {x_dim, y_dim};

// The first argument is the pointer to data in main memory (host_
// →pointer)
// If this is nullptr, then the halide_hexagon_device_interface
// will use its device_malloc implementation to allocate an ion
// memory buffer and also use it as the host pointer.
Halide::Runtime::Buffer<uint8_t> in(nullptr, 2, io_shape);
Halide::Runtime::Buffer<uint8_t> out(nullptr, 2, io_shape);
// The following call will use the device interface
// provided by the DSP to allocate device memory which
// is aligned to the natural vector width.
in.device_malloc(halide_hexagon_device_interface());
out.device_malloc(halide_hexagon_device_interface());
```

3.3.3.1 rpcmem_alloc and rpcmem_free

If you are adding Halide to an existing application that already uses the rpcmem library for allocating zero-copy buffers, use `halide_hexagon_wrap_device_handle()` to attach the memory allocated with `rpcmem_alloc` to a Halide buffer.

The following example illustrates using rpcmem with external buffers in Halide.

```
#include "rpcmem.h"
#include "HalideRuntimeHexagonHost.h"

halide_buffer_t input_buf;
halide_buffer_t output_buf;
rpcmem_init(0);

// Allocate buffers
// Over-allocate by one vector
const int bufsize = stride * height + VLEN;
input_buf.host = (uint8_t*)rpcmem_alloc(25, RPCMEM_DEFAULT_FLAGS,
↪bufsize);
output_buf.host = (uint8_t*)rpcmem_alloc(25, RPCMEM_DEFAULT_FLAGS,
↪bufsize);
if (input_buf.host == NULL || output_buf.host == NULL) {
    printf("Error: Cannot allocate memory\n");
    return 1;
}
halide_hexagon_wrap_device_handle(nullptr, &input_buf, input_buf.host,
↪bufsize);
halide_hexagon_wrap_device_handle(nullptr, &output_buf, output_buf.
↪host, bufsize);

// Free buffers
rpcmem_free(input_buf.host);
rpcmem_free(output_buf.host);
rpcmem_deinit();
```

Note: Remember to over allocate by one vector. This is important because certain optimizations in the Halide compiler assume they can read or write extern buffers one vector past the end of the buffer.

3.3.4 VTCM and scatter-gather operations

Vector Tightly Coupled Memory (VTCM) is fast vector memory available on Hexagon ISA V65 and later. In Halide 2.4.x, HVX v65 is the lowest supported HVX ISA. As such, this feature is available simply by the use of the `hv<sub>x</sub>` target feature.

The following sections describe the two main use cases.

3.3.4.1 Internal (intermediate) buffers

Because VTCM is memory with very low latency, performance can be improved by storing small internal buffers in it. This must be done very carefully as VTCM memory is not large and is shared. Therefore, it is possible that some allocations might not fit in VTCM. In such cases, the pipeline will exit with the error code, `halide_error_code_vtcm_out_of_memory`. Typically, such issues are fixed by reducing tile sizes.

Following is an example of using VTCM for intermediate buffers.

```
Input<Buffer<uint8_t>> input{ "input", 2};
Output<Buffer<uint8_t>> output{ "output", 2};
Func f{"f"};

f(x, y) = 2*input(x, y);
output(x, y) = f(x, y) + 2;

f.compute_at(output, y)
  .store_in(MemoryType::VTCM)
  .vectorize(x, 128);
```

3.3.4.2 Scatter-gather operations

Scatter-gather operations in HVX work exclusively on data in the VTCM.

Note: Scatter instructions on the Hexagon DSP only operate on 16-bit and 32-bit datatypes. To use these operations on 8-bit datatypes, add a stage (`Func`) to cast the appropriate buffers to 16-bit datatypes before doing the scatter. This can be followed by a cast back down to an 8-bit datatype. For an example, see [Scatter operations on 8-bit data](#).

Gathers are simply table lookups in Halide. The following example shows the Halide pipeline code doing a gather.

```
Input<Buffer<int16_t>> input{ "input", 1};
Input<Buffer<int16_t>> lut{ "lut", 1};
Output<Buffer<int16_t>> output{ "output", 1};
Func lut_vtcm{"lut_vtcm"}, output_vtcm{"output_vtcm"};
Var x{"x"};

lut_vtcm(x) = lut(x);
output_vtcm(x) = lut_vtcm(clamp(input(x), 0, input.dim(0).extent()-1));
```

(continues on next page)

(continued from previous page)

```

output(x) = output_vtcm(x);

lut_vtcm.compute_at(output, Var::outermost())
    .store_in(MemoryType::VTCM)
    .vectorize(x, 128);
output_vtcm.compute_at(output, Var::outermost())
    .store_in(MemoryType::VTCM)
    .vectorize(x, 128);

```

This example is a typical lookup operation in Halide, where:

- `lut` is the look up table (LUT)
- `input` represents the indices to be looked up in the table
- `output` is the output buffer

For the Halide compiler to generate an HVX gather instruction, we have placed both the output and the LUT (`lut`) by defining `output_vtcm` and `lut_vtcm`, respectively. The `store_in` directive is then used to ensure that these instructions are placed in VTCM.

Scatters, on the other hand, simply invert the relationship between `lut_vtcm` and `output_vtcm` from the previous example. However, because the output buffer might not have values defined for all indices, a pure definition stage is required and then an update definition (*Pure and Update definitions*) is used for the scatter.

The following example shows the Halide pipeline code doing a scatter.

```

Input<Buffer<int16_t>> input{"input", 1};
Output<Buffer<int16_t>> output{"output", 1};
Func output_vtcm{"output_vtcm"};
Var x{"x"};

// pure definition
output_vtcm(x) = cast<int16_t>(0);
// update definition
output_vtcm(clamp(input(x), 0, input.dim(0).extent()-1)) = x*input(x);
output(x) = output_vtcm(x);

output_vtcm
    .allow_race_conditions()
    .compute_at(output, Var::outermost())
    .store_in(MemoryType::VTCM)
    .vectorize(x, 128);

output_vtcm
    .update(0)
    .vectorize(x, 128);

```

If the values of `input` contain duplicates, then vectorizing the update definition is essentially unsafe. However, if the programmer can guarantee that there are no duplicates, such vectorization can be done by allowing the Halide compiler to ignore race conditions (`allow_race_conditions`).

By changing a scatter to accumulate values, you can generate scatter-accumulates which are supported by HVX ISA. This is extremely useful in computing vectorized histograms. The following example shows the Halide pipeline code for a scatter-accumulate.

```
Input<Buffer<int32_t>> input{"input", 1};
Func histogram{"histogram"};

// Pure definition.
histogram(x) = cast<int16_t>(0);

// Update definition - a reduction definition
histogram(input(x)) += 1;
```

The following example shows the scheduling histograms for generating scatter-accumulates.

```
histogram
    .allow_race_conditions()
    .compute_at(output, Var::outermost())
    .store_in(MemoryType::VTCM)
    .vectorize(x, 64);
histogram
    .update(0)
    .vectorize(x, 64);
```

3.3.4.3 Scatter operations on 8-bit data

HVX supports scatter-gather instructions only on 16-bit and 32-bit data. Halide uses 16-bit predicated vgather instructions to do gathers on 8-bit data. However, to scatter 8-bit data, you must explicitly cast the data to a wider type followed by casting to a narrower type after scattering.

Following is an example of scatter operation on 8-bit data.

```
Input<Buffer<uint8_t>> input{ "input", 1};
Output<Buffer<uint8_t>> output{ "output", 1};
Func f{"f"};
Var x{"x"};

// Cast to a wider type
f(x) = cast<int16_t>(0);
// Update definition
f(clamp(input(x), 0, input.dim(0).extent()-1)) = 1;
// Cast back to the narrower type
output(x) = cast<uint8_t>(f(x));

f.compute_at(output, Var::outermost())
    .store_in(MemoryType::VTCM)
    .vectorize(x, 128);
```

3.3.5 Power and performance APIs

For increased control over the power and performance of your application, specify a power level before powering on HVX. To do this, use the functions provided by Halide to request (vote for) a specific performance mode. You can also explicitly specify individual performance parameters if you do not want to choose from one of the predefined power levels.

The following example shows the application-side API that specifies the power level of the application (voting).

```
#include "HalideRuntimeHexagonHost.h"
int halide_hexagon_set_performance_mode(void *user_context, halide_
    ↪ hexagon_power_mode_t mode, bool dcvs_enable=true);
// user_context - Ignore. Can be NULL
// mode         - Power level value similar to selecting a voltage_
    ↪ corner
//               as documented in the Hexagon SDK documentation.
// dcvs_enable  - Turn DCVS participation on or off. The default value_
    ↪ is True. For details about DCVS, see the Hexagon
//               SDK documentation
```

The following table lists UBWC DMA with the Halide Power APIs.

Mode value	Equivalent voltage corner (See Hexagon SDK Documentation)	Alternate name of mode
halide_hexagon_power_lo	HAP_DCVS_VCORNER_SVS	halide_hexagon_power_svs
halide_hexagon_power_no	HAP_DCVS_VCORNER_NOM	'' ''
halide_hexagon_power_tu	HAP_DCVS_VCORNER_TURBO	'' ''
halide_hexagon_power_tu	HAP_DCVS_VCORNER_TURBO_	'' ''
halide_hexagon_power_tu	HAP_DCVS_VCORNER_TURBO_	'' ''
halide_hexagon_power_tu	HAP_DCVS_VCORNER_TURBO_	'' ''
halide_hexagon_power_ma	HAP_DCVS_VCORNER_MAX	'' ''
halide_hexagon_power_de	HAP_DCVS_VCORNER_DISABI	'' ''
halide_hexagon_power_lo	HAP_DCVS_VCORNER_SVSPLI	halide_hexagon_power_svs_plus
halide_hexagon_power_lo	HAP_DCVS_VCORNER_SVS2	halide_hexagon_power_svs2
halide_hexagon_power_no	HAP_DCVS_VCORNER_NOMPLI	'' ''

The alternative to using predefined power levels is to set individual performance parameters instead. However, we recommend using the predefined power-level approach over this one.

To set individual performance parameters, use `halide_hexagon_set_performance`, as shown in the following application-side API example.

```
#include "HalideRuntimeHexagonHost.h"
halide_hexagon_power_t perf;
perf.set_mips = 1;
perf.mipsPerThread = 825;
perf.mipsTotal = 1650;
perf.set_bus_bw = 1;
```

(continues on next page)

(continued from previous page)

```

perf.bwMegabytesPerSec = 18750;
perf.busbwUsagePercentage = 100;
perf.set_latency = 1;
perf.latency = 10;
halide_hexagon_set_performance (NULL , &perf );

```

An application can be made to consume lower power than it would if the chosen power mode is `halide_hexagon_power_low`, if you let the device run the clock settings at a lower power level when the application is idle. Do this by setting the power level to `halide_hexagon_power_default` after the application is finished using the cDSP. This usage is shown in the following application-side API example:

```

#include "HalideRuntimeHexagonHost.h"

// To avoid the cost of powering HVX on in each call of the pipeline,
// set performance mode to turbo and power HVX on once now.
halide_hexagon_set_performance_mode(NULL, halide_hexagon_power_turbo);
halide_hexagon_power_hvx_on(NULL);
printf("Running pipeline...\n");
double time = benchmark(iterations, 10, [&]() {
    int result = pipeline(&in, &out);
    if (result != 0) {
    }
});
// We're done with HVX for now, power it off.
halide_hexagon_power_hvx_off(NULL);
// Set performance mode back to the default for lower idle power usage
halide_hexagon_set_performance_mode(NULL, halide_hexagon_power_
↪default);

```

3.3.6 Profiling

Profiling authored code allows you to understand the performance bottlenecks in your programs. Halide for HVX provides a several methods of profiling depending on the modes of execution.

3.3.6.1 Target feature for profiling

If the `profile` target feature is used in the target argument when running a generator, the Halide compiler inserts code into the pipeline so that it can be profiled at runtime.

This target feature is supported in both Device Offload and Device Standalone modes. The Halide pipeline source code is not required to change. However, in Device Standalone mode, you must call `halide_profiler_report` after the call to your Halide pipeline.

For example, consider the following sample pipeline to profile and the corresponding application code that calls the pipeline:

```

Expr height = input.height();
bounded_input(x, y) = repeat_edge_x(input)(x, y);

```

(continues on next page)

(continued from previous page)

```

max_y(x, y) = max(bounded_input(x, clamp(y-1, 0, height-1)),
bounded_input(x, clamp(y, 0, height-1)),
bounded_input(x, clamp(y+1, 0, height-1)));
output(x, y) = max(max_y(x-1, y), max_y(x, y), max_y(x+1, y));

//Schedule
if (get_target().has_feature(Target::HVX)) {
    Expr ht = output.dim(1).extent();
    bounded_input
        .compute_at(Func(output), y)
        .align_storage(x, 128)
        .vectorize(x, vector_size*2, TailStrategy::RoundUp);
    output
        .hexagon()
        .split(y, yo, y, ht/2)
        .tile(x, y, xi, yi, vector_size*2, 8, TailStrategy::RoundUp)
        .vectorize(xi)
        .unroll(yi);
    output.prefetch(input, y, 2);
    output.parallel(yo);
}

```

The following example shows the application code that enables Device Offload mode profiling.

```

// Set up buffers for the pipeline as usual
// Then, call the pipeline
dilate3x3(input, output);

// Now, just before finishing up, call halide_profiler_report
halide_profiler_report(nullptr);

```

Now recompile and run the generator. If the target argument to the generator was arm-64-android-hvx, use arm-64-android-hvx-profile for profiling. Rebuild your application and run it as usual. To see the profiling data, use adb.

```

$> adb logcat | grep halide
07-15 22:35:31.112 13729 13729 I halide : dilate3x3
07-15 22:35:31.112 13729 13729 I halide : total time: 195.286346 ms
↳ samples: 151 runs: 100 time/run: 1.952863 ms
07-15 22:35:31.112 13729 13729 I halide : average threads used: 1.
↳ 192053
07-15 22:35:31.112 13729 13729 I halide : heap allocations: 0 peak
↳ heap usage: 0 bytes
07-15 22:35:31.112 13729 13729 I halide : output: 1.
↳ 212ms (62%) threads: 0.729
07-15 22:35:31.112 13729 13729 I halide : bounded_input: 0.
↳ 739ms (37%) threads: 2.000

```

Only the stages that have a schedule (not computed inline) appear in the profile. In this profile example, output and bounded_input appear but max_y does not because it is computed inline. In Device

Standalone mode, the profile output can be seen using QXDM or mini-dm.

3.3.6.2 Use hexagon-profiler for profiling

The Hexagon Tools are shipped with a profiling utility called `hexagon-profiler`. This utility provides instruction-level profiler information such as the number of stalls, types of stalls, and number of packet commits. You can use this tool when Halide pipelines are run in Simulator Offload and Simulator Standalone modes.

In Simulator Offload mode, before running the application, you must set the `HL_HEXAGON_PACKET_ANALYZE` environment variable to the name of the file in which you want the simulator to write profiling information. For example, `HL_HEXAGON_PACKET_ANALYZE` is set to `profile.json`, and the simulator is set to use Timing mode by enabling `HL_HEXAGON_TIMING`.

```
export HL_HEXAGON_PACKET_ANALYZE=profile.json
export HL_HEXAGON_TIMING=1
```

When you run your application as shown in the following example, the simulator will output a profile to `profile.json`, with the shell command line to run to process `profile.json`.

```
To generate profile: hexagon-profiler --packet_analyze --json=profile.
→json
--elf=libhalide_shared_runtimeT1530632564432956148P5181.so:0x68000 -o
profile.html
To generate profile: hexagon-profiler --packet_analyze --json=profile.
→json
--elf=libhalide_hexagon_codeT1530632562748212127P5181.so:0x5f000 -o
profile.html
```

To view profile information for the pipeline itself, run the second command in the example. This will provide a visualization for the pipeline profile and not the runtime library profile. To view the profile, open `profile.html` in a browser.

In Simulator Standalone mode, use `--timing` and `--packet_analyze` when running your application on `hexagon-sim` to get profile information that can be fed to `hexagon-profiler`. To learn more about these options and the `hexagon-profiler`, see the Hexagon Profiler User Guide that is part of the Hexagon Document Bundle distributed with the Hexagon Tools.

3.3.6.3 Use Instrumented Trace (ITRACE) for profiling

New in version 2.5.0.

The ITRACE library allows you to monitor events in parallel on multiple domains of Qualcomm devices. With the ITRACE library, you can register a set of Performance Monitoring Unit (PMU) events on the cDSP. You can also identify one or more code sections during which the registered PMU events will be monitored.

To profile Halide pipelines executing in Device Standalone mode, use the ITRACE library by leveraging tracing in the Halide runtime. By using the `trace_realizations` target feature, you can use ITRACE to profile an entire Halide pipeline on HVX.

You can also profile individual stages that are not scheduled inline. While you can use the `trace_pipeline` target feature to profile complete pipelines, you can enable profiling on individual

Funcs by using the `trace_realizations` scheduling directive on them.

For example, consider the following sample pipeline to profile and the application code:

```
Expr height = input.height();
bounded_input(x, y) = repeat_edge(input)(x, y);
max_y(x, y) = max(bounded_input(x, clamp(y-1, 0, height-1)),
bounded_input(x, clamp(y, 0, height-1)),
bounded_input(x, clamp(y+1, 0, height-1)));
output(x, y) = max(max_y(x-1, y), max_y(x, y), max_y(x+1, y));

//Schedule
if (get_target().has_feature(Target::HVX)) {
  Expr ht = output.dim(1).extent();
  bounded_input
    .compute_at(Func(output), y)
    .align_storage(x, 128)
    .trace_realizations() //add to profile using ITRACE
    .vectorize(x, vector_size*2, TailStrategy::RoundUp);
  output
    .hexagon()
    .split(y, yo, y, ht/2)
    .tile(x, y, xi, yi, vector_size*2, 8, TailStrategy::RoundUp)
    .trace_realizations() //add to profile using ITRACE
    .vectorize(xi)
    .unroll(yi);
  output.prefetch(input, y, 2);
  output.parallel(yo);
}
```

As shown in this example, you can use the `trace_realizations` scheduling directive to profile an individual Func. Alternatively, the `trace_realizations` target feature can be added to the target when a generator is run. For example:

- `hexagon-32-qurt-hvx-trace_realizations` – Will profile each individual Func that has not been scheduled inline,
- `hexagon-32-qurt-hvx-trace_pipeline` – Will profile the entire pipeline as one entity.

To learn more about ITRACE, see the Hexagon SDK documentation.

3.4 Halide HVX built-ins

New in version 2.5.0.

Halide built-ins for HVX allow for finer control over the HVX instructions generated by the Halide compiler. Use these built-ins to improve the performance of your pipeline. They ensure the generation of certain HVX instructions, which would otherwise be hard for the compiler to generate out of typical Halide expressions written by a user. For example, it is hard for the compiler to generate the HVX instruction, `vror`, without using Halide built-ins for HVX.

Similarly, certain multiply-accumulate instructions can be reliably generated by using the built-ins. For

example, the following expression, when vectorized with an appropriate factor, should result in the generation of `vmpa`:

```
// Assume input is of type UInt(8)
output(x, y) = il6(input(x, y)) * 2 + input(x + 1, y) * 3
```

However, a more reliable way of generating the same `vmpa` instruction is by using Halide built-ins for HVX:

```
hvx_builtin(Int(16), "add_2mpy.vub.vub.b.b", {input(x, y), input(x + 1,
↪ y), 2, 3});
```

The `hvx_builtin` function is provided for embedding specific HVX instructions into the Halide pipeline. This gives you fine-grained control over code generation by the compiler, while also providing all the benefits of scheduling directives.

For a complete example, see the `conv3x3` example in the Halide SDK (Halide/Examples/offload/apps/hexagon_benchmarks/conv3x3_generator.cpp). Run the generator by setting the `GeneratorParam use_builtins` to `true`.

Note: `hvx_builtin` is supported only for the `hvx` target feature.

3.4.1 Supported Halide HVX built-ins

This section includes a table that lists all the supported Halide HVX built-ins and corresponding HVX instructions generated by the Halide compiler.

Some HVX instructions produce interleaved outputs while some instructions expect deinterleaved inputs. When using these built-ins, do not be concerned about interleaving outputs or deinterleaving inputs. For example, a widening multiply followed by a store to memory results in the interleaving of the result of the multiply before the store. Thus, the following expression:

```
hvx_builtin(Int(16), "mpy.vb.vb", {i8_1, i8_2})
```

Is lowered to this Halide Intermediate Representation (IR):

```
(int16x128)halide.hexagon.interleave.vh((int16x128)halide.hexagon.mpy.
↪vb.vb(i8_1, i8_2))
```

And subsequently generates the following assembly:

```
v7:6.h = vmpy(v6.b, v7.b)
v3:2 = vshuff(v7, v6, r6)
```

Any stage using `hvx_builtin` must be vectorized.

Apart from the following table, there are two more built-ins: `make_interleave` and `make_concat`, which are used to interleave and concatenate vectors respectively. They can accept any number of arguments of the same type and must have return types that are the same as the argument types. These built-ins are meant to be used only inside arguments for other `hvx_builtins`.

Types ending in `v` indicate that a vector is required. For example: `i32` is a scalar type, while `i32v` is a vector of `i32` elements.

Per the following table, to multiply a 16-bit signed value and a 16-bit unsigned value and vectorize them, use one of the following approaches:

```
// i(x) is of type i16 and j(x) is of type u16.
f(x) = i(x) * j(x)
f.vectorize(x, 64);
```

or

```
f(x) = hvx_builtin(Int(32), "mpy.vh.vuh", {i(x), j(x)});
f.vectorize(x, 64);
```

Both approaches should generate a `vmpy (vx.h, vy.uh)` HVX instruction.

HVX built-in name	HVX instruction	Ret type	Argument types
<code>zxt.vub</code>	<code>vzb</code>	<code>u16v</code>	<code>u8v</code>
<code>zxt.vuh</code>	<code>vzh</code>	<code>u32v</code>	<code>u16v</code>
<code>sxt.vb</code>	<code>vsb</code>	<code>i16v</code>	<code>i8v</code>
<code>sxt.vh</code>	<code>vsh</code>	<code>i32v</code>	<code>i16v</code>
<code>unpack.vub</code>	<code>vunpackub</code>	<code>u16v</code>	<code>u8v</code>
<code>unpack.vuh</code>	<code>vunpackuh</code>	<code>u32v</code>	<code>u16v</code>
<code>unpack.vb</code>	<code>vunpackb</code>	<code>i16v</code>	<code>i8v</code>
<code>unpack.vh</code>	<code>vunpackh</code>	<code>i32v</code>	<code>i16v</code>
<code>trunc.vh</code>	<code>vshuffeb</code>	<code>i8v</code>	<code>i16v</code>
<code>trunc.vw</code>	<code>vshufeh</code>	<code>i16v</code>	<code>i32v</code>
<code>trunclo.vh</code>	<code>vshuffob</code>	<code>i8v</code>	<code>i16v</code>
<code>trunclo.vw</code>	<code>vshufoh</code>	<code>i16v</code>	<code>i32v</code>
<code>trunc_satub.vh</code>	<code>vsathub</code>	<code>u8v</code>	<code>i16v</code>
<code>trunc_sath.vw</code>	<code>vsatwh</code>	<code>i16v</code>	<code>i32v</code>
<code>trunc_satuh.vuw</code>	<code>vsatuwuh</code>	<code>u16v</code>	<code>u32v</code>
<code>trunc_satub_rnd.vh</code>	<code>vroundhub</code>	<code>u8v</code>	<code>i16v</code>
<code>trunc_satb_rnd.vh</code>	<code>vroundhb</code>	<code>i8v</code>	<code>i16v</code>
<code>trunc_satub_rnd.vuh</code>	<code>vrounduhub</code>	<code>u8v</code>	<code>u16v</code>
<code>trunc_satuh_rnd.vw</code>	<code>vroundwuh</code>	<code>u16v</code>	<code>i32v</code>
<code>trunc_sath_rnd.vw</code>	<code>vroundwh</code>	<code>i16v</code>	<code>i32v</code>
<code>trunc_satuh_rnd.vuw</code>	<code>vrounduwuh</code>	<code>u16v</code>	<code>u32v</code>
<code>pack_satub.vh</code>	<code>vpackhub_sat</code>	<code>u8v</code>	<code>i16v</code>
<code>pack_satuh.vw</code>	<code>vpackwuh_sat</code>	<code>u16v</code>	<code>i32v</code>
<code>pack_satb.vh</code>	<code>vpackhb_sat</code>	<code>i8v</code>	<code>i16v</code>
<code>pack_sath.vw</code>	<code>vpackwh_sat</code>	<code>i16v</code>	<code>i32v</code>
<code>pack.vh</code>	<code>vpackeb</code>	<code>i8v</code>	<code>i16v</code>
<code>pack.vw</code>	<code>vpackeh</code>	<code>i16v</code>	<code>i32v</code>
<code>packhi.vh</code>	<code>vpackob</code>	<code>i8v</code>	<code>i16v</code>
<code>packhi.vw</code>	<code>vpackoh</code>	<code>i16v</code>	<code>i32v</code>
<code>add_vuh.vub.vub</code>	<code>vaddubh</code>	<code>u16v</code>	<code>u8v, u8v</code>

continues on next page

Table 1 – continued from previous page

HVX built-in name	HVX instruction	Ret	Argument types type
add_vw.vh.vh	vaddhw	i32v	i16v, i16v
add_vuw.vuh.vuh	vadduhw	u32v	u16v, u16v
sub_vh.vub.vub	vsububh	i16v	u8v, u8v
sub_vw.vh.vh	vsubhw	i32v	i16v, i16v
sub_vw.vuh.vuh	vsubuhw	i32v	u16v, u16v
sat_add.vub.vub	vaddubsat	u8v	u8v, u8v
sat_add.vuh.vuh	vadduhsat	u16v	u16v, u16v
sat_add.vuw.vuw	vadduwsat	u32v	u32v, u32v
sat_add.vh.vh	vaddhsat	i16v	i16v, i16v
sat_add.vw.vw	vaddwsat	i32v	i32v, i32v
sat_add.vub.vub.dv	vaddubsat_dv	u8v	u8v, u8v
sat_add.vuh.vuh.dv	vadduhsat_dv	u16v	u16v, u16v
sat_add.vuw.vuw.dv	vadduwsat_dv	u32v	u32v, u32v
sat_add.vh.vh.dv	vaddhsat_dv	i16v	i16v, i16v
sat_add.vw.vw.dv	vaddwsat_dv	i32v	i32v, i32v
sat_sub.vub.vub	vsububsat	i8v	u8v, u8v
sat_sub.vuh.vuh	vsubuhsat	i16v	u16v, u16v
sat_sub.vh.vh	vsubhsat	i16v	i16v, i16v
sat_sub.vw.vw	vsubwsat	i32v	i32v, i32v
sat_sub.vub.vub.dv	vsububsat_dv	i8v	u8v, u8v
sat_sub.vuh.vuh.dv	vsubuhsat_dv	i16v	u16v, u16v
sat_sub.vh.vh.dv	vsubhsat_dv	i16v	i16v, i16v
sat_sub.vw.vw.dv	vsubwsat_dv	i32v	i32v, i32v
abs.vh	vabsh	u16v	i16v
abs.vw	vabsw	u32v	i32v
abs.vb	vabsb	u8v	i8v
absd.vub.vub	vabsdiffub	u8v	u8v, u8v
absd.vuh.vuh	vabsdiffuh	u16v	u16v, u16v
absd.vh.vh	vabsdiffh	u16v	i16v, i16v
absd.vw.vw	vabsdiffw	u32v	i32v, i32v
avg.vub.vub	vavgub	u8v	u8v, u8v
avg.vuh.vuh	vavguh	u16v	u16v, u16v
avg.vuw.vuw	vavguw	u32v	u32v, u32v
avg.vb.vb	vavgb	i8v	i8v, i8v
avg.vh.vh	vavgv	i16v	i16v, i16v
avg.vw.vw	vavgw	i32v	i32v, i32v
avg_rnd.vub.vub	vavgubrnd	u8v	u8v, u8v
avg_rnd.vuh.vuh	vavguhrnd	u16v	u16v, u16v
avg_rnd.vuw.vuw	vavgwurnd	u32v	u32v, u32v
avg_rnd.vb.vb	vavgbrnd	i8v	i8v, i8v
avg_rnd.vh.vh	vavgvrnd	i16v	i16v, i16v
avg_rnd.vw.vw	vavgwrnd	i32v	i32v, i32v
navg.vub.vub	vnavgub	i8v	u8v, u8v
navg.vb.vb	vnavgb	i8v	i8v, i8v
navg.vh.vh	vnavgv	i16v	i16v, i16v

continues on next page

Table 1 – continued from previous page

HVX built-in name	HVX instruction	Ret	Argument types type
navg.vw.vw	vnavgw	i32v	i32v, i32v
mul.vh.vh	vmpyih	i16v	i16v, i16v
mul.vh.b	vmpyihb	i16v	i16v, i8
mul.vw.h	vmpyihw	i32v	i32v, i16
mul.vw.b	vmpyiwb	i32v	i32v, i8
add_mul.vh.vh.vh	vmpyih_acc	i16v	i16v, i16v, i16v
add_mul.vh.vh.b	vmpyihb_acc	i16v	i16v, i16v, i8
add_mul.vw.vw.h	vmpyihw_acc	i32v	i32v, i32v, i16
add_mul.vw.vw.b	vmpyiwb_acc	i32v	i32v, i32v, i8
mpy.vub.vub	vmpyubv	u16v	u8v, u8v
mpy.vuh.vuh	vmpyuhv	u32v	u16v, u16v
mpy.vb.vb	vmpybv	i16v	i8v, i8v
mpy.vh.vh	vmpyhv	i32v	i16v, i16v
add_mpy.vuh.vub.vub	vmpyubv_acc	u16v	u16v, u8v, u8v
add_mpy.vuw.vuh.vuh	vmpyuhv_acc	u32v	u32v, u16v, u16v
add_mpy.vh.vb.vb	vmpybv_acc	i16v	i16v, i8v, i8v
add_mpy.vw.vh.vh	vmpyhv_acc	i32v	i32v, i16v, i16v
mpy.vub.vb	vmpybusv	i16v	u8v, i8v
mpy.vh.vuh	vmpyhus	i32v	i16v, u16v
add_mpy.vh.vub.vb	vmpybusv_acc	i16v	i16v, u8v, i8v
add_mpy.vw.vh.vuh	vmpyhus_acc	i32v	i32v, i16v, u16v
mpy.vub.ub	vmpyub	u16v	u8v, u8
mpy.vuh.uh	vmpyuh	u32v	u16v, u16
mpy.vh.h	vmpyh	i32v	i16v, i16
mpy.vub.b	vmpybus	i16v	u8v, i8
add_mpy.vuh.vub.ub	vmpyub_acc	u16v	u16v, u8v, u8
add_mpy.vuw.vuh.uh	vmpyuh_acc	u32v	u32v, u16v, u16
add_mpy.vh.vub.b	vmpybus_acc	i16v	i16v, u8v, i8
satw_add_mpy.vw.vh. h	vmpyhsat_acc	i32v	i32v, i16v, i16
add_4mpy.vub.vub	vrmpyubv	u32v	u8v, u8v
add_4mpy.vb.vb	vrmpybv	i32v	i8v, i8v
add_4mpy.vub.vb	vrmpybusv	i32v	u8v, i8v
acc_add_4mpy.vuw. vub.vub	vrmpyubv_acc	u32v	u32v, u8v, u8v
acc_add_4mpy.vw.vb. vb	vrmpybv_acc	i32v	i32v, i8v, i8v
acc_add_4mpy.vw. vub.vb	vrmpybusv_acc	i32v	i32v, u8v, i8v
add_2mpy.vub.b	vdmpybus	i16v	u8v, i32
add_2mpy.vh.b	vdmpyhb	i32v	i16v, i32
acc_add_2mpy.vh. vub.b	vdmpybus_acc	i16v	i16v, u8v, i32
acc_add_2mpy.vw.vh. b	vdmpyhb_acc	i32v	i32v, i16v, i32

continues on next page

Table 1 – continued from previous page

HVX built-in name	HVX instruction	Ret	Argument types type
add_2mpy.vh.h	vdmpyhsat	i32v	i16v, i32
add_2mpy.vh.uh	vdmpyhsusat	i32v	i16v, u32
add_2mpy.vh.vh	vdmpyhvsat	i32v	i16v, i16v
add_3mpy.vub.b	vtmpybus	i16v	u8v, i32
add_3mpy.vb.b	vtmpyb	i16v	i8v, i32
add_3mpy.vh.b	vtmpyhb	i32v	u16v, i32
acc_add_3mpy.vh.vub.b	vtmpybus_acc	i16v	i16v, u8v, i32
acc_add_3mpy.vh.vb.b	vtmpyb_acc	i16v	i16v, i8v, i32
acc_add_3mpy.vw.vh.b	vtmpyhb_acc	i32v	i32v, u16v, i32
add_4mpy.vub.b	vrmpybus	i32v	u8v, i32
add_4mpy.vub.ub	vrmpyub	u32v	u8v, u32
acc_add_4mpy.vw.vub.b	vrmpybus_acc	i32v	i32v, u8v, i32
acc_add_4mpy.vuw.vub.ub	vrmpyub_acc	u32v	u32v, u8v, u32
trunc_satw_mpy2_rnd.vh.vh	vmpyhvsrs	i16v	i16v, i16v
trunc_satw_mpy2.vh.h	vmpyhss	i16v	i16v, i16
trunc_satw_mpy2_rnd.vh.h	vmpyhsrc	i16v	i16v, i16
max.vub.vub	vmaxub	u8v	u8v, u8v
max.vuh.vuh	vmaxuh	u16v	u16v, u16v
max.vh.vh	vmaxh	i16v	i16v, i16v
max.vw.vw	vmaxw	i32v	i32v, i32v
max.vhf.vhf	vmax_hf	f16v	f16v, f16v
max.vsf.vsf	vmax_sf	f32v	f32v, f32v
min.vub.vub	vminub	u8v	u8v, u8v
min.vuh.vuh	vminuh	u16v	u16v, u16v
min.vh.vh	vminh	i16v	i16v, i16v
min.vw.vw	vminw	i32v	i32v, i32v
min.vhf.vhf	vmin_hf	f16v	f16v, f16v
min.vsf.vsf	vmin_sf	f32v	f32v, f32v
shr.vuh.vh	vlsrhv	u16v	u16v, u16v
shr.vuw.vw	vlsrwv	u32v	u32v, u32v
shr.vh.vh	vasrhv	i16v	i16v, u16v
shr.vw.vw	vasrwv	i32v	i32v, u32v
trunc_satub_shr_rnd.vh	vasrhubrnds	u8v	i16v, u16
trunc_satb_shr_rnd.vh	vasrhbrnds	i8v	i16v, u16

continues on next page

Table 1 – continued from previous page

HVX built-in name	HVX instruction	Ret	Argument types type
trunc_satub_shr_rnd.vuh	vasruhbrndsatsat	u8v	u16v, u16
trunc_satuh_shr_rnd.vw	vasrwuhrndsatsat	u16v	i32v, u32
trunc_sath_shr_rnd.vw	vasrwhrndsatsat	i16v	i32v, u32
trunc_satuh_shr_rnd.vuw	vasruwuhrndsatsat	u16v	u32v, u32
shl.vuh.vh	vaslhv	u16v	u16v, u16v
shl.vuw.vw	vaslwv	u32v	u32v, u32v
shl.vh.vh	vaslhv	i16v	i16v, u16v
shl.vw.vw	vaslwv	i32v	i32v, u32v
shr.vuh.h	vlsrh	u16v	u16v, u16
shr.vuw.w	vlsrw	u32v	u32v, u32
shr.vh.h	vasrh	i16v	i16v, u16
shr.vw.w	vasrw	i32v	i32v, u32
shl.vuh.h	vaslh	u16v	u16v, u16
shl.vuw.w	vaslw	u32v	u32v, u32
shl.vh.h	vaslh	i16v	i16v, u16
shl.vw.w	vaslw	i32v	i32v, u32
add_shr.vh.vh.uh	vasrh_acc	i16v	i16v, i16v, i16
add_shl.vh.vh.uh	vaslh_acc	i16v	i16v, i16v, i16
add_shr.vw.vw.uw	vasrw_acc	i32v	i32v, i32v, i32
add_shl.vw.vw.uw	vaslw_acc	i32v	i32v, i32v, i32
trunc_shr.vw.uw	vasrwh	i16v	i32v, u32
trunc_satub_shr.vh.uh	vasrhubsatsat	u8v	i16v, u16
trunc_satuh_shr.vw.uw	vasrwuhsatsat	u16v	i32v, u32
trunc_sath_shr.vw.uw	vasrwhsatsat	i16v	i32v, u32
vrwr	vrwr	u8v	u8v, i32
cls.vh	vnormamth	u16v	u16v
cls.vw	vnormamtw	u32v	u32v
mul.vw.vw	Custom Halide implementation: vmpyieoh + vmpyiewuh_acc	i32v	i32v, i32v
mul.vw.vh	Custom Halide implementation: vaslw + vmpyiowh + vmpyiowh	i32v	i32v, i16v
mul.vw.vuh	Custom Halide implementation: vlsrw + vmpyiewuh + vmpyiewuh	i32v	i32v, u16v
mul.vuw.vuh	Custom Halide implementation: vshufeh + vshufoh + vmpyuhv + vmpyuhv + vaslw	u32v	u32v, u16v

continues on next page

Table 1 – continued from previous page

HVX built-in name	HVX instruction	Ret	Argument types
mul.vuw.vuw	Custom Halide implementation: vshufef + vshufef + vshufef + vshufef + vmpyuhv + vmpyuhv + vmpyuhv_acc	u32	u32v, u32v
trunc_mpy.vw.vw	Custom Halide implementation: vmpyewuh + vmpyowh_sacc + vasrw	i32v	i32v, i32v
trunc_satdw_mpy2.vw.vw	Custom Halide implementation: vmpyewuh + vmpyowh_sacc	i32v	i32v, i32v
trunc_satdw_mpy2_rnd.vw.vw	Custom Halide implementation: vmpyewuh + vmpyowh_rnd_sacc	i32v	i32v, i32v
shl.vub.b	Custom Halide implementation: vzb + vaslh + vaslh + vshuffefb	i8v	u8v, i8
shl.vb.b	Custom Halide implementation: vzb + vaslh + vaslh + vshuffefb	i8v	i8v, i8
shr.vub.b	Custom Halide implementation: vlsrh + vlsrh + vshuffefb	i8v	u8v, i8
shr.vb.b	Custom Halide implementation: vasrh + vasrh + vshuffefb	i8v	i8v, i8
shl.vub.vb	Custom Halide implementation: vzb + vsb + vaslhv + vaslhv + vshuffefb	i8v	u8v, i8v
shl.vb.vb	Custom Halide implementation: shl.vub.vb``	i8v	i8v, i8v
shr.vub.vb	Custom Halide implementation: vzb + vsb + vlsrhv + vlsrhv + vshuffefb	i8v	u8v, i8v
shr.vb.vb	Custom Halide implementation: vsb + vsb + vasrhv + vasrhv + vshuffefb	i8v	i8v, i8v
add_2mpy.vub.vub.b.b	Custom Halide implementation: vmpabus	i16v	u8v, u8v, i8, i8
acc_add_2mpy.vh.vub.vub.b.b	Custom Halide implementation: vmpabus_acc	i16v	i16v, u8v, u8v, i8, i8
add_2mpy.vh.vh.b.b	Custom Halide implementation: vmpahb	i32v	i16v, i16v, i8, i8
acc_add_2mpy.vw.vh.vh.b.b	Custom Halide implementation: vmpahb_acc	i32v	i32v, i16v, i16v, i8, i8
trunc_satuh.vw	Custom Halide implementation: vasrwuhsat	i16v	i32v
vtmpy.vub.vub.b.b	Custom Halide implementation: vtmpybus	i16v	u8v, u8v, i8, i8

continues on next page

Table 1 – continued from previous page

HVX built-in name	HVX instruction			Ret	Argument types
					type
vtmpy.vb.vb.b.b	Custom	Halide	implementation:	i16v	i8v, i8v, i8, i8
	vtmpyb				
vtmpy.vh.vh.b.b	Custom	Halide	implementation:	i32v	i16v, i16v, i8, i8
	vtmpyhb				
vrmpy_odd.vub.vub.w	Custom	Halide	implementation:	i32v	u8v, u8v, i32
	vrmpybusi				with immediate 1
acc_vrmpy_odd.vw.vub.vub.w	Custom	Halide	implementation:	i32v	i32v, u8v, u8v, i32
	vrmpybusi_acc				with immediate 1
vrmpy_even.vub.vub.w	Custom	Halide	implementation:	i32v	u8v, u8v, i32
	vrmpybusi				with immediate 0
acc_vrmpy_even.vw.vub.vub.w	Custom	Halide	implementation:	i32v	i32v, u8v, u8v, i32
	vrmpybusi_acc				with immediate 0
add_4mpy.vub.b.stencil	Custom	Halide	implementation:	i32v	u8v, i32
	vrmpybusi				+ vrmpybusi
add_4mpy.vub.ub.stencil	Custom	Halide	implementation:	u32	u8v, u32
	vrmpyubi				+ vrmpyubi

3.4.2 Usage and examples for HVX built-ins

Usage:

```
hvx_builtin(<return type such as Int(8), UInt(8), Int(16),.....>
            <builtin_name>, <vector of arguments with correct types>)
```

Use the following Halide expressions to interpret the table:

```
Expr i8_1 = in_i8(x), i8_2 = in_i8(x + 16), i8_3 = in_i8(x + 32), i8_4_
↳ in_i8(x + 48);
Expr u8_1 = in_u8(x), u8_2 = in_u8(x + 16), u8_3 = in_u8(x + 32), u8_4_
↳ in_u8(x + 48);
Expr i16_1 = in_i16(x), i16_2 = in_i16(x + 16), i16_3 = in_i16(x + 32);
Expr u16_1 = in_u16(x), u16_2 = in_u16(x + 16), u16_3 = in_u16(x + 32);
Expr i32_1 = in_i32(x), i32_2 = in_i32(x + 16), i32_3 = in_i32(x + 32);
Expr u32_1 = in_u32(x), u32_2 = in_u32(x + 16), u32_3 = in_u32(x + 32);

Expr u8_4x_0 = in_u8(rfac * x + 0);
Expr u8_4x_1 = in_u8(rfac * x + 1);
Expr u8_4x_2 = in_u8(rfac * x + 2);
Expr u8_4x_3 = in_u8(rfac * x + 3);
Expr u8_4x_32 = in_u8(rfac * x + 32 + 0);
Expr u8_4x_33 = in_u8(rfac * x + 32 + 1);
```

(continues on next page)

(continued from previous page)

```

Expr u8_4x_34 = in_u8(rfac * x + 32 + 2);
Expr u8_4x_35 = in_u8(rfac * x + 32 + 3);
Expr i8_4x_0 = in_i8(rfac * x + 0);
Expr i8_4x_1 = in_i8(rfac * x + 1);
Expr i8_4x_2 = in_i8(rfac * x + 2);
Expr i8_4x_3 = in_i8(rfac * x + 3);
Expr i8_4x_32 = in_i8(rfac * x + 32 + 0);
Expr i8_4x_33 = in_i8(rfac * x + 32 + 1);
Expr i8_4x_34 = in_i8(rfac * x + 32 + 2);
Expr i8_4x_35 = in_i8(rfac * x + 32 + 3);
Expr tmp = hvx_builtin(UInt(8), "make_interleave", {u8_4x_0, u8_4x_1,
↳ u8_4x_2, u8_4x_3});
Expr tmp_32 = hvx_builtin(UInt(8), "make_interleave", {u8_4x_32, u8_4x_
↳ 33, u8_4x_34, u8_4x_35});
Expr tmp2 = hvx_builtin(Int(8), "make_interleave", {i8_4x_0, i8_4x_1,
↳ i8_4x_2, i8_4x_3});
Expr tmp34 = hvx_builtin(Int(8), "make_interleave", {i8_4x_32, i8_4x_
↳ 33, i8_4x_34, i8_4x_35});

```

The following table shows sample uses for `hvx_builtin`.

Assembl	Halide expr.	HVX built-in equivalent
<code>vror(V: in_u8(:</code> <code>r)</code>	<code>in_u8(rfac * x + 5) % 128</code>	<code>hvx_builtin(UInt(8), "vror.ub", {u8_1, 5})</code>
<code>vror(V: in_u16</code> <code>r)</code>	<code>in_u16(rfac * x + 7) % 64</code>	<code>hvx_builtin(UInt(16), "vror.uh", {u8_1, 7})</code>
<code>vror(V: in_u32</code> <code>r)</code>	<code>in_u32(rfac * x + 9) % 32</code>	<code>hvx_builtin(UInt(32), "vror.uw", {u8_1, 9})</code>
<code>vror(V: in_i8(:</code> <code>r)</code>	<code>in_i8(rfac * x + 2) % 128</code>	<code>hvx_builtin(Int(8), "vror.b", {i8_1, 2})</code>

continues on next page

Table 2 – continued from previous page

Assembly	Halide expr.	HVX built-in equivalent
<code>vrwr(V: in_i16 r)</code>	<code>/ 64 + (x + 3) % 64)</code>	<code>hvx_builtin(Int(16), "vrwr.h", {i8_1, 3})</code>
<code>vrwr(V: in_i32 r)</code>	<code>/ 32 + (x + 4) % 32)</code>	<code>hvx_builtin(Int(32), "vrwr.w", {i8_1, 4})</code>
<code>vunpackl u16(u8_1 ub)</code>		<code>hvx_builtin(UInt(16), "unpack.vub", {u8_1})</code>
<code>vunpackl i16(u8_1 ub)</code>		<code>i16(hvx_builtin(UInt(16), "unpack.vub", {u8_1}))</code>
<code>vunpackl u32(u16_1 uh)</code>		<code>hvx_builtin(UInt(32), "unpack.vuh", {u16_1})</code>
<code>vunpackl i32(u16_1 uh)</code>		<code>i32(hvx_builtin(UInt(32), "unpack.vuh", {u16_1}))</code>
<code>vunpackl u16(i8_1 b)</code>		<code>u16(hvx_builtin(Int(16), "unpack.vb", {i8_1}))</code>
<code>vunpackl i16(i8_1 b)</code>		<code>hvx_builtin(Int(16), "unpack.vb", {i8_1})</code>
<code>vunpackl u32(i16_1 h)</code>		<code>u32(hvx_builtin(Int(32), "unpack.vh", {i16_1}))</code>
<code>vunpackl i32(i16_1 h)</code>		<code>hvx_builtin(Int(32), "unpack.vh", {i16_1})</code>
<code>vunpackl u32(u8_1 ub)</code>		<code>hvx_builtin(UInt(32), "unpack.vub", {u8_1})</code>
<code>vunpackl i32(u8_1 ub)</code>		<code>hvx_builtin(Int(32), "unpack.vub", {u8_1})</code>
<code>vunpackl u32(i8_1 b)</code>		<code>u32(hvx_builtin(Int(16), "unpack.vb", {i8_1}))</code>
<code>vunpackl i32(i8_1 b)</code>		<code>hvx_builtin(Int(32), "unpack.vb", {i8_1})</code>
<code>Vx. h = vadd(V: u16(u8_1 ub, Vz. ub)</code>	<code>u16(u8_1 hvx_builtin(UInt(16), "add_vuh.vub.vub", {u8_1, u8_2}) + vadd(V: u16(u8_1 ub, Vz. ub)</code>	

continues on next page

Table 2 – continued from previous page

Assembly	Halide expr.	HVX built-in equivalent
Vx. w = vadd(V, uh, Vz. uh)	u32(u16_1 + u16_2) u32(u16_1 + u16_2)	hvx_builtin(UInt(32), "add_vuw.vuh.vuh", {u16_1, u16_2})
Vx. w = vadd(V, h, Vz. h)	i32(i16_1 + i16_2) i32(i16_1 + i16_2)	hvx_builtin(Int(32), "add_vw.vh.vh", {i16_1, i16_2})
vadd(V, ub, Vy. ub):sat	u8_sat(u8_1 + u8_2) u16(u8_1 + u8_2)	hvx_builtin(UInt(8), "sat_add.vub.vub", {u8_1, u8_2})
vadd(V, uh, Vy. uh):sat	u16_sat(u16_1 + u16_2) u32(u16_1 + u16_2)	hvx_builtin(UInt(16), "sat_add.vuh.vuh", {u16_1, u16_2})
vadd(V, h, Vy. h):sat	i16_sat(i16_1 + i16_2) i32(i16_1 + i16_2)	hvx_builtin(Int(16), "sat_add.vh.vh", {i16_1, i16_2})
vadd(V, w, Vy. w):sat	i32_sat(i32_1 + i32_2) i64(i32_1 + i32_2)	hvx_builtin(Int(32), "sat_add.vw.vw", {i32_1, i32_2})
vadd(V, uw, Vy. uw):sat	u32_sat(u32_1 + u32_2) u64(u32_1 + u32_2)	hvx_builtin(UInt(32), "sat_add.vuw.vuw", {u32_1, u32_2})
Vx. h = vsub(V, ub, Vz. ub)	u16(u8_1 - u8_2) u16(u8_1 - u8_2)	hvx_builtin(Int(16), "sub_vh.vub.vub", {u8_1, u8_2})
Vxx. h = vsub(V, ub, Vz. ub)	i16(u8_1 - u8_2) i16(u8_1 - u8_2)	hvx_builtin(Int(16), "sub_vh.vub.vub", {u8_1, u8_2})

continues on next page

Table 2 – continued from previous page

Assembly	Halide expr.	HVX built-in equivalent
Vx. w = vsub(V, uh, Vz. uh)	u32(u16_1 - u16_2))	u32(hvx_builtin(Int(32), "sub_vw.vuh.vuh", {u16_1, u16_2}))
Vxx. w = vsub(V, uh, Vz. uh)	i32(u16_1 - u16_2))	u32(hvx_builtin(Int(32), "sub_vw.vuh.vuh", {u16_1, u16_2}))
Vx. w = vsub(V, h, Vz. h)	i32(i16_1 - i16_2))	u32(hvx_builtin(Int(32), "sub_vw.vh.vh", {i16_1, i16_2}))
vsub(V, ub, Vy. ub):sat	u8_sat(u8_1 - u8_2))	u8(hvx_builtin(Int(8), "sat_sub.vub.vub", {u8_1, u8_2}))
vsub(V, uh, Vy. uh):sat	u16_sat(u16_1 - u16_2))	u16(hvx_builtin(Int(16), "sat_sub.vuh.vuh", {u16_1, u16_2}))
vsub(V, h, Vy. h):sat	i16_sat(i16_1 - i16_2))	u16(hvx_builtin(Int(16), "sat_sub.vh.vh", {i16_1, i16_2}))
vsub(V, w, Vy. w):sat	i32_sat(i32_1 - i32_2))	u32(hvx_builtin(Int(32), "sat_sub.vw.vw", {i32_1, i32_2}))
vadd(V, ub, Vyy. ub):sat	u8_sat(u8_1 + u8_2))	u8(hvx_builtin(UInt(8), "sat_add.vub.vub.dv", {u8_1, u8_2}))
vadd(V, uh, Vyy. uh):sat	u16_sat(u16_1 + u16_2))	u16(hvx_builtin(UInt(16), "sat_add.vuh.vuh.dv", {u16_1, u16_2}))
vadd(V, h, Vy. h):sat	i16_sat(i16_1 + i16_2))	u32(hvx_builtin(UInt(16), "sat_add.vh.vh.dv", {i16_1, i16_2}))

continues on next page

Table 2 – continued from previous page

Assembly	Halide expr.	HVX built-in equivalent
vadd(V: i32_sat w, Vyy. i64(i32 w):sat	+	hvx_builtin(Int(32), "sat_add.vw.vw.dv", {i32_1, i32_2})
vadd(V: u32_sat uw, Vyy. u64(u32 uw):sat	+	hvx_builtin(UInt(32), "sat_add.vuw.vuw.dv", {u32_1, u32_2})
vsub(V: u8_sat ub, Vyy. i16(u8_ ub):sat	-	u8(hvx_builtin(Int(8), "sat_sub.vub.vub.dv", {u8_1, u8_2}))
vsub(V: u16_sat uh, Vyy. i32(u16_ uh):sat	-	u16(hvx_builtin(Int(16), "sat_sub.vuh.vuh.dv", {u16_1, u16_2}))
vsub(V: i16_sat h, Vyy. i32(i16_ h):sat	-	hvx_builtin(Int(16), "sat_sub.vh.vh.dv", {i16_1, i16_2})
vsub(V: i32_sat w, Vyy. i64(i32_ w):sat	-	hvx_builtin(Int(32), "sat_sub.vw.vw.dv", {i32_1, i32_2})
vavg(V: u8((u16_ ub, Vy. u16(u8_ ub) / 2)	+	hvx_builtin(UInt(8), "avg.vub.vub", {u8_1, u8_2})
vavg(V: u8((u16_ ub, Vy. u16(u8_ ub):rnd + 1) / 2)	+	hvx_builtin(UInt(8), "avg_rnd.vub.vub", {u8_1, u8_2})
vavg(V: u16((u16_ uh, Vy. u32(u16_ uh) / 2)	+	hvx_builtin(UInt(16), "avg.vuh.vuh", {u16_1, u16_2})
vavg(V: u16((u16_ uh, Vy. u32(u16_ uh):rnd + 1) / 2)	+	hvx_builtin(UInt(16), "avg_rnd.vuh.vuh", {u16_1, u16_2})

continues on next page

Table 2 – continued from previous page

Assembly	Halide expr.	HVX built-in equivalent
vavg(V: i16((i16_1, i16_2)), h, Vy. + h)	$\frac{i32(i16_1) + i32(i16_2)}{2}$	<code>hvx_builtin(Int(16), "avg.vh.vh", {i16_1, i16_2})</code>
vavg(V: i16((i16_1, i16_2)), h, Vy. + h):rnd	$\frac{i32(i16_1) + i32(i16_2) + 1}{2}$	<code>hvx_builtin(Int(16), "avg_rnd.vh.vh", {i16_1, i16_2})</code>
vavg(V: i32((i32_1, i32_2)), w, Vy. + w)	$\frac{i64(i32_1) + i64(i32_2)}{2}$	<code>hvx_builtin(Int(32), "avg.vw.vw", {i32_1, i32_2})</code>
vavg(V: i32((i32_1, i32_2)), w, Vy. + w):rnd	$\frac{i64(i32_1) + i64(i32_2) + 1}{2}$	<code>hvx_builtin(Int(32), "avg_rnd.vw.vw", {i32_1, i32_2})</code>
vavg(V: i8((i8_1, i8_2)), b, Vy. + b)	$\frac{i16(i8_1) + i16(i8_2)}{2}$	<code>hvx_builtin(Int(8), "avg.vb.vb", {i8_1, i8_2})</code>
vavg(V: u32((u32_1, u32_2)), uw, + Vy. u64(u32_1, u32_2) uw)	$\frac{u64(u32_1) + u64(u32_2)}{2}$	<code>hvx_builtin(UInt(32), "avg.vuw.vuw", {u32_1, u32_2})</code>
vnavg(V: i8((i8_1, i8_2)), ub, - Vy. i16(u8_1, u8_2) ub)	$\frac{i16(u8_1) - i16(u8_2)}{2}$	<code>hvx_builtin(Int(8), "navg.vub.vub", {u8_1, u8_2})</code>
vnavg(V: i16((i16_1, i16_2)), h, Vy. - h)	$\frac{i32(i16_1) - i32(i16_2)}{2}$	<code>hvx_builtin(Int(16), "navg.vh.vh", {i16_1, i16_2})</code>
vnavg(V: i32((i32_1, i32_2)), w, Vy. - w)	$\frac{i64(i32_1) - i64(i32_2)}{2}$	<code>hvx_builtin(Int(32), "navg.vw.vw", {i32_1, i32_2})</code>
vavg(V: i8((i8_1, i8_2)), b, Vy. + b)	$\frac{i16(i8_1) + i16(i8_2)}{2}$	<code>hvx_builtin(Int(8), "avg.vb.vb", {i8_1, i8_2})</code>

continues on next page

Table 2 – continued from previous page

Assembly	Halide expr.	HVX built-in equivalent
vavg(V: i8_1 b, Vy. b):rnd + 1) / 2)		hvx_builtin(Int(8), "avg_rnd.vb.vb", {i8_1, i8_2})
vavg(V: u32_1 uw, Vy. uw) + u64(u32_2) / 2)		hvx_builtin(UInt(32), "avg.vuw.vuw", {u32_1, u32_2})
vavg(V: u32_1 uw, Vy. uw):rnd + 1) / 2)		hvx_builtin(UInt(32), "avg_rnd.vuw.vuw", {u32_1, u32_2})
vnavg(V: i8_1 b, Vy. b) - i16(i8_2) / 2)		hvx_builtin(Int(8), "navg.vb.vb", {i8_1, i8_2})
vlsr(V: u8_1 h, Vy. h) >> (u8_2 % 8)		u8(hvx_builtin(UInt(16), "shr.vuh.vh", {u16(u8_1), u16(u8_2) % 8}))
vlsr(V: u16_1 h, Vy. h) >> (u16_2 % 16)		hvx_builtin(UInt(16), "shr.vuh.vh", {u16_1, u16_2 % 16})
vlsr(V: u32_1 w, Vy. w) >> (u32_2 % 32)		hvx_builtin(UInt(32), "shr.vuw.vw", {u32_1, u32_2 % 32})
vasr(V: i8_1 h, Vy. h) >> (u8_2 % 8)		i8(hvx_builtin(Int(16), "shr.vh.vh", {i16(i8_1), u16(u8_2 % 8)}))
vasr(V: i16_1 h, Vy. h) >> (u16_2 % 16)		hvx_builtin(Int(16), "shr.vh.vh", {i16_1, u16_2 % 16})
vasr(V: i32_1 w, Vy. w) >> (u32_2 % 32)		hvx_builtin(Int(32), "shr.vw.vw", {i32_1, u32_2 % 32})

continues on next page

Table 2 – continued from previous page

Assembl ¹	Halide expr.	HVX built-in equivalent
vasr(V: u8_sat h, Vy.h, r):sat		hvx_builtin(UInt(8), "trunc_satub_shr.vh.uh", {i16_1, >> 4}) 4))
vasr(V: u16_sat w, Vy.w, r):sat		hvx_builtin(UInt(16), "trunc_satuh_shr.vw.uw", {i32_1, >> 8}) 8))
vasr(V: i16_sat w, Vy.w, r):sat		hvx_builtin(Int(16), "trunc_sath_shr.vw.uw", {i32_1, >> 8}) 8))
vasr(V: i16(i32_1 w, Vy. w, r)	>> 8)	hvx_builtin(Int(16), "trunc_shr.vw.uw", {i32_1, 8}) >> 8)
vasl(V: u8_1 h, Vy. h)	<< (u8_2 % 8)	u8(hvx_builtin(UInt(16), "shl.vuh.vh", {u8_1, u8_2 % 8}))
vasl(V: u16_1 h, Vy. h)	<< (u16_2 % 16)	hvx_builtin(UInt(16), "shl.vuh.vh", {u16_1, u16_2 % 16}))
vasl(V: u32_1 w, Vy. w)	<< (u32_2 % 32)	hvx_builtin(UInt(32), "shl.vuw.vw", {u32_1, u32_2 % 32}))
vasl(V: i8_1 h, Vy. h)	<< (u8_2 % 8)	i8(hvx_builtin(Int(16), "shl.vh.vh", {i8_1, u8_2 % 8})) << (u8_2 % 8)
vasl(V: i16_1 h, Vy. h)	<< (u16_2 % 16)	hvx_builtin(Int(16), "shl.vh.vh", {i16_1, u16_2 % 16}) << (u16_2 % 16)
vasl(V: i32_1 w, Vy. w)	<< (u32_2 % 32)	hvx_builtin(Int(32), "shl.vw.vw", {i32_1, u32_2 % 32}) << (u32_2 % 32)
vlsr(V: u8_1 h, Vy. h)	>> (i8_2 % 16 - 8)	u8(hvx_builtin(UInt(16), "shr.vuh.vh", {u8_1, u16(i8_2 % 16 - 8) }))) >> (i8_2 % 16 - 8)

continues on next page

Table 2 – continued from previous page

Assembly	Halide expr.	HVX built-in equivalent
vlsr(V: u16_1 h, Vy. h)	>> (i16_2 % 32 - 16)	hvx_builtin(UInt(16), "shr.vuh.vh", {u16_1, u16(i16_2 % 32 - 16)})
vlsr(V: u32_1 w, Vy. w)	>> (i32_2 % 64 - 32)	hvx_builtin(UInt(32), "shr.vuw.vw", {u32_1, u32(i32_2 % 64 - 32)})
vasr(V: i8_1 h, Vy. h)	>> (i8_2 % 16 - 8)	i8(hvx_builtin(Int(16), "shr.vh.vh", {i8_1, u16(i8_2 % 16 - 8)}))
vasr(V: i16_1 h, Vy. h)	>> (i16_2 % 32 - 16)	hvx_builtin(Int(16), "shr.vh.vh", {i16_1, u16(i16_2 % 32 - 16)})
vasr(V: i32_1 w, Vy. w)	>> (i32_2 % 64 - 32)	hvx_builtin(Int(32), "shr.vw.vw", {i32_1, u32(i32_2 % 64 - 32)})
vasl(V: u8_1 h, Vy. h)	<< (i8_2 % 16 - 8)	u8(hvx_builtin(UInt(16), "shl.vuh.vh", {u8_1, u16(i8_2 % 16 - 8)}))
vasl(V: u16_1 h, Vy. h)	<< (i16_2 % 32 - 16)	hvx_builtin(UInt(16), "shl.vuh.vh", {u16_1, u16(i16_2 % 32 - 16)})
vasl(V: u32_1 w, Vy. w)	<< (i32_2 % 64 - 32)	hvx_builtin(UInt(32), "shl.vuw.vw", {u32_1, u32(i32_2 % 64 - 32)})
vasl(V: i8_1 h, Vy. h)	<< (i8_2 % 16 - 8)	i8(hvx_builtin(Int(16), "shl.vh.vh", {i8_1, u16(i8_2 % 16 - 8)}))

continues on next page

Table 2 – continued from previous page

Assembly	Halide expr.	HVX built-in equivalent
vasl(V: i16_1 h, Vy. h)	<< (i16_2 % 32 - 16)	hvx_builtin(Int(16), "shl.vh.vh", {i16_1, u16(i16_2 % 32 - 16)})
vasl(V: i32_1 w, Vy. w)	<< (i32_2 % 64 - 32)	hvx_builtin(Int(32), "shl.vw.vw", {i32_1, u32(i32_2 % 64 - 32)})
vlsr(V: u8_1 uh, r)	>> (u8(y) % 8)	u8(hvx_builtin(UInt(16), "shr.vuh.h", {u8_1, u8(y) % 8}))
vlsr(V: u16_1 uh, r)	>> (u16(y) % 16)	hvx_builtin(UInt(16), "shr.vuh.h", {u16_1, u16(y) % 16})
vlsr(V: u32_1 uw, r)	>> (u32(y) % 32)	hvx_builtin(UInt(32), "shr.vuw.w", {u32_1, u32(y) % 32})
vasr(V: i8_1 h, r)	>> (u8(y) % 8)	i8(hvx_builtin(Int(16), "shr.vh.h", {i8_1, u8(y) % 8}))
vasr(V: i16_1 h, r)	>> (u16(y) % 16)	hvx_builtin(Int(16), "shr.vh.h", {i16_1, u16(y) % 16})
vasr(V: i32_1 w, r)	>> (u32(y) % 32)	hvx_builtin(Int(32), "shr.vw.w", {i32_1, u32(y) % 32})
vasl(V: u8_1 h, r)	<< (u8(y) % 8)	u8(hvx_builtin(UInt(16), "shl.vuh.h", {u8_1, u8(y) % 8}))
vasl(V: u16_1 h, r)	<< (u16(y) % 16)	hvx_builtin(UInt(16), "shl.vuh.h", {u16_1, u16(y) % 16})

continues on next page

Table 2 – continued from previous page

Assembly	Halide expr.	HVX built-in equivalent
vasl(V: u32_1 w, r)	<< (u32(y) % 32)	hvx_builtin(UInt(32), "shl.vuw.w", {u32_1, u32(y) % 32})
vasl(V: i8_1 h, r)	<< (u8(y) % 8)	i8(hvx_builtin(Int(16), "shl.vh.h", {i8_1, u8(y) % 8}))
vasl(V: i16_1 h, r)	<< (u16(y) % 16)	hvx_builtin(Int(16), "shl.vh.h", {i16_1, u16(y) % 16})
vasl(V: i32_1 w, r)	<< (u32(y) % 32)	hvx_builtin(Int(32), "shl.vw.w", {i32_1, u32(y) % 32})
vlsr(V: u8_1 uh, r)	>> (i8(y) % 16 - 8)	u8(hvx_builtin(UInt(16), "shr.vuh.h", {u8_1, u8(i8(y) % 16 - 8)}))
vlsr(V: u16_1 uh, r)	>> (i16(y) % 32 - 16)	hvx_builtin(UInt(16), "shr.vuh.h", {u16_1, u16(i16(y) % 32 - 16)})
vlsr(V: u32_1 uw, r)	>> (i32(y) % 64 - 32)	hvx_builtin(UInt(32), "shr.vuw.w", {u32_1, u32(i32(y) % 64 - 32)})
vasr(V: i8_1 h, r)	>> (i8(y) % 16 - 8)	i8(hvx_builtin(Int(16), "shr.vh.h", {i8_1, u8(i8(y) % 16 - 8)}))
vasr(V: i16_1 h, r)	>> (i16(y) % 32 - 16)	hvx_builtin(Int(16), "shr.vh.h", {i16_1, u16(i16(y) % 32 - 16)})

continues on next page

Table 2 – continued from previous page

Assembly	Halide expr.	HVX built-in equivalent
vasr(V: i32_1 w, r)	>> (i32(y) % 64 - 32)	hvx_builtin(Int(32), "shr.vw.w", {i32_1, u32(i32(y) % 64 - 32)})
vasl(V: u8_1 h, r)	<< (i8(y) % 16 - 8)	u8(hvx_builtin(UInt(16), "shl.vuh.h", {u8_1, u8(i8(y) % 16 - 8)}))
vasl(V: u16_1 h, r)	<< (i16(y) % 32 - 16)	hvx_builtin(UInt(16), "shl.vuh.h", {u16_1, u16(i16(y) % 32 - 16)})
vasl(V: u32_1 w, r)	<< (i32(y) % 64 - 32)	hvx_builtin(UInt(32), "shl.vuw.w", {u32_1, u32(i32(y) % 64 - 32)})
vasl(V: i8_1 h, r)	<< (i8(y) % 16 - 8)	i8(hvx_builtin(Int(16), "shl.vh.h", {i8_1, u8(i8(y) % 16 - 8)}))
vasl(V: i16_1 h, r)	<< (i16(y) % 32 - 16)	hvx_builtin(Int(16), "shl.vh.h", {i16_1, u16(i16(y) % 32 - 16)})
vasl(V: i32_1 w, r)	<< (i32(y) % 64 - 32)	hvx_builtin(Int(32), "shl.vw.w", {i32_1, u32(i32(y) % 64 - 32)})
vpacke h, Vy. h)	u8(u16_ u8(hvx_builtin(Int(8), "pack.vh", {i16(u16_1)}))	
vpacke h, Vy. h)	u8(i16_ u8(hvx_builtin(Int(8), "pack.vh", {i16_1})))	
vpacke h, Vy. h)	i8(u16_ hvx_builtin(Int(8), "pack.vh", {i16(u16_1)}))	

continues on next page

Table 2 – continued from previous page

Assembly	Halide expr.	HVX built-in equivalent
vpacke h, Vy. h)	i8(i16_	hvx_builtin(Int(8), "pack.vh", {i16_1})
vpacke w, Vy. w)	u16(u32_	u16(hvx_builtin(Int(16), "pack.vw", {i32(u32_1)}))
vpacke w, Vy. w)	u16(i32_	u16(hvx_builtin(Int(16), "pack.vw", {i32_1}))
vpacke w, Vy. w)	i16(u32_	hvx_builtin(Int(16), "pack.vw", {i32(u32_1)})
vpacke w, Vy. w)	i16(i32_	hvx_builtin(Int(16), "pack.vw", {i32_1})
vpacko h, Vy. >> 8) h)	u8(u16_	u8(hvx_builtin(Int(8), "packhi.vh", {i16(u16_1)}))
vpacko h, Vy. >> 8) h)	u8(i16_	u8(hvx_builtin(Int(8), "packhi.vh", {i16_1}))
vpacko h, Vy. >> 8) h)	i8(u16_	hvx_builtin(Int(8), "packhi.vh", {i16(u16_1)})
vpacko h, Vy. >> 8) h)	i8(i16_	hvx_builtin(Int(8), "packhi.vh", {i16_1})
vpacko w, Vy. >> w) 16)	u16(u32_	u16(hvx_builtin(Int(16), "packhi.vw", {i32(u32_1)}))
vpacko w, Vy. >> w) 16)	u16(i32_	u16(hvx_builtin(Int(16), "packhi.vw", {i32_1}))
vpacko w, Vy. >> w) 16)	i16(u32_	hvx_builtin(Int(16), "packhi.vw", {i32(u32_1)})
vpacko w, Vy. >> w) 16)	i16(i32_	hvx_builtin(Int(16), "packhi.vw", {i32_1})
vshuffe b, Vy. * b) 127)	u8(u16_	u8(hvx_builtin(Int(8), "trunc.vh", {i16(u8_1) * 127}))
vshuffe b, Vy. * 63) b)	u8(i16_	u8(hvx_builtin(Int(8), "trunc.vh", {i16(i8_1) * 63}))

continues on next page

Table 2 – continued from previous page

Assembly	Halide expr.	HVX built-in equivalent
vshufflb, Vy. b)	i8(u16 * 127)	hvx_builtin(Int(8), "trunc.vh", {i16(u8_1) * 127})
vshufflb, Vy. b)	i8(i16 * 63)	hvx_builtin(Int(8), "trunc.vh", {i16(i8_1) * 63})
vshufflh, Vy. h)	u16(u32 * 32767)	u16(hvx_builtin(Int(16), "trunc.vw", {i32(u16_1) * 32767}))
vshufflh, Vy. h)	u16(i32 * 16383)	u16(hvx_builtin(Int(16), "trunc.vw", {i32(i16_1) * 16383}))
vshufflh, Vy. h)	i16(u32 * 32767)	hvx_builtin(Int(16), "trunc.vw", {i32(u16_1) * 32767})
vshufflh, Vy. h)	i16(i32 * 16383)	hvx_builtin(Int(16), "trunc.vw", {i32(i16_1) * 16383})
vshufflb, Vy. b)	u8((u16 * 127) >> 8)	u8(hvx_builtin(Int(8), "trunclo.vh", {i16(u8_1) * 127}))
vshufflb, Vy. b)	u8((i16 * 63) >> 8)	u8(hvx_builtin(Int(8), "trunclo.vh", {i16(i8_1) * 63}))
vshufflb, Vy. b)	i8((u16 * 127) >> 8)	hvx_builtin(Int(8), "trunclo.vh", {i16(u8_1) * 127})
vshufflb, Vy. b)	i8((i16 * 63) >> 8)	hvx_builtin(Int(8), "trunclo.vh", {i16(i8_1) * 63})
vshufflh, Vy. h)	u16((u32 * 32767) >> 16)	u16(hvx_builtin(Int(16), "trunclo.vw", {i32(u16_1) * 32767}))
vshufflh, Vy. h)	u16((i32 * 16383) >> 16)	u16(hvx_builtin(Int(16), "trunclo.vw", {i32(i16_1) * 16383}))

continues on next page

Table 2 – continued from previous page

Assembly	Halide expr.	HVX built-in equivalent
vshuffl h, Vy. h) >> 16)	i16((u * 32767) >> 16)	hvx_builtin(Int(16), "trunclo.vw", {i32(u16_1) * 32767})
vshuffl h, Vy. h) >> 16)	i16((i * 16383) >> 16)	hvx_builtin(Int(16), "trunclo.vw", {i32(i16_1) * 16383})
Vx. ub = vpack(h, Vz. h):sat	u8_sat	hvx_builtin(UInt(8), "pack_satub.vh", {i16_1})
Vx. b = vpack(h, Vz. h):sat	i8_sat	hvx_builtin(Int(8), "pack_satb.vh", {i16_1})
Vx. uh = vpack(w, Vz. w):sat	u16_sat	hvx_builtin(UInt(16), "pack_satuh.vw", {i32_1})
Vx. h = vpack(w, Vz. w):sat	i16_sat	hvx_builtin(Int(16), "pack_sath.vw", {i32_1})
Vx. ub = vsat(V h, Vz. h)	u8_sat << 1) 1})	hvx_builtin(UInt(8), "trunc_satub.vh", {i16(i8_1) << 1})
Vx. uh = vasr(V w, Vz.w, r):sat	u16_sat << 1) 8})	hvx_builtin(UInt(16), "trunc_satuh_shr.vw.uw", {i32, 8})

continues on next page

Table 2 – continued from previous page

Assembly	Halide expr.	HVX built-in equivalent
Vx. h = vsat (Vz. w, Vz. w)	i16_sat << 1)	hvx_builtin(Int(16), "trunc_sath.vw", {i32(i16_1) << 1})
Vx. ub = vpack (' h, Vz. h):sat	u8_sat	hvx_builtin(UInt(8), "pack_satub.vh", {hvx_builtin(Int(16), "pack_sath.vw", {i32_1})})
Vx. b = vpack (' h, Vz. h):sat	i8_sat	hvx_builtin(Int(8), "pack_satb.vh", {hvx_builtin(Int(16), "pack_sath.vw", {i32_1})})
Vx. h = vsat (Vz. w, Vz. w)	u8_sat << 8)	hvx_builtin(UInt(8), "pack_satub.vh", {hvx_builtin(Int(16), "trunc_sath.vw", {i32(i16_1) << 8})})
Vx. uh = vsat (Vz. uw, Vz. uw)	u16_sat	hvx_builtin(UInt(16), "trunc_satuh.vuw", {u32_1})
vround h, Vy. h)	u8_sat + 128) / 256)	hvx_builtin(UInt(8), "trunc_satub_rnd.vh", {i16_1})
vround h, Vy. h)	i8_sat + 128) / 256)	hvx_builtin(Int(8), "trunc_satb_rnd.vh", {i16_1})
vround uh, Vy. uh)	u8_sat + 128) / 256)	hvx_builtin(UInt(8), "trunc_satub_rnd.vuh", {u16_1})

continues on next page

Table 2 – continued from previous page

Assembly	Halide expr.	HVX built-in equivalent
vround w, Vy. w)	u16_sat + 32768) / 65536)	hvx_builtin(UInt(16), "trunc_satuh_rnd.vw", {i32_1})
vround w, Vy. w)	i16_sat + 32768) / 65536)	hvx_builtin(Int(16), "trunc_sath_rnd.vw", {i32_1})
vround w, Vy. w)	u16_sat + 32768) / 65536)	hvx_builtin(UInt(16), "trunc_satuh_rnd.vw", {i32_1})
vround w, Vy. w)	i16_sat + 32768) / 65536)	hvx_builtin(Int(16), "trunc_sath_rnd.vw", {i32_1})
vround uw, Vy. uw)	u16_sat + 32768) / 65536)	hvx_builtin(UInt(16), "trunc_satuh_rnd.vuw", {u32_1})
Vx. ub = vasr(V; h, Vz.h, r):rnd	u8_sat + 8) / 16)	hvx_builtin(UInt(8), "trunc_satub_shr_rnd.vh", {i16_1, 4})
Vx. b = vasr(V; h, Vz.h, r):rnd	i8_sat + 16) / 32)	hvx_builtin(Int(8), "trunc_satb_shr_rnd.vh", {i16_1, 5})
Vx. ub = vasr(V; uh, Vz. uh, r):rnd	u8_sat + 32) / 64)	hvx_builtin(UInt(8), "trunc_satub_shr_rnd.vuh", {u16_1, 6})

continues on next page

Table 2 – continued from previous page

Assembly	Halide expr.	HVX built-in equivalent
Vx. uh = vasr(V: w, Vz.w, r):rnd	u16_sat + 64) / 128)	hvx_builtin(UInt(16), "trunc_satuh_shr_rnd.vw", {i32_1, 7})
Vx. h = vasr(V: w, Vz.w, r):rnd	i16_sat + 8) / 256)	hvx_builtin(Int(16), "trunc_sath_shr_rnd.vw", {i32_1, 8})
Vx. uh = vasr(V: w, Vz.w, r):rnd	u16_sat + 9) / 512)	hvx_builtin(UInt(16), "trunc_satuh_shr_rnd.vw", {i32_1, 9})
Vx. h = vasr(V: w, Vz.w, r):rnd	i16_sat + 10) / 1024)	hvx_builtin(Int(16), "trunc_sath_shr_rnd.vw", {i32_1, 10})
Vx. uh = vasr(V: uw, Vz. uw, r):rnd	u16_sat + {u32_1, 11}) / 2048)	hvx_builtin(UInt(16), "trunc_satuh_shr_rnd.vuw", {u32_1, 11})
vmax(V: ub, Vy. ub)	max(u8_1, u8_2)	hvx_builtin(UInt(8), "max.vub.vub", {u8_1, u8_2})
vmax(V: uh, Vy. uh)	max(u16_1, u16_2)	hvx_builtin(UInt(16), "max.vuh.vuh", {u16_1, u16_2})
vmax(V: h, Vy. h)	max(i16_1, i16_2)	hvx_builtin(Int(16), "max.vh.vh", {i16_1, i16_2})
vmax(V: w, Vy. w)	max(i32_1, i32_2)	hvx_builtin(Int(32), "max.vw.vw", {i32_1, i32_2})

continues on next page

Table 2 – continued from previous page

Assembly	Halide expr.	HVX built-in equivalent
<code>vmin(V: min(u8_1, u8_2), ub, Vy. ub)</code>		<code>hvx_builtin(UInt(8), "min.vub.vub", {u8_1, u8_2})</code>
<code>vmin(V: min(u16_1, u16_2), uh, Vy. uh)</code>		<code>hvx_builtin(UInt(16), "min.vuh.vuh", {u16_1, u16_2})</code>
<code>vmin(V: min(i16_1, i16_2), h, Vy. h)</code>		<code>hvx_builtin(Int(16), "min.vh.vh", {i16_1, i16_2})</code>
<code>vmin(V: min(i32_1, i32_2), w, Vy. w)</code>		<code>hvx_builtin(Int(32), "min.vw.vw", {i32_1, i32_2})</code>
<code>vabsd(V: absd(u8_1, u8_2), ub, Vy. ub)</code>		<code>hvx_builtin(UInt(8), "absd.vub.vub", {u8_1, u8_2})</code>
<code>vabsd(V: absd(u16_1, u16_2), uh, Vy. uh)</code>		<code>hvx_builtin(UInt(16), "absd.vuh.vuh", {u16_1, u16_2})</code>
<code>vabsd(V: absd(i16_1, i16_2), h, Vy. h)</code>		<code>hvx_builtin(UInt(16), "absd.vh.vh", {i16_1, i16_2})</code>
<code>vabsd(V: absd(i32_1, i32_2), w, Vy. w)</code>		<code>hvx_builtin(UInt(32), "absd.vw.vw", {i32_1, i32_2})</code>
<code>vmpa(V: 5 * (i32(i16_1, i16_2, 5, 35)) h, r. b) + 7 * i32(i16_1, i16_2, 5, -35))</code>		<code>hvx_builtin(Int(32), "add_2mpy.vh.vh.b.b", {i16_1, i16_2, 5, 35})</code>
<code>vmpa(V: 5 * (i32(i16_1, i16_2, 5, -35)) h, r. b) - 7 * i32(i16_1, i16_2, 5, 35))</code>		<code>hvx_builtin(Int(32), "add_2mpy.vh.vh.b.b", {i16_1, i16_2, 5, -35})</code>
<code>vabs(V: abs(i16_1), h)</code>		<code>hvx_builtin(UInt(16), "abs.vh", {i16_1})</code>
<code>vabs(V: abs(i32_1), w)</code>		<code>hvx_builtin(UInt(32), "abs.vw", {i32_1})</code>
<code>vabs(V: abs(i8_1), b)</code>		<code>hvx_builtin(UInt(8), "abs.vb", {i8_1})</code>

continues on next page

Table 2 – continued from previous page

Assembly	Halide expr.	HVX built-in equivalent
vmpy(V: u16(u8_1, u8_2), ub, Vy. ub)	u16(u8_1, u8_2) * u16(u8_1, u8_2)	hvx_builtin(UInt(16), "mpy.vub.vub", {u8_1, u8_2})
vmpy(V: i16(i8_1, i8_2), b, Vy. b)	i16(i8_1, i8_2) * i16(i8_1, i8_2)	hvx_builtin(Int(16), "mpy.vb.vb", {i8_1, i8_2})
vmpy(V: u32(u16_1, u16_2), uh, Vy. uh)	u32(u16_1, u16_2) * u32(u16_1, u16_2)	hvx_builtin(UInt(32), "mpy.vuh.vuh", {u16_1, u16_2})
vmpy(V: i32(i16_1, i16_2), h, Vy. h)	i32(i16_1, i16_2) * i32(i16_1, i16_2)	hvx_builtin(Int(32), "mpy.vh.vh", {i16_1, i16_2})
vmpyi(V: i16_1, i16_2), h, Vy. h)	i16_1 * i16_2	hvx_builtin(Int(32), "mul.vh.vh", {i16_1, i16_2})
vmpyio(V: i32_1, i32_2), w, Vy. h)	i32_1 * i32_2	hvx_builtin(Int(32), "mul.vw.vh", {i32_1, i16_1})
vmpyie(V: i32_1, i32_2), w, Vy. uh)	i32_1 * i32_2	hvx_builtin(Int(32), "mul.vw.vuh", {i32_1, u16_1})
vmpy(V: u32_1, u32_2), uh, Vy. uh)	u32_1 * u32_2	hvx_builtin(UInt(32), "mul.vuw.vuw", {u32_1, u16_1})
vmpyie(V: i32_1, i32_2), h, Vy. h)	i32_1 * i32_2	hvx_builtin(Int(32), "mul.vw.vw", {i32_1, i32_2})
vmpy(V: i16(u8_1, u8_2), ub, Vy.b)	i16(u8_1, u8_2) * i16(u8_1, u8_2)	hvx_builtin(Int(16), "mpy.vub.vb", {u8_1, i8_2})
vmpy(V: i32(u16_1, u16_2), h, Vy. uh)	i32(u16_1, u16_2) * i32(u16_1, u16_2)	hvx_builtin(Int(32), "mpy.vh.vuh", {i16_2, u16_1})
vmpy(V: i16(i8_1, i8_2), ub, Vy.b)	i16(u8_1, u8_2) * i16(u8_1, u8_2)	hvx_builtin(Int(16), "mpy.vub.vb", {u8_2, i8_1})
vmpy(V: i32(i16_1, i16_2), h, Vy. uh)	i32(u16_1, u16_2) * i32(u16_1, u16_2)	hvx_builtin(Int(32), "mpy.vh.vuh", {i16_1, u16_2})
vmpy(V: i16(u8_1, u8_2), ub, r. b)	i16(u8_1, u8_2) * 3	hvx_builtin(Int(16), "mpy.vub.b", {u8_1, 3})

continues on next page

Table 2 – continued from previous page

Assembly	Halide expr.	HVX built-in equivalent
vmpy(V: i32(i16_1), h, r. * 10 h)		hvx_builtin(Int(32), "mpy.vh.h", {i16_1, 10})
vmpy(V: u16(u8_1), ub, r. * 3 ub)		hvx_builtin(UInt(16), "mpy.vub.ub", {u8_1, 3})
vmpy(V: u32(u16_1), uh, r. * 10 uh)		hvx_builtin(UInt(32), "mpy.vuh.uh", {u16_1, 10})
vmpy(V: 3 * ub, r. i16(u8_1) b)		hvx_builtin(Int(16), "mpy.vub.b", {u8_1, 3})
vmpy(V: 10 * h, r. i32(i16_1) h)		hvx_builtin(Int(32), "mpy.vh.h", {i16_1, 10})
vmpy(V: 3 * ub, r. u16(u8_1) ub)		hvx_builtin(UInt(16), "mpy.vub.ub", {u8_1, 3})
vmpy(V: 10 * uh, r. u32(u16_1) uh)		hvx_builtin(UInt(32), "mpy.vuh.uh", {u16_1, 10})
vmpyi(' i16_1 h, r. * 127 b)		hvx_builtin(Int(16), "mul.vh.b", {i16_1, 127})
vmpyi(' 127 * h, r. i16_1 b)		hvx_builtin(Int(16), "mul.vh.b", {i16_1, 127})
vmpyi(' i32_1 w, r. * h) 32767		hvx_builtin(Int(32), "mul.vw.h", {i32_1, 32767})
vmpyi(' 32767 w, r. * h) i32_1		hvx_builtin(Int(32), "mul.vw.h", {i32_1, 32767})
Vx. i16_1 h += + vmpyi(' i16_2 h, Vz. * h) i16_3		hvx_builtin(Int(16), "add_mul.vh.vh.vh", {i16_1, i16_2, i16_3})
Vx. i16_1 h += + vmpyi(' i16_2 h, r. * 127 b)		hvx_builtin(Int(16), "add_mul.vh.vh.b", {i16_1, i16_2, 127})

continues on next page

Table 2 – continued from previous page

Assembly	Halide expr.	HVX built-in equivalent
Vx. w += vmpyi('w, r. h)	i32_1 + i32_2 * 32767	hvx_builtin(Int(32), "add_mul.vw.vw.h", {i32_1, i32_2, 32767})
Vx. h += vmpyi('h, r. b)	i16_1 + 127 * i16_2	hvx_builtin(Int(16), "add_mul.vh.vh.b", {i16_1, i16_2, 127})
Vx. w += vmpyi('w, r. h)	i32_1 + 32767 * i32_2	hvx_builtin(Int(32), "add_mul.vw.vw.h", {i32_1, i32_2, 32767})
Vx. uh += vmpy(V: ub, Vz. ub)	u16_1 + u16(u8_1 * u8_2)	hvx_builtin(UInt(16), "add_mpy.vuh.vub.vub", {u16_1, u8_1, u8_2})
Vx. uw += vmpy(V: uh, Vz. uh)	u32_1 + u32(u16_1 * u16_2)	hvx_builtin(UInt(32), "add_mpy.vuw.vuh.vuh", {u32_1, u16_1, u16_2})
Vx. h += vmpy(V: b, Vz. b)	i16_1 + i16(i8_1 * i8_2)	hvx_builtin(Int(16), "add_mpy.vh.vb.vb", {i16_1, i8_1, i8_2})
Vx. w += vmpy(V: h, Vz. h)	i32_1 + i32(i16_1 * i16_2)	hvx_builtin(Int(32), "add_mpy.vw.vh.vh", {i32_1, i16_1, i16_2})
Vx. h += vmpy(V: ub, Vz.b)	i16_1 + i16(u8_1 * i8_2)	hvx_builtin(Int(16), "add_mpy.vh.vub.vb", {i16_1, u8_1, i8_2})

continues on next page

Table 2 – continued from previous page

Assembly	Halide expr.	HVX built-in equivalent
Vx. w += vmpy (V: h, Vz. uh)	i32_1 + i32 (i1 * i32 (u1	hvx_builtin(Int (32), "add_mpy.vw.vh.vuh", {i32_1, i16_1, u16_2})
Vx. h += vmpy (V: ub, Vz.b)	i16_1 + i16 (u8_ * i16 (i8_	hvx_builtin(Int (16), "add_mpy.vh.vub.vb", {i16_1, u8_1, i8_2})
Vx. w += vmpy (V: h, Vz. uh)	i32_1 + i32 (i1 * i32 (u1	hvx_builtin(Int (32), "add_mpy.vw.vh.vuh", {i32_1, i16_1, u16_2})
Vx. h += vmpy (V: ub, Vz.b)	i16_1 + i16 (i8_ * i16 (u8_	hvx_builtin(Int (16), "add_mpy.vh.vub.vb", {i16_1, u8_2, i8_1})
Vx. w += vmpy (V: h, Vz. uh)	i32_1 + i32 (u1 * i32 (i1	hvx_builtin(Int (32), "add_mpy.vw.vh.vuh", {i32_1, i16_2, u16_1})
Vx. h += vmpy (V: ub, Vz.b)	i16_1 + i16 (i8_ * i16 (u8_	hvx_builtin(Int (16), "add_mpy.vh.vub.vb", {i16_1, u8_2, i8_1})
Vx. w += vmpy (V: h, Vz. uh)	i32_1 + i32 (u1 * i32 (i1	hvx_builtin(Int (32), "add_mpy.vw.vh.vuh", {i32_1, i16_2, u16_1})
Vx. w += vmpy (V: h, r. h):sat	i32_1 + i32 (i1 * 32767	hvx_builtin(Int (32), "satw_add_mpy.vw.vh.h", {i32_1, i16_1, 32767})
Vx. w += vmpy (V: h, r. h):sat	i32_1 + 32767 * i32 (i1	hvx_builtin(Int (32), "satw_add_mpy.vw.vh.h", {i32_1, i16_1, 32767})

continues on next page

Table 2 – continued from previous page

Assembly	Halide expr.	HVX built-in equivalent
Vx. uh += vmpy(V: ub, r. ub)	u16_1 + u16(u8_ * 255	hvx_builtin(UInt(16), "add_mpy.vuh.vub.ub", {u16_1, u8_1, 255})
Vx. h += vmpy(V: ub, r. b)	i16_1 + i16(u8_ * 127	hvx_builtin(Int(16), "add_mpy.vh.vub.b", {i16_1, u8_1, 127})
Vx. uw += vmpy(V: uh, r. uh)	u32_1 + u32(u1_ * 65535	hvx_builtin(UInt(32), "add_mpy.vuw.vuh.uh", {u32_1, u16_1, 65535})
Vx. uh += vmpy(V: ub, r. ub)	u16_1 + 255 * u16(u8_ ub)	hvx_builtin(UInt(16), "add_mpy.vuh.vub.ub", {u16_1, u8_1, 255})
Vx. h += vmpy(V: ub, r. b)	i16_1 + 127 * i16(u8_ b)	hvx_builtin(Int(16), "add_mpy.vh.vub.b", {i16_1, u8_1, 127})
Vx. uw += vmpy(V: uh, r. uh)	u32_1 + 65535 * u32(u1_ uh)	hvx_builtin(UInt(32), "add_mpy.vuw.vuh.uh", {u32_1, u16_1, 65535})
Vx. h += vmpy(V: ub, r. b)	i16_1 + i16(u8_ * b) -127	hvx_builtin(Int(16), "add_mpy.vh.vub.b", {i16_1, u8_1, -127})
Vx. h += vmpyi('h, r. b)	i16_1 + i16_2 * b) -127	hvx_builtin(Int(16), "add_mul.vh.vh.b", {i16_1, i16_2, -127})
Vx. w += vmpy(V: h, r. h)	i32_1 + i32(i1_ * 32767	hvx_builtin(Int(32), "satw_add_mpy.vw.vh.h", {i32_1, i16_1, 32767})

continues on next page

Table 2 – continued from previous page

Assembly	Halide expr.	HVX built-in equivalent
Vx. w += vmpy (V: h, r. h)	i32_1 + 32767 * i32 (i1	hvx_builtin(Int (32), "satw_add_mpy.vw.vh.h", {i32_1, i16_1, 32767})
vmpy (V: h, Vy. h) :<<1	Factor: {1, 2} in i16_sat * i32 (i1 * factor + 16384) / 32768)	hvx_builtin(Int (16), "trunc_satw_mpy2_rnd.vh.vh", {i16_1, i16_2 * factor})
vmpyo (' w, Vy. h)	Factor: {1, 2} in i32 ((i * i64 (i3 * factor / (i64 (1 << 32))	hvx_builtin(Int (32), "trunc_mpy.vw.vw", {i32_1, i32_2 * factor})
vmpyo (' w, Vy. h) :<<1	Factor: {1, 2} in i32_sat * factor * i64 (i3 / (i64 (1 << 31))	hvx_builtin(Int (32), "trunc_satdw_mpy2.vw.vw", {i32_1 * factor, i32_2})

continues on next page

Table 2 – continued from previous page

Assembly	Halide expr.	HVX built-in equivalent
<pre>vmpyo('w, Vy. h) :<<1</pre>	<pre>Factor: {1, 2} in i32_sat * i64(i32 * factor + (1 << 30)) / (i64(1 << 31))</pre>	<pre>hvx_builtin(Int(32), "trunc_satdw_mpy2_rnd.vw.vw", {i32_1, i32_2 * factor})</pre>
<pre>vmpy(V: h, r. h) :<<1</pre>	<pre>Scalar: {32766, 32767} in i16_sat * scalar / 32768)</pre>	<pre>hvx_builtin(Int(16), "trunc_satw_mpy2.vh.h", {i16_1, scalar})</pre>
<pre>vmpy(V: h, r. h) :<<1</pre>	<pre>Scalar: {32766, 32767} in i16_sat * i32(i16 / 32768)</pre>	<pre>hvx_builtin(Int(16), "trunc_satw_mpy2.vh.h", {i16_1, scalar})</pre>
<pre>vmpy(V: h, r. h) :<<1</pre>	<pre>Scalar: {32766, 32767} in i16_sat * scalar + 16384) / 32768)</pre>	<pre>hvx_builtin(Int(16), "trunc_satw_mpy2_rnd.vh.h", {i16_1, scalar})</pre>

continues on next page

Table 2 – continued from previous page

Assembly	Halide expr.	HVX built-in equivalent
vmpy (V: h, r. h) :<<1	Scalar: {32766, 32767} in i16_sat * i32(i1 + 16384) / 32768)	hvx_builtin(Int(16), "trunc_satw_mpy2_rnd.vh.h", {i16_1, scalar})
vmpyo('w, Vy. h)	Scalar: {INT_M - 1, INT_MA in i32((i * scalar / (i64(1 << 32))	hvx_builtin(Int(32), "trunc_mpy.vw.vw", {i32_1, scalar})
vmpyo('w, Vy. h)	Scalar: {INT_M - 1, INT_MA in i32((s * i64(i3 / (i64(1 << 32))	hvx_builtin(Int(32), "trunc_mpy.vw.vw", {i32_2, scalar})

continues on next page

Table 2 – continued from previous page

Assembly	Halide expr.	HVX built-in equivalent
<pre> vmpyo('w, Vy. h) :<<1 </pre>	<pre> Scalar: {INT_M scalar} - 1, INT_MA in i32_sat * scalar / (i64(1 << 31)) </pre>	<pre> hvx_builtin(Int(32), "trunc_satdw_mpy2.vw.vw", {i32_1, scalar}) </pre>
<pre> vmpyo('w, Vy. h) :<<1 </pre>	<pre> Scalar: {INT_M scalar} - 1, INT_MA in i32_sat * i64(i3: / (i64(1 << 31)) </pre>	<pre> hvx_builtin(Int(32), "trunc_satdw_mpy2.vw.vw", {i32_2, scalar}) </pre>
<pre> vmpyo('w, Vy. h) :<<1 </pre>	<pre> Scalar: {INT_M {i32_1, scalar} - 1, INT_MA in i32_sat * scalar + (1 << 30)) / (i64(1 << 31)) </pre>	<pre> hvx_builtin(Int(32), "trunc_satdw_mpy2_rnd.vw.vw", {i32_1, scalar}) </pre>

continues on next page

Table 2 – continued from previous page

Assembly	Halide expr.	HVX built-in equivalent
<pre> vmpyo (w, Vy, h) :<<1 </pre>	<pre> Scalar: {INT_M - 1, INT_MA in i32_sat * i64(i32 + (1 << 30)) / (i64(1 << 31)) </pre>	<pre> hvx_builtin(Int(32), "trunc_satdw_mpy2_rnd.vw.vw", {i32_2, scalar}) </pre>
<pre> vmpa (V: ub, r. b) </pre>	<pre> i16(u8_ * 127 + i16(u8_ * -128 </pre>	<pre> hvx_builtin(Int(16), "add_2mpy.vub.vub.b.b", {u8_1, u8_2, 127, -128}) </pre>
<pre> vmpa (V: ub, r. b) </pre>	<pre> i16(u8_ * 127 + 126 * i16(u8_ </pre>	<pre> hvx_builtin(Int(16), "add_2mpy.vub.vub.b.b", {u8_1, u8_2, 127, 126}) </pre>
<pre> vmpa (V: ub, r. b) </pre>	<pre> -100 * i16(u8_ + 40 * i16(u8_ </pre>	<pre> hvx_builtin(Int(16), "add_2mpy.vub.vub.b.b", {u8_1, u8_2, -100, 40}) </pre>
<pre> Vx. h += vmpa (V: ub, r. b) </pre>	<pre> 2 * i16(u8_ + 3 * i16(u8_ + i16_1 </pre>	<pre> hvx_builtin(Int(16), "acc_add_2mpy.vh.vub.vub.b.b", {i16_1, u8_1, u8_2, 2, 3}) </pre>
<pre> vmpa (V: h, r. b) </pre>	<pre> i32(i16_ * 2 + i16_2, 2, 3) i32(i16_ * 3 </pre>	<pre> hvx_builtin(Int(32), "add_2mpy.vh.vh.b.b", {i16_1, i16_2, 2, 3}) </pre>

continues on next page

Table 2 – continued from previous page

Assembly	Halide expr.	HVX built-in equivalent
vmpa(V: i32(i16_1, h, r. b) * 2 + 3 * i32(i16_1, h, r. b))		hvx_builtin(Int(32), "add_2mpy.vh.vh.b.b", {i16_1, i16_2, 2, 3})
vmpa(V: 2 * i32(i16_1, h, r. b) + 3 * i32(i16_1, h, r. b))		hvx_builtin(Int(32), "add_2mpy.vh.vh.b.b", {i16_1, i16_2, 2, 3})
Vx. w += vmpa(V: i32(i16_1, h, r. b) + 3 * i32(i16_1, h, r. b))		hvx_builtin(Int(32), "acc_add_2mpy.vw.vh.vh.b.b", {i32_1, i16_1, i16_2, 2, 3})
vdmpy('ub, r. b) * 127 + i16(in_u8 * x + 1)) * -128		hvx_builtin(Int(16), "add_2mpy.vub.b", {hvx_builtin(UInt(8), "make_interleave", {in_u8(2 * x), in_u8(2 * x + 1)}), (int32_t)0x807F807F})
vdmpy('h, r. b) * 2 + i32(in_u8 * x + 1)) * 3		hvx_builtin(Int(32), "add_2mpy.vh.b", {hvx_builtin(Int(16), "make_interleave", {in_i16(2 * x), in_i16(2 * x + 1)}), (int32_t)0x03020302})
Vx. h += vdmpy('ub, r. b) * 120 + i16(in_u8 * x + 1)) * -50 + i16_1		hvx_builtin(Int(16), "acc_add_2mpy.vh.vub.b", {i16_1, hvx_builtin(UInt(8), "make_interleave", {in_u8(2 * x), in_u8(2 * x + 1)}), (int32_t)0xCE78CE78})

continues on next page

Table 2 – continued from previous page

Assembly	Halide expr.	HVX built-in equivalent
Vx. w += vdmpy('h, r. b)	i32(in_ * x)) 80 + i32(in_ * x + 1)) * 33 + i32_1	hvx_builtin(Int(32), "acc_add_2mpy.vw.vh.b", {i32_1, hvx_builtin(Int(16), "make_interleave", {in_i16(2 * x), in_i16(2 * x + 1)}), (int32_t)0x21502150})
vrmpy('ub, r. ub)	u32(u8_ * 255 + u32(u8_ * 254 + u32(u8_ * 253 + u32(u8_ * 252	hvx_builtin(UInt(32), "add_4mpy.vub.ub", {hvx_builtin(UInt(8), "make_interleave", {u8_1, u8_2, u8_3, u8_4}), Expr((uint32_t)0xFCFDFEFF)})
vrmpy('ub, r. b)	i32(u8_ * 127 + i32(u8_ * -128 + i32(u8_ * 126 + i32(u8_ * -127	hvx_builtin(Int(32), "add_4mpy.vub.b", {hvx_builtin(UInt(8), "make_interleave", {u8_1, u8_2, u8_3, u8_4}), (int32_t)0x817E807F})
Vx. uw += vrmpy('ub, r. ub)	u32_1 + u32(u8_ * 2 + u32(u8_ * 3 + u32(u8_ * 4 + u32(u8_ * 5	hvx_builtin(UInt(32), "acc_add_4mpy.vuw.vub.ub", {u32_1, hvx_builtin(UInt(8), "make_interleave", {u8_1, u8_2, u8_3, u8_4}), Expr((uint32_t)0x05040302)})

continues on next page

Table 2 – continued from previous page

Assembly	Halide expr.	HVX built-in equivalent
Vx. w += vrmpy('ub, r. b)	i32_1 + i32(u8_0 * 2 + i32(u8_0 * -3 + i32(u8_0 * -4 + i32(u8_0 * 5	hvx_builtin(Int(32), "acc_add_4mpy.vw.vub.b", {i32_1, hvx_builtin(UInt(8), "make_interleave", {u8_1, u8_2, u8_3, u8_4}), (int32_t)0x05FCFD02})
vrmpy('ub, r. b)	i32(u8_0 + i32(u8_0 * -2 + i32(u8_0 * 3 + i32(u8_0 * -4	hvx_builtin(Int(32), "add_4mpy.vub.b", {hvx_builtin(UInt(8), "make_interleave", {u8_1, u8_2, u8_3, u8_4}), (int32_t)0xFC03FE01})
Vx. w += vrmpy('ub, r. b)	i32_1 + i32(u8_0 * 2 + i32(u8_0 * 3 + i32(u8_0 * 4	hvx_builtin(Int(32), "acc_add_4mpy.vw.vub.b", {i32_1, hvx_builtin(UInt(8), "make_interleave", {u8_1, u8_2, u8_3, u8_4}), (int32_t)0x04030201})

continues on next page

Table 2 – continued from previous page

Assembly	Halide expr.	HVX built-in equivalent
vrmpy('u32(u1ub, r.ub)	* 255) + u32(u1ub) * 254) + u32(u1ub) * 253) + u32(u1ub) * 252)	hvx_builtin(UInt(32), "add_4mpy.vub.ub", {hvx_builtin(UInt(8), "make_interleave", {u8_1, u8_2, u8_3, u8_4}), Expr((uint32_t)0xFCFDFEFF)})
Vx. w += vrmpy('u32(i1ub, r.ub)	i32_1 + i32(i1ub) * 2) + i32(i1ub) * -3) + i32(i1ub) * -4) + i32(i1ub) * 5)	hvx_builtin(Int(32), "acc_add_4mpy.vw.vub.b", {i32_1, hvx_builtin(UInt(8), "make_interleave", {u8_1, u8_2, u8_3, u8_4}), (int32_t)0x05FCFD02})
vrmpy('u32(u8ub, Vy. ub)	u32(u8ub) * u8_1 + u32(u8ub) * u8_2 + u32(u8ub) * u8_3 + u32(u8ub) * u8_4	hvx_builtin(UInt(32), "add_4mpy.vub.vub", {hvx_builtin(UInt(8), "make_interleave", {u8_1, u8_2, u8_3, u8_4}), hvx_builtin(UInt(8), "make_interleave", {u8_1, u8_2, u8_3, u8_4})})

continues on next page

Table 2 – continued from previous page

Assembly	Halide expr.	HVX built-in equivalent
<pre> vrmpy('b, Vy. b) i32(i8_1 * i8_2 + i32(i8_1 * i8_3 + i32(i8_1 * i8_4 </pre>	<pre> i32(i8_1 * i8_2 + i32(i8_1 * i8_3 + i32(i8_1 * i8_4 </pre>	<pre> hvx_builtin(Int(32), "add_4mpy.vb.vb", {hvx_builtin(Int(8), "make_interleave", {i8_1, i8_2, i8_3, i8_4}), hvx_builtin(Int(8), "make_interleave", {i8_1, i8_2, i8_3, i8_4})}) </pre>
<pre> Vx. uw += vrmpy('ub, Vz. ub) u32(u8_1 * u8_2 + u32(u8_1 * u8_3 + u32(u8_1 * u8_4 </pre>	<pre> u32_1 + u32(u8_1 * u8_2 + u32(u8_1 * u8_3 + u32(u8_1 * u8_4 </pre>	<pre> hvx_builtin(UInt(32), "acc_add_4mpy.vuw.vub.vub", {u32_1, hvx_builtin(UInt(8), "make_interleave", {u8_1, u8_2, u8_3, u8_4}), hvx_builtin(UInt(8), "make_interleave", {u8_1, u8_2, u8_3, u8_4})}) </pre>

continues on next page

Table 2 – continued from previous page

Assembly	Halide expr.	HVX built-in equivalent
Vx. w += vrmpy(b,Vz. b)	i32_1 + i32(i8_ i8_3, i8_4} * i8_1 + i32(i8_ * i8_2 + i32(i8_ * i8_3 + i32(i8_ * i8_4	hvx_builtin(Int(32), "acc_add_4mpy.vw.vb.vb", {i32_1, hvx_builtin(Int(8), "make_interleave", {i8_1, i8_2, hvx_builtin(Int(8), "make_interleave", {i8_1, i8_2, i8_3, i8_4}))
vmpa(V: ub,r. b)	i16(u8_ * 127 + i16(u8_ * -126 + i16(u8_ * 125 + i16(u8_ * 124	hvx_builtin(Int(16), "acc_add_2mpy.vh.vub.vub.b.b", {hvx_builtin(Int(16), "add_2mpy.vub.vub.b.b", {u8_1, u8_2, 127, -126}), u8_3, u8_4, 125, 124})
Vx. w += vasl(V: w,r)	u32_1 + (u32_2 * 8)	u32(hvx_builtin(Int(32), "add_shl.vw.vw.uw", {i32(u32_1), i32(u32_2), 3}))
Vx. w += vasl(V: w,r)	i32_1 + (i32_2 * 8)	hvx_builtin(Int(32), "add_shl.vw.vw.uw", {i32_1, i32_2, 3})
Vx. w += vasr(V: w,r)	i32_1 + (i32_2 / 8)	hvx_builtin(Int(32), "add_shr.vw.vw.uw", {i32_1, i32_2, 3})

continues on next page

Table 2 – continued from previous page

Assembly	Halide expr.	HVX built-in equivalent
Vx. w += vasl(V, w, r)	i32_1 + (i32_2 << u32(y % 32))	hvx_builtin(Int(32), "add_shl.vw.vw.uw", {i32_1, i32_2, y % 32})
Vx. w += vasr(V, w, r)	i32_1 + (i32_2 >> u32(y % 32))	hvx_builtin(Int(32), "add_shr.vw.vw.uw", {i32_1, i32_2, y % 32})
Vx. h += vasl(V, h, r)	i16_1 + (i16_2 << u16(y % 16))	hvx_builtin(Int(16), "add_shl.vh.vh.uh", {i16_1, i16_2, i16(y % 16)})
Vx. h += vasr(V, h, r)	i16_1 + (i16_2 >> u16(y % 16))	hvx_builtin(Int(16), "add_shr.vh.vh.uh", {i16_1, i16_2, i16(y % 16)})
Vx. h += vasl(V, h, r)	u16_1 + (u16_2 * 16)	u16(hvx_builtin(Int(16), "add_shl.vh.vh.uh", {i16(u16_1), i16(u16_2), 4}))
Vx. h += vasl(V, h, r)	i16_1 + (i16_2 * 16)	hvx_builtin(Int(16), "add_shl.vh.vh.uh", {i16_1, i16_2, 4})
Vx. h += vasl(V, h, r)	u16_1 + (16 * u16_2)	u16(hvx_builtin(Int(16), "add_shl.vh.vh.uh", {i16(u16_1), i16(u16_2), 4}))
Vx. h += vasl(V, h, r)	i16_1 + (16 * i16_2)	hvx_builtin(Int(16), "add_shl.vh.vh.uh", {i16_1, i16_2, 4})

continues on next page

Table 2 – continued from previous page

Assembly	Halide expr.	HVX built-in equivalent
Vx. h += vasr(V, h, r)	i16_1 + (i16_2 / 16)	hvx_builtin(Int(16), "add_shr.vh.vh.uh", {i16_1, i16_2, 4})
vnormat h)	max(count_ count_	i16(hvx_builtin(UInt(16), "cls.vh", {u16(i16_1)}) + 1)
vnormat w)	max(count_ count_	i32(hvx_builtin(UInt(32), "cls.vw", {u32(i32_1)}) + 1)
Vx. uw = vrmpy('RDom ub, r. ub)	rfac = 4 and r(0, 4) in sum(u1 * x + r)))	u16(hvx_builtin(UInt(32), "add_4mpy.vub.ub", {tmp, Expr((uint32_t)0x01010101)}))
Vx. uw = vrmpy('RDom ub, r. ub)	rfac = 4 and r(0, 4) in sum(u1 * x + r)))	u16(hvx_builtin(UInt(32), "add_4mpy.vub.ub", {tmp, Expr((uint32_t)0x01010101)}))
Vx. uw = vrmpy('RDom ub, r. ub)	rfac = 4 and r(0, 4) in sum(u1 * x + r)) * u8(r))	u16(hvx_builtin(UInt(32), "add_4mpy.vub.ub", {tmp, Expr((uint32_t)0x03020100)}))
Vx. w = vrmpy('RDom ub, r. b)	rfac = 4 and r(0, 4) in sum(i1 * x + r)) * i8(r))	i16(hvx_builtin(Int(32), "add_4mpy.vub.b", {tmp, (int32_t)0x03020100}))

continues on next page

Table 2 – continued from previous page

Assembly	Halide expr.	HVX built-in equivalent
Vx. uw = vrmpy('RDom ub, Vz. ub)	rfac = 4 and r(0, 4) in sum(u1 * x + r)) * in_u8(* x + r + 32))	u16(hvx_builtin(UInt(32), "add_4mpy.vub.vub", {tmp, tmp_32}))
Vx. w = vrmpy('RDom ub, Vz.b)	rfac = 4 and r(0, 4) in sum(i1 * x + r)) * in_i8(* x + r + 32))	i16(hvx_builtin(Int(32), "add_4mpy.vub.vb", {tmp, tmp34}))
Vx. w = vrmpy('RDom b,Vz. b)	rfac = 4 and r(0, 4) in sum(i1 * x + r)) * in_i8(* x + r + 32))	i16(hvx_builtin(Int(32), "add_4mpy.vb.vb", {tmp2, tmp34}))
Vx. uw = vrmpy('RDom ub,r. ub)	rfac = 4 and r(0, 4) in sum(u3: * x + r)) * 34)	Expr(hvx_builtin(UInt(32), "add_4mpy.vub.ub", {tmp, (uint32_t)0x22222222}))

continues on next page

Table 2 – continued from previous page

Assembl	Halide expr.	HVX built-in equivalent
Vx. w = vrmpy(' RDom ub, r. b)	rfac = 4 and r(0, 4) in sum(i3: * x + r)) * (-1))	hvx_builtin(Int(32), "add_4mpy.vub.b", {tmp, (int32_t) 0xFFFFFFFF})
Vx. w = vrmpy(' RDom ub, r. b)	rfac = 4 and r(0, 4) in sum(i1: * x + r)) * (-1))	il6(hvx_builtin(Int(32), "add_4mpy.vub.b", {tmp, (int32_t) 0xFFFFFFFF}))
Vxx. uw = vrmpy(' RDom ub, r.ub, #z)	rfac = 4 and r(0, 4) in sum(u3: + r)))	hvx_builtin(UInt(32), "add_4mpy.vub.ub.stencil", {hvx_builtin(UInt(8), "make_concat", {u8_1, in_u8(x + 4))}, 0x01010101})
Vxx. uw = vrmpy(' RDom ub, r.ub, #z)	rfac = 4 and r(0, 4) in sum(u3: + r)) * 34)	hvx_builtin(UInt(32), "add_4mpy.vub.ub.stencil", {hvx_builtin(UInt(8), "make_concat", {u8_1, in_u8(x + 4))}, 0x22222222})
Vxx. w = vrmpy(' RDom ub, r.b, #z)	rfac = 4 and r(0, 4) in sum(u3: + r)) * i8(r))	hvx_builtin(Int(32), "add_4mpy.vub.b.stencil", {hvx_builtin(UInt(8), "make_concat", {u8_1, in_u8(x + 4))}, (int32_t) 0x03020100})

continues on next page

Table 2 – continued from previous page

Assembly	Halide expr.	HVX built-in equivalent
Vxx. w = vrmpy('ub, r.b, #z)	rfac = 4 and RDom r(0, 4) in sum(i3: + r)) * i8(-r)	hvx_builtin(Int(32), "add_4mpy.vub.b.stencil", {hvx_builtin(UInt(8), "make_concat", {u8_1, in_u8(x + 4))}, (int32_t)0xFDFEFFF00})
Vxx. w = vrmpy('ub, r.b, #z)	rfac = 4 and RDom r(0, 4) in sum(i3: + r)) * (-1))	hvx_builtin(Int(32), "add_4mpy.vub.b.stencil", {hvx_builtin(UInt(8), "make_concat", {u8_1, in_u8(x + 4))}, (int32_t)0xFFFFFFFF})
Vx. h = vdmpy('ub, r.b)	rfac = 2 and RDom r(0, 2) in sum(i1: * x + r2)) * 34)	hvx_builtin(Int(16), "add_2mpy.vub.b", {hvx_builtin(UInt(8), "make_interleave", {in_u8(2 * x), in_u8(2 * x + 1))}, (int32_t)0x22222222})
Vx. w = vdmpy('h, r. b)	rfac = 2 and RDom r(0, 2) in sum(i3: * x + r2)) * 15246)	hvx_builtin(Int(32), "add_2mpy.vh.b", {hvx_builtin(Int(16), "make_interleave", {in_i16(2 * x), in_i16(2 * x + 1))}, (int32_t)0x01010101}) * 15246

continues on next page

Table 2 – continued from previous page

Assembly	Halide expr.	HVX built-in equivalent
Vx. w = vdmpy('h, r. b)	rfac = 2 and RDom r(0, 2) in sum(i3: * x + r2)) * (-1246	hvx_builtin(Int(32), "add_2mpy.vh.b", {hvx_builtin(Int(16), "make_interleave", {in_i16(2 * x), in_i16(2 * x + 1)}), (int32_t)0x01010101}) * (-1246
Vx. w = vdmpy('h, r. b)	rfac = 2 and RDom r(0, 2) in sum(i3: * x + r2 + 2)) * (-1246	hvx_builtin(Int(32), "add_2mpy.vh.b", {hvx_builtin(Int(16), "make_interleave", {in_i16(2 * x + 2), in_i16(2 * x + 3)}), (int32_t)0x01010101}) * (-1246
vtmpy('b, r. b)	rfac = 3 and RDom r(0, 3) in sum(i1: + r3))	hvx_builtin(Int(16), "vtmpy.vb.vb.b.b", {i8_1, in_i8(x + 2), 1, 1})
vtmpy('ub, r.b)	rfac = 3 and RDom r(0, 3) in sum(i1: + r3))	hvx_builtin(Int(16), "vtmpy.vub.vub.b.b", {u8_1, in_u8(x + 2), 1, 1})
vtmpy('h, r. b)	rfac = 3 and RDom r(0, 3) in sum(i3: + r3))	hvx_builtin(Int(32), "vtmpy.vh.vh.b.b", {i16_1, in_i16(x + 2), 1, 1})

continues on next page

Table 2 – continued from previous page

Assembly	Halide expr.	HVX built-in equivalent
<code>Vx = vrord(V, 128, r)</code>	<code>in_u8(: hvx_builtin(UInt(8), "vrord.ub", {u8_1, 5}));</code>	<code>((x + 5) % 128)</code>
<code>Vx = vrord(V, 64, r)</code>	<code>in_u16 hvx_builtin(UInt(16), "vrord.uh", {u16_1, 6});</code>	<code>((x + 6) % 64)</code>
<code>Vx = vrord(V, 32, r)</code>	<code>in_u32 hvx_builtin(UInt(32), "vrord.uw", {u32_1, 7});</code>	<code>((x + 7) % 32)</code>
<code>Vx = vrord(V, 128, r)</code>	<code>in_i8(: hvx_builtin(Int(8), "vrord.b", {i8_1, 2});</code>	<code>((x + 2) % 128)</code>
<code>Vx = vrord(V, 64, r)</code>	<code>in_i16 hvx_builtin(Int(16), "vrord.h", {i16_1, 3});</code>	<code>((x + 3) % 64)</code>
<code>Vx = vrord(V, 32, r)</code>	<code>in_i32 hvx_builtin(Int(32), "vrord.w", {i32_1, 4});</code>	<code>((x + 4) % 32)</code>

3.5 Troubleshooting

3.5.1 User controllable exits

By default, when the Halide runtime detects an error, it does not trigger a crash dump. However, this default behavior can be altered by making a call to `halide_hexagon_set_error_fault_mask` in the application code before calling the Halide pipeline.

```
halide_hexagon_set_error_fault_mask(void *user_context, bool enable_
    ↪all_errors, int *lev_list);
// user_context      : Can be NULL
// enable_all_errors : boolean value which if set to true will trigger_
    ↪a crash dump for any error.
// lev_list          : Zero-terminated list of Halide error codes that_
    ↪will trigger a crash dump
//                    if encountered.
//                    Ignored if enable_all_errors is true.
```

The following example uses `halide_hexagon_set_error_fault_mask` to trigger a crash dump when Halide is out of memory.

```
int err_list[] = {halide_error_code_out_of_memory, 0};
halide_hexagon_set_error_fault_mask(NULL, false, err_list);
```

The following example uses `halide_hexagon_set_error_fault_mask` to trigger a crash dump for any error.

```
halide_hexagon_set_error_fault_mask(NULL, true, NULL);
```

Although this API is supported only in the two Offload modes (*Halide execution modes*), it does nothing in Simulator Offload mode and is only useful in Device Offload mode.

For more information on the error codes that Halide supports, see `halide_error_code_t` in `Halide.h` in your Halide installation.

3.5.2 Target malloc tracing

`halide_malloc` is a function in the Halide runtime that a pipeline calls when it must allocate memory for *Internal Buffers*. In Device Offload mode, calls to `halide_malloc` can be optionally traced by making a call to `halide_set_malloc_tracing`. In Simulator Offload mode, calling `halide_set_malloc_tracing` has no effect.

```
halide_hexagon_set_malloc_tracing(void *user_context, halide_hexagon_
    ↪malloc_trace_level_t)
// user_context      : Can be NULL
// tracing_level      : halide_hexagon_malloc_no_trace produces no_
    ↪malloc trace
//                    halide_hexagon_malloc_min_trace produces_
    ↪minimal trace
//                    halide_hexagon_malloc_max_trace produces very_
    ↪verbose trace
```

The Halide runtime does not ask the system for memory every time `halide_malloc` is called. It

maintains a pre-allocated list of memory buffers, and if a `halide_malloc` call can be satisfied by using one of these buffers, that buffer is returned. If, however, a buffer is not available, `halide_malloc` requests memory from the DSP (using the underlying `malloc`).

If the tracing level is set to `halide_hexagon_malloc_min_trace`, trace messages are produced only if `halide_malloc` calls the underlying system `malloc`. If the tracing level is set to `halide_hexagon_max_trace`, trace messages are produced for every single call to `halide_malloc`. `halide_hexagon_malloc_min_trace` is the recommended trace message level.

In Device Offload mode, `adb` can be used to view the tracing output. When you set the tracing level to `halide_hexagon_malloc_min_trace`, you will see the following output:

```
$> adb logcat | grep halide
01-09 20:11:24.169 4794 4794 I halide : HexagonMallocTraceLogger:
↳37101:Allocate(ptr=0xc805e900,size=65536)
01-09 20:11:24.170 4794 4794 I halide : HexagonMallocTraceLogger:
↳37101:Total_Memory(before=0,after=65536)
01-09 20:11:24.171 4794 4794 I halide : HexagonMallocTraceLogger:
↳28913:Allocate(ptr=0xc806e980,size=65536)
01-09 20:11:24.171 4794 4794 I halide : HexagonMallocTraceLogger:
↳28913:Total_Memory(before=65536,after=131072)
01-09 20:11:24.172 4794 4794 I halide : HexagonMallocTraceLogger:
↳37101:Allocate(ptr=0xc2880200,size=65536)
01-09 20:11:24.173 4794 4794 I halide : HexagonMallocTraceLogger:
↳37101:Total_Memory(before=131072,after=196608)
01-09 20:11:24.173 4794 4794 I halide : HexagonMallocTraceLogger:
↳28913:Allocate(ptr=0xc2890280,size=65536)
01-09 20:11:24.175 4794 4794 I halide : HexagonMallocTraceLogger:
↳28913:Total_Memory(before=196608,after=262144)
01-09 20:11:24.277 4794 4794 I halide : HexagonMallocTraceLogger:
↳28918:Free(ptr=0xc805e900,size=65536)
01-09 20:11:24.280 4794 4794 I halide : HexagonMallocTraceLogger:
↳28918:Total_Memory(before=262144,after=196608)
01-09 20:11:24.284 4794 4794 I halide : HexagonMallocTraceLogger:
↳28918:Free(ptr=0xc806e980,size=65536)
01-09 20:11:24.288 4794 4794 I halide : HexagonMallocTraceLogger:
↳28918:Total_Memory(before=196608,after=131072)
01-09 20:11:24.291 4794 4794 I halide : HexagonMallocTraceLogger:
↳28918:Free(ptr=0xc2880200,size=65536)
01-09 20:11:24.295 4794 4794 I halide : HexagonMallocTraceLogger:
↳28918:Total_Memory(before=131072,after=65536)
01-09 20:11:24.299 4794 4794 I halide : HexagonMallocTraceLogger:
↳28918:Free(ptr=0xc2890280,size=65536)
01-09 20:11:24.302 4794 4794 I halide : HexagonMallocTraceLogger:
↳28918:Total_Memory(before=65536,after=0)
```

3.5.3 Set heap grow size

When more space is required in Device Offload mode, specify the amount of memory to grow the heap by calling `halide_hexagon_mem_set_grow_size` before running a pipeline. For example, to grow the heap by at least 16 MB but no more than `MAX_INT32` (2 GB-1), do the following in the application-side (host-side) code.

```
const long long int grow_min = 16*1024*1024;
const long long int grow_max = MAX_INT32;
halide_hexagon_mem_set_grow_size(NULL, grow_min, grow_max);
```

The default value is equivalent to `halide_hexagon_mem_set_grow_size(0x100000/2, MAX_UINT64)`, which is a minimum of 512 KB and the largest allowed maximum.

3.5.4 Debug target feature

In the two Offload modes, use the debug target feature in your target when running a generator to compile your pipeline. This feature will produce debug messages that can be viewed using `adb`. For example, if the target when running your generator is set to `target=arm-64-android-hvx-debug`, you can view the debug message generated when running your application:

```
$> adb logcat | grep halide
01-13 12:02:23.091 13601 13601 I halide : halide_hexagon_device_and_
    ↳ host_malloc called.
01-13 12:02:23.091 13601 13601 I halide : Hexagon: halide_hexagon_
    ↳ device_malloc (user_context: 0x0, buf: 0x7fe84168c8)
01-13 12:02:23.091 13601 13601 I halide : allocating buffer of_
    ↳ 152 bytes
01-13 12:02:23.091 13601 13601 I halide : halide_malloc size=152 -
    ↳ >
01-13 12:02:23.091 13601 13601 I halide : 0x7a5364e0e0
01-13 12:02:23.091 13601 13601 I halide : host <- 0x7a5364e0e0
01-13 12:02:23.091 13601 13601 I halide : Time: 9.375000e-03 ms
01-13 12:02:23.091 13601 13601 I halide : halide_hexagon_device_and_
    ↳ host_malloc called.
01-13 12:02:23.091 13601 13601 I halide : Hexagon: halide_hexagon_
    ↳ device_malloc (user_context: 0x0, buf: 0x7fe8416918)
01-13 12:02:23.091 13601 13601 I halide : allocating buffer of_
    ↳ 256 bytes
01-13 12:02:23.091 13601 13601 I halide : halide_malloc size=256 -
    ↳ >
01-13 12:02:23.091 13601 13601 I halide : 0x7a536432a0
01-13 12:02:23.091 13601 13601 I halide : host <- 0x7a536432a0
01-13 12:02:23.091 13601 13601 I halide : Time: 8.906000e-03 ms
01-13 12:02:23.091 13601 13601 I halide : Hexagon: halide_hexagon_
    ↳ device_malloc (user_context: 0x0, buf: 0x7fe8416918)
01-13 12:02:23.091 13601 13601 I halide : Hexagon: halide_hexagon_
    ↳ copy_to_device (user_context: 0x0, buf: 0x7fe8416918)
01-13 12:02:23.091 13601 13601 I halide : Time: 1.875000e-03 ms
01-13 12:02:23.091 13601 13601 I halide : Hexagon: halide_hexagon_
```

(continues on next page)

(continued from previous page)

```

↪device_malloc (user_context: 0x0, buf: 0x7fe84168c8)
01-13 12:02:23.091 13601 13601 I halide : Hexagon: halide_hexagon_
↪copy_to_device (user_context: 0x0, buf: 0x7fe84168c8)
01-13 12:02:23.091 13601 13601 I halide :      Time: 3.640000e-04 ms
01-13 12:02:23.091 13601 13601 I halide : Hexagon: halide_hexagon_
↪device_malloc (user_context: 0x0, buf: 0x7fe8416e00)
01-13 12:02:23.091 13601 13601 I halide :      allocating buffer of ↪
↪14745728 bytes
01-13 12:02:23.091 13601 13601 I halide :      host_malloc↪
↪len=14745728 ->
01-13 12:02:23.091 13601 13601 I halide : host_malloc: Using_libion
01-13 12:02:23.091 13601 13601 I halide : host_malloc: ion_alloc_fd↪
↪succeeded
01-13 12:02:23.091 13601 13601 I halide :      0x7a4fdef000
01-13 12:02:23.091 13601 13601 I halide :      Time: 7.411500e-02 ms
01-13 12:02:23.091 13601 13601 I halide : Hexagon: halide_hexagon_run↪
↪(user_context: 0x0, state_ptr: 0x7a5362c060 (163624160), name:↪
↪offload_rpc.curved.s0.__outermost_argv, function: 0x64344128e0 (0))
01-13 12:02:23.091 13601 13601 I halide :      halide_hexagon_remote_
↪get_symbol offload_rpc.curved.s0.__outermost_argv

```

Information about memory allocation and other calls made to the Halide runtime can be gleaned when the debug target feature is used. However, this feature is not supported in the two Standalone modes.

3.5.5 Useful environment variables

Following are the important environment variables in Halide.

HL_HEXAGON_SIM_REMOTE

In Simulator Offload mode, this variable is required to be set. It is the path to the Hexagon simulator wrapper. For most use cases, set it to `Halide/bin/hexagon_sim_remote`.

HL_HEXAGON_SIM_VERBOSE

In Simulator Offload mode, set this variable to 1 to generate verbose messages from the simulator.

HL_HEXAGON_MEMFILL

In Simulator Offload mode, set this variable to a one-byte value as the value to fill all of memory (that is, the initial value of uninitialized memory). The default value is 0x1f.

HL_HEXAGON_TIMING

In Simulator Offload mode, set this variable to 1 to enable timing mode with data-backed caches in the simulator.

HL_HEXAGON_SIM_MIN_TRACE

In Simulator Offload mode, this variable enables a minimal PC trace. Set the value to the name you want for the trace file.

HL_HEXAGON_SIM_TRACE

In Simulator Offload mode, this variable enables a full PC trace. Set the value to the name you want for the trace file.

This variable generates more output than `HL_HEXAGON_SIM_TRACE`.

HL_HEXAGON_SIM_MEM_TRACE

In Simulator Offload mode, this variable enables a memory trace. Set the value to the name you want for the trace file.

This variable generates more output than HL_HEXAGON_SIM_TRACE.

HL_HEXAGON_SIM_DBG_PORT

In Simulator Offload mode, this variable sets up the simulator for remote debugging. Set the value to a TCP port number.

The simulator listens on this port number for remote debugging and write status information during the simulation.

Use this variable with `hexagon-lldb`, which is available as part of the Hexagon Tools.

HL_HEXAGON_PACKET_ANALYZE

In Simulator Offload mode, set this variable to the name you want for the file to which the simulator will write packet analysis information.

This data is in JSON format that is consumed by `hexagon-profiler` which is available as part of the Hexagon Tools. `hexagon-profiler` is used to visualize instruction packet level performance data such as packet commits and stalls in HTML format.

HL_HEXAGON_SIM_CYCLES

In Simulator Offload mode, set this variable to 1 to have the simulator write the number of cycles taken to execute the pipeline. Output is written to stdout.

HL_HEXAGON_SIM_STATS

In Simulator Offload mode, set this variable to 1 to have the simulator write performance statistics to stdout. The following example shows the format of this data.

```
Done!
T0: Insns=6761 Tcycles=3530
T1: Insns=0 Tcycles=0
T2: Insns=0 Tcycles=0
T3: Insns=0 Tcycles=0
T4: Insns=0 Tcycles=0
T5: Insns=0 Tcycles=0
Total: Insns=6761 Pcycles=21192
Simulator speed=0.419547 Mips
Ratio to Real Time (600 MHz) = ~1/456
(elapsed time = 0.016115s)
```

HL_HEXAGON_CODE_SIGNER

In the two Offload modes, set this variable to the path to a script that the Halide compiler will call to sign the offloaded shared objects that the pipeline and the device side runtime has been compiled to. For more information, see [Tools for signing Offload mode pipelines](#).

HL_DEBUG_CODEGEN

This variable is available in all modes and can be used when running a generator to make the Halide compiler output verbose data about the compilation. It accepts integer values that signify increasing levels of verbosity.

This variable is especially useful to inspect assembly code generated for your Halide pipeline when using one of the two Offload modes.

3.5.6 Common compile time or runtime errors

3.5.6.1 Crashed thread due to TLBMISS X (execution)

Sometimes in Device Standalone mode, you might get an error that shows up in QXDM or mini-dm logs. For example:

```
[08500/04] 50:55.650 00: CDSP:##### Process
→on cDSP CRASHED!!!!!! ##### 0725
→platform_qdi_driver.c
[08500/04] 50:55.650 00: CDSP:----- Crash Details are
→furnished below ----- 0725 platform_
→qdi_driver.c
[08500/04] 50:55.650 00: CDSP:process "/frpc/c04d4de0 main-conv3x3a16"
→crashed in thread "/frpc/c04d4de0 " due to TLBMISS X (execution)
→0725 platform_qdi_driver.c
[08500/04] 50:55.650 00: CDSP:Crashed Shared Object ./libconv3x3a16_
→skel.so load address : 0xE040C000 0725 platform_qdi_driver.c
[08500/04] 50:55.650 00: CDSP:fastrpc_shell_3 load address : E6F00000
→and size : D8130 0725 platform_qdi_driver.c
[08500/04] 50:55.650 00: CDSP:Fault PC : 0x0 0725 platform_qdi_driver.c
[08500/04] 50:55.650 00: CDSP:LR : 0xE041420C 0725 platform_qdi_driver.
→c
[08500/04] 50:55.650 00: CDSP:SP : 0xC5E875B8 0725 platform_qdi_driver.
→c
[08500/04] 50:55.650 00: CDSP:Bad va : 0xE0400282 0725 platform_qdi_
→driver.c
[08500/04] 50:55.650 00: CDSP:FP : 0xC5E875F8 0725 platform_qdi_driver.
→c
[08500/04] 50:55.650 00: CDSP:SSR : 0x21F70C60 0725 platform_qdi_
→driver.c
[08500/04] 50:55.650 00: CDSP:Call trace: 0725 platform_qdi_driver.c
[08500/04] 50:55.650 00: CDSP:[<E041420C>] halide_error_access_out_of_
→bounds+0x2AC: (./libconv3x3a16_skel.so) 0725 platform_qdi_driver.c
[08500/04] 50:55.650 00: CDSP:[<E04192F8>] conv3x3a16_halide+0x1BB8: (.
→/libconv3x3a16_skel.so) 0725 platform_qdi_driver.c
[08500/04] 50:55.650 00: CDSP:[<E040FCB8>] conv3x3a16_run+0x124: (./
→libconv3x3a16_skel.so) 0725 platform_qdi_driver.c
[08500/04] 50:55.650 00: CDSP:[<E040F74C>] conv3x3a16_skel_
→invoke+0x24C: (./libconv3x3a16_skel.so) 0725 platform_qdi_driver.c
[08500/04] 50:55.650 00: CDSP:[<E6F76BCC>] mod_table_invoke+0x224:
→(fastrpc_shell_3) 0725 platform_qdi_driver.c
[08500/04] 50:55.650 00: CDSP:[<E6F96EF0>] fastrpc_invoke_
→dispatch+0x170: (fastrpc_shell_3) 0725 platform_qdi_driver.c
[08500/04] 50:55.650 00: CDSP:[<E6F7135C>] HAP_proc_adaptive_
→qos+0x3D0: (fastrpc_shell_3) 0725 platform_qdi_driver.c
[08500/04] 50:55.650 00: CDSP:[<E6F72F30>] _pl_fastrpc_uprocess+0x758:
→(fastrpc_shell_3) 0725 platform_qdi_driver.c
[08500/04] 50:55.650 00: CDSP:----- End of
```

(continues on next page)

(continued from previous page)

```

→Crash Report ----- 0725
→platform_qdi_driver.c

```

The logs say that a TLB miss, specifically TLBMISS X (execution), exception occurred. Looking at the call stack we know that it occurred in the function `halide_error_access_out_of_bounds`. This function is one of the error handler functions in the Halide runtime. These functions are called when the pipeline fails an assert during execution. The function handles the error by printing a message and returning an error code through calls to `halide_error` and `halide_print`.

In the two Standalone modes, you must provide implementations for these functions in the DSP code. Following is an example implementation of the functions that will work in Device Standalone mode.

```

#include "HAP_farf.h"
#undef FARF_LOW
#define FARF_LOW 1

void halide_print(void *user_context, const char *str) {
    FARF(LOW, "%s", str);
}

void halide_error(void *user_context, const char *msg) {
    halide_print(user_context, msg);
}

```

If you run the pipeline again, it will still crash as shown in the following example because it was in an error handler function when the TLB Miss exception occurred. In this case, the output buffer is accessed beyond its extent in dimension 1.

```

[08500/02] 57:00.071 00: CDSP:fastrpc_spawn: Successfully spawned user_
→PD for /frpc/c04d4de0 main-conv3x3a16 (pidA 27003) with asid 12 and_
→load addr 0xe6f00000 0959 fastrpc_loader.c
[08500/02] 57:00.073 00: CDSP:HVX power request successful for client_
→11 0677 fastrpc_kpower.c
[08500/02] 57:00.085 770df:0c: CDSP: open_mod_table_open_dynamic:_
→Module libconv3x3a16_skel.so opened successfully with handle_
→0x7203fa0 0540 mod_table.c
[08500/02] 57:00.086 770df:0c: CDSP: perf_turbo 0051 conv3x3a16_i.c
[08500/02] 57:00.086 00: CDSP:Max MIPS = 2147483647 1167 fastrpc_
→kpower.c
[08500/02] 57:00.086 00: CDSP:Max bus bw = 1636000000 1177 fastrpc_
→kpower.c
[08500/02] 57:00.086 00: CDSP:Setting app type to COMPUTE_CLIENT_CLASS_
→0731 fastrpc_kpower.c
[08500/02] 57:00.086 00: CDSP:Setting clock - mipsTotal: -2,_
→mipsPerThread: 2147483647 0327 fastrpc_kpower.c
[08500/02] 57:00.086 00: CDSP:Setting bus_bw - bwBytePerSec:_
→6543114240, usagePercentage: 100 0342 fastrpc_kpower.c
[08500/02] 57:00.086 00: CDSP:Setting latency 10 0361 fastrpc_kpower.c
[08500/02] 57:00.088 770df:0c: CDSP: power_on 0031 conv3x3a16_i.c

```

(continues on next page)

(continued from previous page)

```

[08500/02] 57:00.090 770df:0c: CDSP: Before Call 0099 conv3x3a16_i.c
[08500/00] 57:00.090 770df:0c: CDSP: In halide_error 0115 conv3x3a16_i.
↳C
[08500/00] 57:00.090 770df:0c: CDSP: Output buffer output is accessed_
↳at 1023, which is beyond the max (1021) in dimension 1 0111_
↳conv3x3a16_i.c
[08500/02] 57:00.090 770df:0c: CDSP: After Call = -4 0101 conv3x3a16_i.
↳C
[08500/02] 57:00.090 770df:0c: CDSP: error = -4 0104 conv3x3a16_i.c
[08500/03] 57:00.090 770df:0c: CDSP: Error 0xffffffff: open_mod_table_
↳handle_invoke failed for handle 0x7203fa0, sc 0x5030100 0785 mod_
↳table.c
[08500/02] 57:00.091 00: CDSP:HVX power released for client 11 0681_
↳fastrpc_kpower.c
[08500/02] 57:00.092 00: CDSP:fastrpc_kill done for pidA 27003 1143_
↳fastrpc_loader.c

```

3.5.6.2 Accessing buffers out of bounds and violating constraints

Sometimes a Halide pipeline might crash with the error code, `halide_error_code_access_out_of_bounds`. Typically, this error occurs because the pipeline tried to access a buffer beyond the extent of a dimension of the buffer passed during execution.

In Device Offload mode, this error message can be seen using `adb`. In Simulator Offload mode, this error is seen on `stdout`.

In the two Standalone modes, you must provide implementations of `halide_error` and `halide_print`. Then, in Device Standalone mode, these messages can be seen using `QXDM` or `mini-dm`, and in Simulator Standalone mode these messages will appear on `stdout`.

Following is an example in Device Standalone mode (`mini-dm`).

```

[08500/02] 57:00.071 00: CDSP:fastrpc_spawn: Successfully spawned user_
↳PD for /frpc/c04d4de0 main-conv3x3a16 (pidA 27003) with asid 12 and_
↳load addr 0xe6f00000 0959 fastrpc_loader.c
[08500/02] 57:00.073 00: CDSP:HVX power request successful for client_
↳11 0677 fastrpc_kpower.c
[08500/02] 57:00.085 770df:0c: CDSP: open_mod_table_open_dynamic:_
↳Module libconv3x3a16_skel.so opened successfully with handle_
↳0x7203fa0 0540 mod_table.c
[08500/02] 57:00.086 770df:0c: CDSP: perf_turbo 0051 conv3x3a16_i.c
[08500/02] 57:00.086 00: CDSP:Max MIPS = 2147483647 1167 fastrpc_
↳kpower.c
[08500/02] 57:00.086 00: CDSP:Max bus bw = 1636000000 1177 fastrpc_
↳kpower.c
[08500/02] 57:00.086 00: CDSP:Setting app type to COMPUTE_CLIENT_CLASS_
↳0731 fastrpc_kpower.c
[08500/02] 57:00.086 00: CDSP:Setting clock - mipsTotal: -2,_
↳mipsPerThread: 2147483647 0327 fastrpc_kpower.c

```

(continues on next page)

(continued from previous page)

```

[08500/02] 57:00.086 00: CDSP:Setting bus_bw - bwBytePerSec:
↳6543114240, usagePercentage: 100 0342 fastrpc_kpower.c
[08500/02] 57:00.086 00: CDSP:Setting latency 10 0361 fastrpc_kpower.c
[08500/02] 57:00.088 770df:0c: CDSP: power_on 0031 conv3x3a16_i.c
[08500/02] 57:00.090 770df:0c: CDSP: Before Call 0099 conv3x3a16_i.c
[08500/00] 57:00.090 770df:0c: CDSP: In halide_error 0115 conv3x3a16_i.
↳C
[08500/00] 57:00.090 770df:0c: CDSP: Output buffer output is accessed
↳at 1023, which is beyond the max (1021) in dimension 1 0111
↳conv3x3a16_i.c
[08500/02] 57:00.090 770df:0c: CDSP: After Call = -4 0101 conv3x3a16_i.
↳C
[08500/02] 57:00.090 770df:0c: CDSP: error = -4 0104 conv3x3a16_i.c
[08500/03] 57:00.090 770df:0c: CDSP: Error 0xffffffff: open_mod_table_
↳handle_invoke failed for handle 0x7203fa0, sc 0x5030100 0785 mod_
↳table.c
[08500/02] 57:00.091 00: CDSP:HVX power released for client 11 0681
↳fastrpc_kpower.c
[08500/02] 57:00.092 00: CDSP:fastrpc_kill done for pidA 27003 1143
↳fastrpc_loader.c

```

This message indicates that the buffer output was accessed at 1023 beyond the maximum in dimension 1 (1021). In other words, the pipeline tried to access the buffer beyond the number of rows in the external buffer (*External Buffers*) passed for output.

When a Halide pipeline is compiled into object code, the Halide compiler adds some asserts based on the schedule used. For example, consider the following code example.

```

Func(output)
  .tile(x, y, xi, yi, 128, 4, TailStrategy::RoundUp)
  .vectorize(xi)
  .unroll(yi);

```

This schedule uses `TailStrategy::RoundUp`. While tiling output, the remainder loops over `x` and `y` are rounded up to the next multiple of the split factor in that dimension (128 in `x` and 4 in `y`).

But if at runtime, an external buffer with only 1022 rows was passed as the buffer for output. The loop over `y`, which was rounded up to the next multiple of 4, will execute until 1024, beyond the extent of the buffer passed for output.

Similarly, the following code example will cause asserts (checked during execution of the pipeline) to be added to the compiled Halide pipeline.

```

input.dim(0).set_min(0);
input.dim(1).set_min(0);
Expr input_stride = input.dim(1).stride();
input.dim(1).set_stride((input_stride/128) * 128);
input.set_host_alignment(128);

```

If during execution the external buffer passed for input is such that its stride in dimension 1 is not a multiple of 128, `halide_error_code_constraint_violated` will be generated.

If the host pointer of external buffer is not aligned to 128, `halide_error_code_unaligned_host_ptr` will be generated.

3.6 Signing for deployment

Software applications can use the cDSP only if one of the following holds true.

- The code executing on the cDSP is signed.
- The device is a test device and has a testsig installed.
- The code that is offloaded to the DSP is offloaded by setting up an unsigned User PD (available only on SM8150 or later devices).

3.6.1 Tools for signing Offload mode pipelines

In Device Offload mode, Halide pipelines are host (Arm) executables containing embedded Hexagon shared objects that are offloaded to the cDSP at runtime. Several tools are provided to help extract these shared objects for signing. The method for signing the objects itself is not defined here because it depends on how the target device has been configured.

If signing can be done during Halide compilation (running your generator), use `hl_signnow` from your Halide installation (it is in the `Halide/tools` directory). Replace the `cp $1 $2` step in the script with the method required to sign a shared object.

The following example shows signing Halide pipeline online, when producing pipeline code (running the generator).

```
$> export HL_HEXAGON_CODE_SIGNER=/path/to/hl_signnow
$> <build and run the generator>
...
...
hl_signnow: signing /tmp/hvx_unsignedy3t97n.so as /tmp/hvx_
↪signedYkst0t.so
hl_signnow: signing /tmp/hvx_unsignedFOHXXK.so as /tmp/hvx_
↪signedCZksV1.so
```

If signing must be done separately from compilation, use `hl_signsav` and `hl_signuse` from your Halide installation (they are in the `Halide/tools` directory). Signing is now a three-step process.

1. Extract and save the shared objects when running the generator by setting the `HL_HEXAGON_CODE_SIGNER` environment variable to `hl_signsav`.
2. Sign the extracted shared objects.
3. Plug the signed shared objects back into the application by rebuilding the application (running the generator again).

For example:

```
$> /bin/rm -rf /tmp/hl_sign_${USER}
$> export HL_HEXAGON_CODE_SIGNER=/path/to/hl_signsav
```

(continues on next page)

(continued from previous page)

```

$> <build and run the generator>
...
...
...
saving /tmp/hvx_unsignedjAQVqB.so as /tmp/hl_sign_${USER}/lib000.so
saving /tmp/hvx_unsignedFOHXXK.so as /tmp/hl_sign_${USER}/lib001.so

$> ls /tmp/hl_sign_${USER}
lib000.so    lib001.so

$> <sign the libraries>

$> export HL_HEXAGON_CODE_SIGNER=/path/to/hl_signuse

$> <build and run the generator>
...
...
...
hl_signuse: copying /tmp/hl_sign_${USER}/lib000.so to /tmp/hvx_
↪signedSRbsjF.so
hl_signuse: copying /tmp/hl_sign_${USER}/lib001.so to /tmp/hvx_
↪signedXLAWUD.so

```

The `/tmp/hl_sign_${USER}` directory will be empty after the second build step because the libraries are moved to a subdirectory called `done` as they are used. If the application is to be built again, move these objects back up one level.

You can also use `hl_signsav` to simply obtain the Hexagon shared objects for examination whenever needed.

Note: On Windows devices, the shared objects are saved in `/temp/hl_sign_%USERNAME%`.

For Windows usage, see `Halide/tools/hl-sign-notes.cmd.txt`.

3.6.2 Unsigned mode execution

Halide pipelines can be run on the Hexagon DSP in Unsigned mode (unsigned User PD) by calling the `halide_hexagon_init_unsigned_mode()` function from the application before calling the Halide pipeline. This function must be called before any Hexagon-related function that uses FastRPC. Calling this function is the first thing that an application should do to ensure that an unsigned User PD is set up for offloading to the cDSP.

The following example shows the application-side (host-side) code use to set up Unsigned mode execution.

```

int main(int argc, char **argv) {
    const int width = 292;
    const int height = 274;

```

(continues on next page)

(continued from previous page)

```
int iterations = 10;

if (halide_hexagon_init_unsigned_mode(NULL) == -1) {
    printf("Unable to set halide_hexagon_init_unsigned_mode\n");
}

const int VLEN=128;
int stride_y = (width + (VLEN)-1) & -(VLEN);
halide_dimension_t x_dim{0, width, 1};
halide_dimension_t y_dim{0, height, stride_y};
halide_dimension_t io_shape[2] = {x_dim, y_dim};

...
...
}
```

Note: This feature is available only on Snapdragon SM8150 devices and later.

4 Autoscheduler

New in version 2.3.0.

Writing schedules ([Schedule](#)) that achieve high performance is difficult. There are two problems:

1. The search space of possible schedules is large.
2. Predicting the runtime of a given schedule in the search space of possible schedules is hard.

Halide provides an Autoscheduler plug-in that can schedule Halide pipelines written for HVX. Following are the important features of the Autoscheduler library:

- It defines a parameterization of the schedule space that enables it to explore a large set of schedules.
- It provides a cost model based on a combination of hand-crafted features and machine learning.

The Autoscheduler uses a variant of beam search to search the space of possible schedules. At each stage of the beam search, it sorts the schedules based on the runtimes predicted by the cost model. The use of a hybrid learned cost model makes it easy for the Autoscheduler to adapt to a given target or algorithm (see [Autotuning](#)).

This Autoscheduler is based on the paper, *Learning to Optimize Halide with Tree Search and Random Programs* (https://halide-lang.org/papers/halide_autoscheduler_2019.pdf).

4.1 Use the Autoscheduler

We support using the Autoscheduler for Halide pipelines that are defined using generators ([Generators](#)). Following are the steps to use the Autoscheduler on a Halide pipeline:

1. Add estimates for all input/output buffers in the generator.
2. Compile the generator as usual with an extra `rdynamic` option.
3. Run the generator with the appropriate target, specifying the generator, parameter, `auto_schedule=true`, and the location of the Autoscheduler plug-in.

This step runs the Autoscheduler for the given pipeline and produces the autoscheduled object file.

The Autoscheduler plug-in library is located at: `Halide/lib/libauto_schedule.so`

For autotuning ([Autotuning](#)), required tools are located at: `Halide/bin`.

4.1.1 Set up a generator for autoscheduling

No change is required in the Halide algorithm part for autoscheduling. The Autoscheduler uses estimates for the region of input and output buffers to make its decisions. Provide estimates for the input and output buffers and any other generator inputs. For example:

```
// set the estimates for input and output buffer dimensions.
const int W = 292;
const int H = 274;

in.dim(0).set_estimate(0, W);
in.dim(1).set_estimate(0, H);

output.dim(0).set_estimate(0, W);
output.dim(1).set_estimate(0, H);
```

Use the provided generator parameter, `auto_schedule` (default value: `false`), to predicate manual scheduling directives, if any, when not using the Autoscheduler. This parameter is required because if any scheduling directives are already present, the generator will issue an error (with the exception of the `.hexagon()` directive. See [User-specified offloading](#)).

For example:

```
if (!auto_schedule) {
    const int vector_size = natural_vector_size<uint8_t>();
    output
        .vectorize(x, vector_size)
        .parallel(y, 16);
}
```

The Autoscheduler generates a schedule for the Hexagon DSP if the host in the target is set to `hexagon` or if the target has the `hvx` target feature.

4.1.2 User-specified offloading

New in version 2.4.0.

User-specified offloading provides more control in deciding which stages of a pipeline are offloaded to the Hexagon DSP using the Autoscheduler. If you add the `hexagon` scheduling directive to a `Func`, the Autoscheduler will generate a schedule that offloads that `Func` and other stages computed within the loopnest of that `Func` to the Hexagon DSP.

However, if no stage is explicitly marked with the `hexagon` scheduling directive, the Autoscheduler, by default, offloads the execution of the entire pipeline to the Hexagon DSP. For example:

```
if (auto_schedule) {
    // This ensures that the autoscheduler generates schedule such_
    ↪ that the output
    // Func and all other funcs computed inside the loopnest of output_
    ↪ are
    // scheduled to be offloaded to Hexagon.
```

(continues on next page)

(continued from previous page)

```
output.hexagon()  
}
```

Note: Apart from the `hexagon` scheduling directive, other scheduling directives are not respected by the Autoscheduler. In some cases, they might cause the generator to issue an error.

4.1.3 Call the pipeline generator with the Autoscheduler

The Autoscheduler is invoked on the pipeline when the following are passed to the generator executable:

- Parameter: `auto_schedule=true`
- Option: `-p Halide/lib/libauto_schedule.so -s Adams2019`, used to point to the Autoscheduler plug-in and specify the name of the Autoscheduler (Adams2019).

Note: If the plug-in library is not passed to the Autoscheduler, Halide uses the default Autoscheduler, which cannot offload to HVX.

A large part of the Autoscheduler plug-in library is written in Halide and uses `libHalide` at runtime. For the Autoscheduler to find `libHalide` symbols when the generator is run, specify the `-rdynamic` compiler option while building the generator.

The `schedule emit` option can be specified to the generator executable to output a `<pipeline>.schedule.h` header file, which contains the schedule produced by the Autoscheduler. This schedule file can be included directly with the pipeline to reproduce the Autoscheduler results (see [Schedule files](#)).

For example, specifying the `-e o,h,schedule emit` options to the generator tells the generator to output the object file, header file, and generated schedule header. If the generator is not directed to `auto_schedule`, the schedule file will not include any scheduling directives.

The `featurization emit` option produces a feature file that is required for autotuning. For details, see [Autotuning](#).

4.1.4 Environment variables

The following Halide environment variables can be used with the Autoscheduler.

HL_WEIGHTS_DIR

Path to custom weights to use for the cost model. This variable is used for autotuning and retraining.

The initially trained weights are located at `Halide/bin/baseline.weights`. The path to `baseline.weights` is not required because the Autoscheduler uses `baseline.weights` by default.

HL_DEBUG_AUTOSCHEDULE

If set, this variable is used for the debug log level for autoschedule generation (overriding the value of `HL_DEBUG_CODEGEN`, if there is a value).

HL_MACHINE_PARAMS

An architecture description string that contains three comma-separated values.

The Autoscheduler plug-in uses only the first term, which loosely defines the number of parallel contexts supported by the hardware. For HVX, the ideal first term is four (for example, `HL_MACHINE_PARAMS=4, ,`).

HL_PREFETCHING

If set to 1, this variable allows the Autoscheduler to emit prefetches of inputs required for computing the innermost tile of output.

This variable will emit prefetches only if the target is `hexagon` or if it has the feature `hvx`.

HL_BEAM_SIZE

This variable is used in the beam search. The default value is 32. To get a *greedy* search instead, set this variable to 1.

HL_RANDOM_DROPOUT

This variable is the chance (as a percent) of accepting each state in the beam, normalized by the number of decisions made.

For example, a value of 5 means there is a 5 percent chance of never rejecting any states in the beam.

HL_SEED

This variable specifies the random seed used by the random dropout.

4.2 Autotuning

The Autoscheduler can be retrained for specific user pipelines, which typically leads to better performance for those pipelines. It is designed so schedules can be importance-sampled based on the cost model. These schedules can then be benchmarked and used to retrain the cost model.

This process of randomly sampling schedules and retraining the cost model helps the Autoscheduler find better performing schedules for a given algorithm (*Algorithm*). This process is called *autotuning*.

The Autoscheduler is shipped with the following tools that allow for autotuning custom pipelines:

- `autotune_loop.py`
- `featurization_to_sample`
- `retrain_cost_model`

- `baseline.weights`

The `autotune_loop.py` script uses the Autoscheduler as follows:

- Compiles a user pipeline with different schedules.
- Benchmarks the compiled pipelines on a Hexagon device, using the `featurization_to_sample` construct samples for each schedule and the corresponding runtimes that are observed.
- Retrains the Autoscheduler cost model on the new samples.

Autotuning requires a compiled generator enabled for autoscheduling (see [Set up a generator for autoscheduling](#)).

Usage:

```
./autotune_loop.py /path/to/some.generator generatortname
```

The following options can be given to `autotune_loop.py`:

- target/-t**
Autotune target. The default target is `host`.
- bld-dir/-b**
Location for any built artifacts. The default location is the current directory.
- batch-size/-s**
Batch of samples to compile in parallel. The default value is 32.
- num-batches/-n**
Number of batches to train for. The default value is 100.
- parallel-factor/-p**
Parallelization factor to be used for schedules. The default value is 16.
- hexagon-sdk-root/-sdk**
Location of the Hexagon SDK. This option is required for any Android-based target.
- run-device-id/-r**
Space-separated list of devices on which to run benchmarking. This option is required for any Android-based target.
- q6-version/-v**
Hexagon (Q6) version of the device. The default value is 65 for v65.
- generator-arg-sets**
Space-separated list of extra generator argument sets used when running the generator.
- weights/-w**
Location of the starting weights file. The default file is `baseline.weights`.

The `autotune_loop.py` script compiles a batch of 32 schedules in parallel and serially benchmarks them on a device. By default, it runs 1000 such batches (retraining after every iteration) before stopping.

The best performing schedule found during autotuning is saved at `build_dir/samples/best.<generatortname>.schedule.h`, which can be used to reproduce the performance seen after autotuning (see [Schedule files](#)).

The script can be stopped at any point, and it saves the current trained weights in `build_dir/samples/updated.weights`. To run the Autoscheduler with new weights, point the `HL_WEIGHTS_DIR` environment variable to this path:

```
export HL_WEIGHTS_DIR=build_dir/samples/updated.weights
```

Weights trained before every batch execution are also saved in the corresponding batch directory. For example, weights used to produce the second batch are in `build_dir/samples/batch_2_0/used.weights`. The script saves the schedule files for all the benchmarked pipelines and the best schedule seen so far.

4.3 Schedule files

The Autoscheduler generates the schedule in a header file that exposes an interface for using the schedule with a pipeline.

For example, the interface to apply the schedule that the Autoscheduler produces after autoscheduling `median3x3` is written to the header file, `median3x3.schedule.h`, as follows:

```
inline void apply_schedule_median3x3(
    ::Halide::Pipeline pipeline,
    ::Halide::Target target
);
```

The following example shows a schedule can be used within a generator.

```
#include median3x3.schedule.h

apply_schedule_median3x3(get_pipeline(), target);
```

Note: Using a schedule file can fail or produce sub-optimal results if there is a change to the pipeline algorithm after the schedule file is generated.

4.4 Autoscheduler examples

The following examples are enabled for autoscheduling:

- `Halide/Examples/offload/apps/modified_census_transform`
- `Halide/Examples/standalone/device/apps/conv3x3`
- `Halide/Examples/offload/apps/hexagon_benchmarks`

Set up the environment as described in ([Setup](#)).

For a manual schedule:

```
make run
```

To automatically schedule this example, set the `AUTOSCHEDULE` variable to `true`.

```
AUTOSCHEDULE=true make run
```

To use the pregenerated schedule file, set the `USE_SCHEDULE` variable to `true`.

```
USE_SCHEDULE=true make run
```

5 Halide with UBWC DMA

- *UBWC DMA basics*
 - *Supported data formats*
 - *Supported Snapdragon devices and Hexagon local memory*
- *Program UBWC DMA with Halide*
 - *UBWC DMA with Halide runtime model*
 - *Halide device interface for UBWC DMA*
 - *Scheduling*
 - *De-interleaving of UV for 4:2:0*
 - *Known limitations*
- *Performance considerations*
 - *Memory locality*
 - *Power API for UBWC DMA*
 - *UBWC DMA concurrency*
- *Examples*

Universal Bandwidth Compression Direct Memory Access (UBWC DMA) is a feature provided by an IP peripheral block on the cDSP. This feature provides DMA capability for transferring data from DDR main memory to Hexagon local memory and back, with the data (image frames) in DDR either compressed (bandwidth-wise) or linear, for the DSP to process. The data in local memory in the DSP is uncompressed.

5.1 UBWC DMA basics

The following table describes the UBWC DMA read and write operations.

UBWC DMA direction	Description
UBWC DMA read	Transfer data from a buffer in DDR main memory to a buffer in Hexagon local memory
UBWC DMA write	Transfer data from a buffer in Hexagon local memory to a buffer in DDR main memory

Note: Do not confuse UBWC DMA with User DMA (Hexagon V68 ISA and later). UBWC DMA is provided on select Qualcomm Snapdragon devices by an IP block external to the cDSP. User DMA is internal to the cDSP and is supported in Halide by the user of the `hexagon_user_dma` scheduling directive. UBWC DMA is supported only in the two Standalone modes.

In Halide, the UBWC DMA operations and interactions are abstracted with the DMA driver calls hidden by the Hexagon UBWC DMA runtime. They are reduced to the following:

- A set of host-side API calls used to manage DMA resources and associate a frame buffer with DMA along with the buffer parameters
- A set of Halide scheduling directives to describe where, when, and how much data to move using UBWC DMA

For more detailed information about UBWC DMA, see the Hexagon SDK documentation.

5.1.1 Supported data formats

The following table lists the UBWC DMA formats. The enumerations (`halide_hexagon_image_fmt_t`) are defined in `Halide/include/HalideRuntimeHexagonDma.h`.

Data format in DDR	Description
Raw	Unformatted and uncompressed, per-byte DMA access
NV12	Linear, 8 bpp semi-planar 4:2:0 YUV format, with each pixel component stored in 8b
NV12-4R	Same as linear NV12
P010	Linear, padded 10 bpp semi-planar 4:2:0 YUV format, with each 10b pixel component represented in 16b; results in 2 bytes per pixel in Y plane and 4 bytes per pixel in UV plane
NV12 UBWC	Bandwidth compressed NV12
NV12-4R UBWC	Bandwidth compressed NV12-4R, encoded differently than NV12 compressed
P010 UBWC	Bandwidth compressed P010
TP10	Bandwidth compressed, tightly packed 10 bpp semi-planar 4:2:0 YUV format; results in 4 bytes for every three pixels in Y plane and 8 bytes for every three pixels in UV plane

Because Halide requires the image frame to be represented by two separate Halide buffers (a Y and a UV buffer), a DMA transaction is specified and scheduled separately for the Y and UV planes. An image-formatted frame buffer must compose and contain either Y (luma) only or both Y (luma) and UV (chroma) planes, per the format specification

5.1.2 Supported Snapdragon devices and Hexagon local memory

Hexagon local memory is a shared and limited resource for the entire compute subsystem. Its available size is device-specific, and its usage must be constrained with a trade-off against the required application performance. The following table lists the device-specific memory information.

Snapdragon device	Total L2 cache	Hexagon local memory type	Size
SM8150	1 MB	Hexagon locked L2 cache	Up to 384 KB
SM8250	1 MB	Hexagon locked L2 cache	Up to 768 KB
SM8350	1 MB	Hexagon locked L2 cache	Up to 768 KB

Hexagon local memory is a shared and limited resource for the entire compute subsystem, whose available size is device specific, and its usage must be constrained with a trade-off against required application performance.

5.2 Program UBWC DMA with Halide

5.2.1 UBWC DMA with Halide runtime model

UBWC DMA with Halide is supported in the two Standalone modes ([Halide execution modes](#)). In Simulator Standalone mode, the generated object file can be used to simulate an end-to-end vision algorithm running on the DSP with the UBWC DMA co-sim add on.

5.2.2 Halide device interface for UBWC DMA

The Halide device interface for UBWC DMA provides a user API for application code to set up and associate buffers to the UBWC DMA device. This gives the application access to the UBWC DMA device services. The functions are summarized in the following table, and the prototypes and detailed descriptions are available in `Halide/include/HalideRuntimeHexagonDma.h`.

API name	Description
<code>halide_hexagon_dma_allocate_engi</code>	Allocate a UBWC DMA engine.
<code>halide_hexagon_dma_power_mode_vo</code>	Vote for a preset performance level to set the DMA power mode as required by the application. This power mode voting is separate from the Hexagon power votes.
<code>halide_hexagon_dma_device_wrap_n</code>	Attach a buffer to the UBWC DMA interface when a raw buffer (<code>halide_buffer_t</code>) is used.
<code>halide_hexagon_dma_prepare_for_c</code>	Before any UBWC DMA transfers, set up the DDR buffer specification for UBWC DMA read.
<code>halide_hexagon_dma_prepare_for_c</code>	Before any UBWC DMA transfers, set up the DDR buffer specification for UBWC DMA write.
<code>halide_hexagon_dma_finish_frame</code>	Call this API after all the ROIs are transferred for a given frame. The buffer passed in a call to this function should be the destination buffer.
<code>halide_hexagon_dma_device_detach</code>	Detach a buffer from the UBWC DMA interface when a raw buffer (<code>halide_buffer_t</code>) is used.
<code>halide_hexagon_dma_deallocate_en</code>	Unmap all the physical UBWC DMA engines after completion of a frame transfer.
<code>halide_hexagon_dma_deinit</code>	Free all the UBWC DMA resources at the end of the process. This API should be called when a process or all the threads spawned from a process have finished the UBWC DMA transfer.

The Hexagon local memory buffer is internal to the generated Halide pipeline that is managed by the Halide UBWC DMA runtime, and it is not visible to the host-side user code. However, pipeline execution might be impacted at runtime by its availability; for details, see [Memory locality](#).

5.2.3 Scheduling

Many scheduling directives have already been covered ([Schedule](#)). This section lists more scheduling directives that are important to working with UBWC DMA.

Directive	Description of typical use
<code>hexagon_ubwc dma_r</code>	UBWC DMA read.
<code>hexagon_ubwc dma_w</code>	UBWC DMA write.
<code>store_in</code>	Specify the type of memory to be allocated for a <code>Func</code> . For UBWC DMA, it is typically Hexagon local memory, <code>MemoryType::LockedCache</code> .
<code>store_at</code>	Place the allocation of Hexagon local memory buffer at a specific stage or loop (that is, it must be outside of inner loops).
<code>reorder_storage</code>	De-interleave the UV semi-plane for 4:2:0 pixel format in the Hexagon local memory buffer.
<code>tile</code>	Define the ROI dimensions that are factored into calculating the required Hexagon local memory buffer size.
<code>bound</code>	For stencil processing, bound DMA access to within the image frame buffer boundary.
<code>fold_storage</code>	For asynchronous or stencil processing, a circular buffer of Hexagon local memory. Allows access in a monotonically increasing or decreasing order, by rotating the buffer. Also factored into the required Hexagon local memory buffer size.
<code>async</code>	Asynchronous control flow between the consumer and producer of fold storage. Allows concurrency of processing data that is already transferred on one fold while simultaneously DMA transferring data on another fold. That is, to prefetch the next required consumer data from the DDR with DMA read, or to store resultant producer data to DDR with DMA write, while in parallel, processing (consuming or producing) another set of data.
<code>split</code>	Splits the dimension of a frame buffer. Precursor to independent DMA access per an outer dimension split when used with the <code>parallel</code> directive.
<code>parallel</code>	Parallelize each split outer dimension, with independently assigned DMA access. Allows for parallel processing and DMA of data for each split.
<code>compute_at</code>	Specify when the producer stage is computed with respect to the consumer stage.
<code>compute_root</code>	Place the computation of the stage at the top level (before any place where it is used).
<code>store_at</code>	Specify where the memory for the producer stage is allocated with respect to the consumer stage.
<code>store_root</code>	Place the allocation of the stage at the top level.
<code>align_storage</code>	Align the dimension specified by <code>x</code> to be a multiple of the alignment specified by <code>align</code> .

Note: You can use `copy_to_host` instead of `hexagon_ubwc dma_read`, and `copy_to_device` instead of `hexagon_ubwc dma_write`. However, these two alternatives will lead to noticeably poor run-time performance of your pipeline.

5.2.4 De-interleaving of UV for 4:2:0

For the 4:2:0 semi-planar format, the UV plane is expected to be after the Y plane when stored within a frame buffer. The Y and UV planes are then cropped appropriately and presented as two individual Halide buffers to the Halide pipeline. The Y buffer is described as 2D, and UV buffer is described as 3D with the UV interleaved.

For details on how to describe the UV interleaved property to Halide, see the NV12 example in `Halide/Examples/standalone/device/apps/dma_nv12_rw/halide/dma_nv12_rw_generator.cpp`.

5.2.5 Known limitations

Following are the known limitations of UBWC DMA with Halide.

Limitation	Description
<code>fold_storage</code>	See limitation for <code>async</code> .
<code>async</code>	Requires use of <code>fold_storage</code> to ping-pong asynchronously between producer and consumer threads.

5.3 Performance considerations

5.3.1 Memory locality

UBWC DMA data transfer into Hexagon local memory increases the transfer efficiency and reduces access latency seen by the Hexagon DSP core. This improves the overall performance of the program. The requirement of Hexagon local memory is mandatory for UBWC DMA as either the destination memory type for UBWC DMA read or the source memory type for UBWC DMA write to DDR.

The Halide pipeline uses the `store_in` scheduling directive to allocate Hexagon local memory.

For each extern buffer provided to the Halide pipeline, there is at least one corresponding instance of a Hexagon local memory buffer allocated to service it. The size of this local buffer is determined by several factors, including the Halide tile size, value size, and amount of storage folded. This is equivalent to the following example.

```
local_size = align(tile_width x tile_height x sizeof(data), 128) x
↳ number_of_folds
```

The number of allocated local memory instances per Halide data buffer is determined by the parallelizable aspect of Halide directives, such as the number of splits for multi-threading. This is roughly represented by the number of DMA threads, which, to avoid unnecessary resource allocation, should be limited to the number of Hexagon cores.

```
number_of_DMA_threads = max(number_of_splits, number_of_hexagon_cores)
```

When supported, local memory is allocated from a part of the L2 cache (Snapdragon device-specific). This is a limited resource and because UBWC DMA data transfers require locked L2 cache, the amount of L2 cache memory available for other processing is reduced. Thus, while increasing the UBWC DMA data

transfer size will increase the efficiency of UBWC DMA, it might impact the performance of the entire cDSP-based workload due to the reduced L2 cache available for compute.

5.3.2 Power API for UBWC DMA

UBWC DMA performance can be controlled by its power modes. Each mode translates into a corresponding set of pre-configured performance parameters. The application determines its performance requirement and votes for a power mode accordingly. This is done using an API that is provided for both the Hexagon DSP (*Power and performance APIs*) and UBWC DMA. UBWC DMA performance/power level is voted for independently from the Hexagon DSP power level.

The power level must be voted for after a UBWC DMA engine has been allocated. You can change your vote at any time during the allocation lifetime of the UBWC DMA engine. If there is no power mode vote, the default power mode is applied. On application exit and before deallocating the DMA engine, vote the power mode back to the default mode.

All enumerated power modes are supported by the API. If a specific Snapdragon device does not have support for a mode, that requested power mode is mapped to the closest (next or higher) appropriate supported mode.

The following example shows the application-side API for specifying the power level of UBWC DMA (voting).

```
#include "HalideRuntimeHexagonHost.h"
#include "HalideRuntimeHexagDma.h"

int halide_hexagon_dma_power_mode_voting(void *user_context, halide_
    ↪ hexagon_power_mode_t mode);
// user_context - Ignore. Can be NULL
// mode         - Power level value similar to selecting a voltage_
    ↪ corner as documented in the Hexagon SDK
//              documentation.
```

For all possible values for the mode argument to `halide_hexagon_dma_power_mode_voting`, see *Power and performance APIs*.

5.3.3 UBWC DMA concurrency

To improve performance, following are some best practices or considerations for concurrency.

- Split a frame for parallel processing to take advantage of the multi-threaded Hexagon DSP core.
 - Processing of each split is scheduled on a separate hardware thread. The number of tasks created is based on the split factor.
 - A key best practice is to keep the number of tasks less than or equal to the number of supported hardware threads.
- Asynchronize between the Halide producer and consumer to allow simultaneous UBWC DMA data transfer and data processing to reduce processing time.
 - UBWC DMA is a hardware accelerator that runs independently of the cDSP core. This method improves performance by removing UBWC DMA transfer waits with ahead-of-time data transfer to ensure that data is already available when the cDSP core needs it.

- The best practice is to transfer the right-sized data that takes longer for the cDSP to process than for UBWC DMA to transfer it.
- It is also a good idea to apply ping-pong buffers, using `fold_storage`, to enable this asynchronous parallelism between producer and consumer. The number of folds is dictated by the algorithm of how much data ahead (future) and behind (history) is required.
- For right sizing the data buffer, see [Memory locality](#) for the local memory constraints.
- Deploy multiple independent UBWC DMA engines to allow for simultaneous transfer of data. This can potentially shorten the overall runtime by reducing the data transfer time.
 - Logistically, the per application UBWC DMA engine separation can be:
 - Per frame buffer UBWC DMA engine.
 - Read vs. write DMA engine.
 - Points to consider for potentially reducing runtime:
 - There are a limited number of available UBWC DMA engines, with only allocated engines shared within the application and not across applications.
 - A transfer unit is sized small (local memory resource constraint) such that the overall difference between serialization and parallelization of a transfer is minimized.
 - An asynchronous ping-pong style DMA transfer might be a better option.

5.4 Examples

All the examples that demonstrate the use of UBWC DMA in Halide are located in `Halide/Examples/standalone/device/apps/`.

Linear data

For details on how to process raw (linear) format data, see `dma_raw_blur_rw_async`. This example also includes a line-buffered schedule that demonstrates whole-line transfer via UBWC DMA.

p010 format data

For details on how to process p010 format data, see `dma_p010_rw_async`.

nv12 format data

For processing nv12 format data asynchronously with storage folding, see `dma_nv12_blur_stencil`. This example also includes details on how to use line buffering in the schedule.

6 Python bindings (Linux only)

New in version 2.3.0.

Python bindings for Halide provide a way to build and compile Halide pipelines from Python. We ship a Halide Python library that is available at `Halide/lib/python3/site-packages` in your Halide installation.

Note: Currently, we support this library on Linux only, and the library is compatible with Python version 3.8.x.

6.1 Dependencies and setup

Because this library depends on `libHalide.so` (available at `Halide/lib`), the path to this file must be added to `LD_LIBRARY_PATH`. To import this library in Python, you must also add `Halide/lib/python3/site-packages` and `Halide/tools` to the `PYTHONPATH` environment variable.

6.2 Defining a Halide pipeline in Python

A Halide pipeline written in Python follows the same ahead-of-time compilation approach described in ([Ahead-of-time compilation](#)). However, we do not use generators ([Generators](#)).

The following example shows a 3x3 dilate filter written as a Halide pipeline in Python:

```
1 import halide as hl
2 input_img = hl.ImageParam(hl.UInt(8), 2)
3 x, y, yo = hl.Var("x"), hl.Var("y"), hl.Var("yo")
4 bounded_input, max_y, output = hl.Func("bounded_input"), hl.Func("max_y
  ↳"), hl.Func("output")
5
6 # Algorithm
7 height = input_img.height()
8 width = input_img.dim(0).extent()
9 x_min = input_img.dim(0).min()
10
11 bounded_input[x, y] = hl.BoundaryConditions.repeat_edge(
12     input_img, [(x_min, width), (hl.Expr(), hl.Expr())])[x, y]
13
14 max_y[x, y] = hl.max(bounded_input[x, hl.clamp(y - 1, 0, height - 1)],
15     bounded_input[x, hl.clamp(y, 0, height - 1)],
```

(continues on next page)

(continued from previous page)

```

16         bounded_input[x, hl.clamp(y + 1, 0, height - 1)])
17
18 output[x, y] = hl.max(max_y[x - 1, y], max_y[x, y], max_y[x + 1, y])
19
20 # Schedule
21 output.hexagon()
22
23 # Compile the pipeline to an executable
24 target = hl.Target()
25 target.os = hl.TargetOS.Android
26 target.arch = hl.TargetArch.ARM
27 target.bits = 64
28 target.set_features([hl.TargetFeature.HVX])
29 args = [input_img]
30 output.compile_to_file("dilate3x3", args, "dilate3x3", target)

```

The algorithm and schedule are written in a similar way to the algorithm and schedule in C++. However, as seen in this example, `Func` is accessed using `[]` in Python instead of `()` in C++. For example, `max_y[x, y]` in Python instead of `max_y(x, y)` in C++.

For cross-compilation, define a `target` to compile the pipeline and a list of inputs (`ImageParam` or `Param`), and pass it to `Func.compile_to_file()` with the name of the pipeline and output file. Running this Python program generates a host object file of the Halide pipeline, which is same as the output produced by AOT compilation of Halide in C++.

Note: This document is simply an introduction to getting started writing Halide pipelines in Python. More resources can be found at <https://halide-lang.org/docs> and https://github.com/halide/Halide/tree/master/python_bindings/tutorial.

6.2.1 Generators in Python

Generators (*Generators*), which are used to define a Halide pipeline in C++, are not available when writing a Halide pipeline in Python. However, we provide a utility called `pygen`, which gives you some of the conveniences of generators.

The following example shows how the dilate filter is modified to use `pygen`.

```

1 import Halide as hl
2 import pygen
3
4 params = pygen.get_params()
5 target = params["target"]
6
7 input_img = hl.ImageParam(hl.UInt(8), 2)
8 x, y, yo = hl.Var("x"), hl.Var("y"), hl.Var("yo")
9 bounded_input, max_y, output = hl.Func("bounded_input"), hl.Func("max_y
  ↳"), hl.Func("output")

```

(continues on next page)

(continued from previous page)

```

10
11 # Algorithm
12 height = input_img.height()
13 width = input_img.dim(0).extent()
14 x_min = input_img.dim(0).min()
15
16 bounded_input[x, y] = hl.BoundaryConditions.repeat_edge(
17     input_img, [(x_min, width), (hl.Expr(), hl.Expr())])[x, y]
18
19 max_y[x, y] = hl.max(bounded_input[x, hl.clamp(y - 1, 0, height - 1)],
20                     bounded_input[x, hl.clamp(y, 0, height - 1)],
21                     bounded_input[x, hl.clamp(y + 1, 0, height - 1)])
22
23 output[x, y] = hl.max(max_y[x - 1, y], max_y[x, y], max_y[x + 1, y])
24
25 # Schedule
26 if target.has_feature(hl.TargetFeature.HVX):
27     output.hexagon()
28
29 # Compile
30 pygen.compile(output, [input_img])

```

Note: `pygen` is located at `Halide/tools`. To import this utility, add this location to the `PYTHONPATH` environment variable.

The `pygen` utility enables the pipeline to be used as a command-line program that is similar to a C++ generator, which takes target and other user-defined parameters as arguments and generates object code for the host. This utility supports flags exposed by a C++ generator with their typical meanings. For example:

- Use `-o` to specify the location of the output files.
- Use `-e` to specify the type of output files to be generated.

The `pygen.get_params()` method returns a dictionary of parameters as received from the command line.

The `pygen.add_params()` method can be used to add user-defined parameters, as shown in the following example.

```
pygen.add_param("use_parallel_sched", bool, True)
```

This method adds a Halide parameter called `use_parallel_sched`, which is a Boolean with a default value of `True`. The target string supplied at the command line can be queried as one of the parameters to make target-dependent choices in the pipeline.

The `pygen.compile()` method takes the output `Func` to be compiled and a list of inputs to the pipeline as arguments, and it generates an object file for the target obtained from command line.

Run this Python program as follows:

```
dilate3x3.py -g dilate3x3 -o ./dilate3x3.o -e o,h -f dilate3x3_
↳target=arm-64-android-hvx
```

6.3 Differences between Python bindings and the C++ API

The Python bindings attempt to mimic the Halide C++ API as closely as possible, with some differences where the C++ idiom is either inappropriate or impossible.

- Most APIs that take a variadic argument list of integers in C++ take an explicit list in Python. For example, the typical version of the `Buffer` constructor in C++ offers variadic and list versions:

```
Buffer<>(Type t, int extent_dim_0, int extent_dim_1, ...,
        extent_dim_N, string name = "")
```

```
Buffer<>(Type t, vector<int> extents, string name = "")
```

In Python, only the second variant is provided.

- `Func` and `Buffer` access is performed using `[]` rather than `()`.
- Some classes in the Halide API are not provided because they are wrapped with standard Python idioms:
 - `Halide::Tuple` does not exist in the Python bindings; an ordinary Python tuple of `Halide::Expr` is used instead.
 - `Halide::Realization` does not exist in the Python bindings; an ordinary Python tuple of `Halide::Buffer` is used instead.
 - `Halide::Error` and others do not exist; standard Python error handling is used instead.
- Static and instance method overloads with the same name in the same class are not allowed. Thus, some convenience methods are missing from `Halide::Var`.
- Templated types (notably `Halide::Buffer<>` and `Halide::Param<>`) are not provided, for obvious reasons. Only the equivalents of `Halide::Buffer<void>` and `Halide::Param<void>` are supported.
- Only symbols in the `Halide` namespace are supported. Classes and methods that involve using the `Halide::Internal` namespace are not provided.
- The functions in `Halide::ConciseCasts` are present in the top-level `Halide` module in Python, rather than a submodule. For example, use `hl.i8_sat()`, not `hl.ConciseCasts.i8_sat()`.
- No mechanism is provided for overriding any runtime functions from Python.
- No mechanism is provided for supporting `Func::define_extern`.
- `Buffer::for_each_value()` is hard to implement well in Python; it is omitted entirely for now.
- `Func::in` becomes `Func.in_` because `in` is a Python keyword.

6.3.1 Enhancements to the C++ API

The `Buffer` class supports the Python Buffer Protocol (<https://www.python.org/dev/peps/pep-3118/>), and thus it is easily and cheaply converted to and from other compatible objects (such as NumPy's `ndarray`), with storage being shared.

6.4 Halide Python examples

Example Halide pipelines written in Python can be found at `Examples/offload/apps/hexagon_benchmarks` along with their C++ counterparts.

Before running the examples, ensure that correct Python version is installed (currently supported Python 3.8.x). Also, if multiple Python v3 versions are present, set the `PYTHON` environment variable to point to the location of the Python version to be used.

Set up the environment as described in ([Setup](#)). Then, set `USE_PYTHON` to 1 for running the Python examples:

```
USE_PYTHON=1 make run
```

7 Halide examples

The Halide directory provides some sample pipelines that demonstrate the use of Halide.

The examples are structured in the following manner

```
Halide/
├── Examples/
│   ├── include
│   │   └── test_report.h
│   ├── Makefile
│   ├── offload
│   │   ├── apps
│   │   ├── Makefile
│   │   └── Makefile.common
│   ├── setup-env.sh
│   ├── standalone
│   │   ├── device
│   │   ├── Makefile
│   │   └── simulator
│   └── unsetup-env.sh
```

Halide/Examples/offload has pipelines that demonstrate the use of Device and Simulator Offload modes.

Halide/Examples/standalone has pipelines that demonstrate the use of Device and Simulator Standalone modes.

The following sections provide steps for running examples on Linux. For Windows usage, see Halide/Examples.txt.

7.1 Setup

First, set up the environment for the examples by editing Halide/Examples/setup-env.sh. In particular, set up the path to HEXAGON_SDK_ROOT, and then set up the environment as follows:

```
$> source setup-env.sh
```

This process sets up a variable called RUN_DEVICE_ID in your environment. This variable is a list of the DEVICE IDs of all the devices connected to the host. All device-based examples (Device Standalone and Device Offload modes) will now run on all the devices in RUN_DEVICE_ID.

To limit the list to a subset of all the devices connected to your machine, reset RUN_DEVICE_ID to the DEVICE ID of a specific device.

To unset the environment that was set up via `setup-env.sh`, use `unsetup-env.sh`.

7.1.1 Device setup

To run examples in Device Standalone or Device Offload mode, set up Halide runtime libraries on all the devices in `RUN_DEVICE_ID`. Run the following commands:

```
$> adb push $HALIDE_ROOT/lib/arm-32-android/libhalide_hexagon_host.so /
    ↪system/lib/
$> adb push $HALIDE_ROOT/lib/arm-64-android/libhalide_hexagon_host.so /
    ↪system/lib64/
$> adb shell mkdir -p /system/lib/rfsa/adsp/
$> adb push $HALIDE_ROOT/lib/v65/libhalide_hexagon_remote_skel.so /
    ↪system/lib/rfsa/adsp/

$> adb push $HALIDE_ROOT/lib/arm-32-android/libhalide_hexagon_host.so /
    ↪vendor/lib/
$> adb push $HALIDE_ROOT/lib/arm-64-android/libhalide_hexagon_host.so /
    ↪vendor/lib64/
$> adb shell mkdir -p /vendor/lib/rfsa/adsp/
$> adb push $HALIDE_ROOT/lib/v65/libhalide_hexagon_remote_skel.so /
    ↪vendor/lib/rfsa/adsp/

$> sysroot=$HEXAGON_SDK_ROOT/tools/android-ndk-r25c/toolchains/llvm/
    ↪prebuilt/linux-x86_64/sysroot
$> adb push $sysroot/usr/lib/arm-linux-androideabi/libc++_shared.so /
    ↪system/lib/
$> adb push $sysroot/usr/lib/aarch64-linux-android/libc++_shared.so /
    ↪system/lib64/
$> adb push $sysroot/usr/lib/arm-linux-androideabi/libc++_shared.so /
    ↪vendor/lib/
$> adb push $sysroot/usr/lib/aarch64-linux-android/libc++_shared.so /
    ↪vendor/lib64/
```

Note: For LE devices, the partition `/vendor` should be replaced by `/usr` in the commands above.

Further, `libhalide_hexagon_host.so` should be pushed from `$HALIDE_ROOT/lib/arm-32-linux` for 32-bit targets and from `$HALIDE_ROOT/lib/arm-64-linux` for 64-bit targets.

7.2 Run examples

All examples can now be run from the command line by executing the following command from the `Examples` directory.

```
$> make all
```

Note: To demonstrate support for LE targets, `offload/apps/hexagon_benchmarks` and

`standalone/device/apps/conv3x3/` have been modified to compile for LE targets using Linaro toolset. To run examples for LE targets, export `OS_FLAVOR=linux` before running the above command. The target string for running generators for device offload examples on LE devices is `arm-64-linux-hvx`.

7.2.1 Run only a single execution mode

Instead of running all the examples, you can run each execution mode individually as follows:

```
$> make <mode>
```

In this case, `mode` can be `sim-standalone`, `sim-offload`, `dev-standalone`, or `dev-offload`.

8 Indices and tables

- `genindex`
- `modindex`
- `search`

LEGAL INFORMATION

Your access to and use of this material, along with any documents, software, specifications, reference board files, drawings, diagnostics and other information contained herein (collectively this “Material”), is subject to your (including the corporation or other legal entity you represent, collectively “You” or “Your”) acceptance of the terms and conditions (“Terms of Use”) set forth below. If You do not agree to these Terms of Use, you may not use this Material and shall immediately destroy any copy thereof.

1) Legal Notice.

This Material is being made available to You solely for Your internal use with those products and service offerings of Qualcomm Technologies, Inc. (“Qualcomm Technologies”), its affiliates and/or licensors described in this Material, and shall not be used for any other purposes. If this Material is marked as “Qualcomm Internal Use Only”, no license is granted to You herein, and You must immediately (a) destroy or return this Material to Qualcomm Technologies, and (b) report Your receipt of this Material to qualcomm.support@qti.qualcomm.com. This Material may not be altered, edited, or modified in any way without Qualcomm Technologies’ prior written approval, nor may it be used for any machine learning or artificial intelligence development purpose which results, whether directly or indirectly, in the creation or development of an automated device, program, tool, algorithm, process, methodology, product and/or other output. Unauthorized use or disclosure of this Material or the information contained herein is strictly prohibited, and You agree to indemnify Qualcomm Technologies, its affiliates and licensors for any damages or losses suffered by Qualcomm Technologies, its affiliates and/or licensors for any such unauthorized uses or disclosures of this Material, in whole or part.

Qualcomm Technologies, its affiliates and/or licensors retain all rights and ownership in and to this Material. No license to any trademark, patent, copyright, mask work protection right or any other intellectual property right is either granted or implied by this Material or any information disclosed herein, including, but not limited to, any license to make, use, import or sell any product, service or technology offering embodying any of the information in this Material.

THIS MATERIAL IS BEING PROVIDED “AS IS” WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESSED, IMPLIED, STATUTORY OR OTHERWISE. TO THE MAXIMUM EXTENT PERMITTED BY LAW, QUALCOMM TECHNOLOGIES, ITS AFFILIATES AND/OR LICENSORS SPECIFICALLY DISCLAIM ALL WARRANTIES OF TITLE, MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, SATISFACTORY QUALITY, COMPLETENESS OR ACCURACY, AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MOREOVER, NEITHER QUALCOMM TECHNOLOGIES, NOR ANY OF ITS AFFILIATES AND/OR LICENSORS, SHALL BE LIABLE TO YOU OR ANY THIRD PARTY FOR ANY EXPENSES, LOSSES, USE, OR ACTIONS HOWSOEVER INCURRED OR UNDERTAKEN BY YOU IN RELIANCE ON THIS MATERIAL.

Certain product kits, tools and other items referenced in this Material may require You to accept additional terms and conditions before accessing or using those items.

Technical data specified in this Material may be subject to U.S. and other applicable export control laws. Transmission contrary to U.S. and any other applicable law is strictly prohibited.

Nothing in this Material is an offer to sell any of the components or devices referenced herein.

This Material is subject to change without further notification.

In the event of a conflict between these Terms of Use and the *Website Terms of Use* on www.qualcomm.com or the *Qualcomm Privacy Policy* referenced on www.qualcomm.com, these Terms of Use will control. In the event of a conflict between these Terms of Use and any other agreement (written or click-through, including, without limitation any non-disclosure agreement) executed by You and Qualcomm Technologies or a Qualcomm Technologies affiliate and/or licensor with respect to Your access to and use of this Material, the other agreement will control.

These Terms of Use shall be governed by and construed and enforced in accordance with the laws of the State of California, excluding the U.N. Convention on International Sale of Goods, without regard to conflict of laws principles. Any dispute, claim or controversy arising out of or relating to these Terms of Use, or the breach or validity hereof, shall be adjudicated only by a court of competent jurisdiction in the county of San Diego, State of California, and You hereby consent to the personal jurisdiction of such courts for that purpose.

2) Trademark and Product Attribution Statements.

Qualcomm is a trademark or registered trademark of Qualcomm Incorporated. Arm is a registered trademark of Arm Limited (or its subsidiaries) in the U.S. and/or elsewhere. The Bluetooth® word mark is a registered trademark owned by Bluetooth SIG, Inc. Other product and brand names referenced in this Material may be trademarks or registered trademarks of their respective owners.

Snapdragon and Qualcomm branded products referenced in this Material are products of Qualcomm Technologies, Inc. and/or its subsidiaries. Qualcomm patented technologies are licensed by Qualcomm Incorporated.